

文档编号	TY-SPDD-20231030A
版本	A
页数	141 页

科学计算与系统建模仿真平台 开放系统架构

*Science & Engineering Computing
Open System Architecture*

苏州同元软控信息技术有限公司版权所有，未经授权不得转载或以其它方式使用；
版权所有，侵权必究。

科学计算与系统建模仿真项目组

编制	刘奇、郭俊峰、田显钊、 刘尚、张新城、王天飞	生效日期	2023 年 10 月 30 日
审核	郭俊峰	批准	刘奇

文件变更摘要

日期	版本	变更说明	修订	审核	批准
2022.9.7	V0.1	北向接口场景分析、初步定义接口分层并示例	张新城	郭俊峰	刘奇
2022.9.14	V0.2	内核层 API 是原有的南向接口 服务层 API 定义函数库/模型库/APP 的接口定义和开发规范 应用层 API 定义外部系统集成接口	田显钊	郭俊峰	刘奇
2022.9.21	V0.3	进一步明确了多层接口的内涵和适用场景 补充完善了各层接口的规范列表	张新城	郭俊峰	刘奇
2022.9.27	V0.4	将 4 个内核级 API 场景整合为两个 服务级 API 扩充为 4 个场景	田显钊	郭俊峰	刘奇
2022.10.5	V0.5	确定服务级 API 标准化 C 接口思路	张新城	郭俊峰	刘奇
2023.5.16	V0.6	1.App 全部改为 APP 2.将其他平台分层架构详细介绍的合并进来 3.7.1 章节修改为与高校版本一致，且删除内核章节 4.7.2 章节按照新格式和内容进行调整 5.对系统仿真图进行调整	王天飞	郭俊峰	刘奇
2023.7.20	V0.7	系统建模仿真 API 和 APP 根据评审修改合并	王天飞	郭俊峰	刘奇
2023.10.30	V1.0	整理内核层科学计算算法 API	刘尚	郭俊峰	刘奇

苏州同元软控信息技术有限公司
版权所有，侵权必究。

目 录

1. 前言	1
2. 范围	1
3. 术语、定义和缩略语	1
3.1 术语和定义.....	1
3.2 缩略语.....	2
4. 引用文件	2
5. 平台分层架构	2
5.1 内核层.....	4
5.2 平台层.....	5
5.3 应用层.....	7
6. 开放架构和接口使用场景	7
6.1 内核级 API 使用场景	7
6.1.1 场景 1：扩展或替换基础数学算法.....	7
6.1.2 场景 2：扩展或替换模型求解算法.....	7
6.1.3 场景 3：扩展或替换科学计算与系统建模求解内核.....	8
6.2 平台级 API 使用场景	8
6.2.1 场景 1：通过平台 API 开发 APP	8
6.2.2 场景 2：通过平台 API 定制平台	8
6.2.3 场景 3：通过平台 API 集成平台	8
6.3 资源开发规范使用场景.....	8
6.3.1 场景 1：基于规范开发函数库.....	8
6.3.2 场景 2：基于规范开发模型库.....	9
6.3.3 场景 3：基于规范开发 APP	9
7. 开放架构和接口详细说明	9
7.1 内核层.....	9
7.1.1 科学计算算法.....	9
7.1.2 模型求解算法.....	42
7.2 平台层.....	54

7.2.1 科学计算环境.....	54
7.2.2 系统建模仿真环境.....	93
7.3 应用层.....	100
7.3.1 函数库开发规范.....	100
7.3.2 模型库开发规范.....	112
7.3.3 APP 开发规范.....	136
附录 1：内核级开放接口	141
附录 1.1 科学计算环境数学算法替换用户手册.....	141
附录 1.2 系统建模仿真环境内核算法替换用户手册.....	141
附录 2：平台层开放接口	141
附录 2.1 科学计算环境平台层 API 详细说明	141
附录 2.2 系统建模仿真环境平台层 API 详细说明	141
附录 3：应用资源开发手册	141
附录 3.1 科学计算环境 APP 开发用户手册.....	141
附录 3.2 系统建模仿真环境 APP 开发用户手册	141
附录 3.3 模型库和函数库开发规范.....	141

1. 前言

科学计算与系统建模仿真平台是当前工业软件中行业应用最广、生态影响最大、依赖粘度最强的工业软件，通过计算数学的软件化与工程知识的模型化，支撑用户在开展科学计算、数据处理、产品设计、仿真分析等工作中实现能力、效率与质量的有效提升，在以模型-数据为中心的数字化时代更有望成为数字化核心能力平台。

科学计算与系统建模仿真开放系统架构，定义了一套面向云环境的科学计算与系统建模仿真平台架构和接口标准化方案，支持开发者基于统一的接口规范，以一致的方式开发函数库、模型库和 APP，实现平台共建，丰富应用生态。

2. 范围

本规范提出了科学计算与系统建模仿真开放系统架构，规定了科学计算与系统建模仿真平台各层级对外开放的标准接口协议、基础接口功能、接口调用方式等内容，给出基于开放接口开发函数、模型、APP 等资源的示例。

本规范可供南向数学算法/求解器开发者、北向函数/模型/APP 开发者、平台集成服务商、测评机构等单位参考使用。

3. 术语、定义和缩略语

3.1 术语和定义

- 科学计算环境：针对科学与工程中遇到的矩阵运算、微分方程求解、数据分析、信号处理、控制设计与优化等问题，提供基于高性能科学计算语言的脚本开发、调试、运行及后处理可视化环境。
- 系统建模仿真环境：针对工程物理系统中机械、电气、控制、流体等单学科或多学科耦合的系统级分析及验证问题，提供基于多领域统一建模规范的图形化建模、仿真、求解及后处理一体化环境。
- 函数库：函数库是科学计算环境中的各类基础函数集合，包括：基础语言、基础数学、基础信号、DSP、通信和控制系统等。
- 模型库：模型库是系统建模仿真环境中的各领域模型集合，包括

Modelica 标准库（涵盖机电液控热领域）、基础信号、DSP、通信和控制系统模型库等。

- **APPs:** APP 是带交互界面的应用程序，提供面向特定场景的专业应用，如控制系统分析应用。APP 通常依赖函数库或模型库，GUI 实现交互入口，专业算法调用底层函数。

3.2 缩略语

下列缩略语适用于本文件。

- FMI 功能样机接口 Functional Mock-up Interface
- BLAS 基础线性代数子程序库 Basic Linear Algebra Subprograms
- LAPACK 线性代数软件包 Linear Algebra PACKage
- FFTW 快速计算离散傅里叶变换的标准 C 语言程序集 the Faster Fourier Transform in the West
- DSP 数字信号处理 Digital Signal Processing

4. 引用文件

- FMI 规范: <https://fmi-standard.org/docs/3.0/>
- BLAS 规范: <https://netlib.org/blas>
- LAPACK 规范: <http://www.netlib.org/lapack>
- FFTW: <https://www.fftw.org/>

5. 平台分层架构

贯彻从底层算法到上层应用的完全开放策略，科学计算与系统建模仿真开放系统架构在最高抽象级别上划分为三个层次：内核层、平台层和应用层。

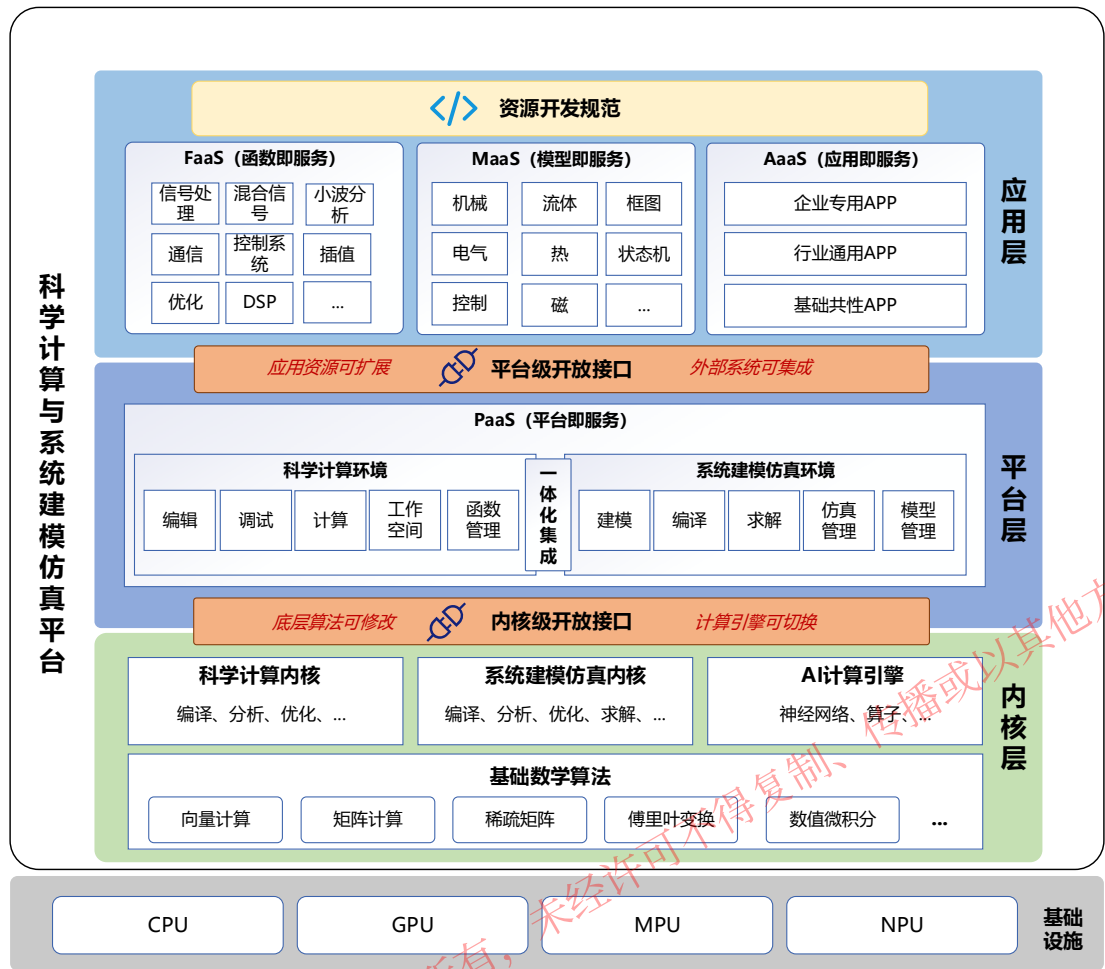


图 5-1 分层架构

(1). **内核层**：内核层是科学计算与系统建模仿真平台的最底层，负责算法函数和仿真模型的编译运行，主要由基础数学算法库、模型求解算法库及科学计算与系统建模仿真内核组成。

(2). **平台层**：平台层是科学计算与系统建模仿真的集成开发环境，为函数、模型、APP 等资源提供开发、调试、集成、测试、部署等全生命周期支持，主要由科学计算环境和系统建模仿真环境组成。

(3). **应用层**：应用层由函数库、模型库、APP 等应用资源组成，以服务形式支持用户解决基础共性、行业通用、企业专用问题，应用资源基于平台层提供的开放接口，采用统一的资源开发规范开发。

内核层和平台层提供了开放、标准接口供开发者和外部系统调用，应用层则定义了一套开发规范，支持函数、模型和 APP 资源的开发。

(1). **内核级开发接口**：支持底层算法可替换，开发者可设置、替换科学计算与系统建模仿真平台底层数值算法、数学包、仿真求解算法、求解器等。

(2). **平台级开发接口**：支持应用资源可扩展，开发者可基于平台级开放接口，采用多语言高效开发函数库、模型库、APP 等应用资源；支持外部系统可集成，

第三方系统可以松耦合方式，整体集成科学计算与系统建模仿真平台。

(3). 资源开发规范：定义了一套开发规范，支持函数、模型和 APP 资源的开发。

5.1 内核层

内核层是科学计算与系统建模仿真平台的最底层，负责算法函数和仿真模型的编译运行，主要由基础数学算法库、模型求解算法库及科学计算与系统建模仿真内核组成。内核层定义了一套接口规范，包括底层算法接口和上层内核接口。底层算法接口对科学计算算法、模型求解算法的接口进行规约，符合算法接口标准即可接入科学计算与系统建模仿真平台；上层内核接口提供一组内核原子 API，支持模型编译、模型分析、模型求解、代码生成、仿真结果读写等操作。

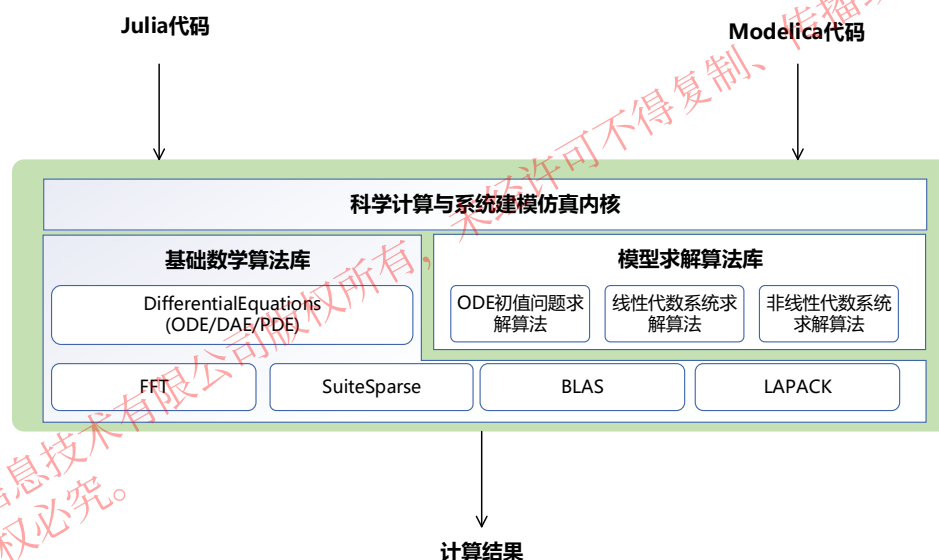


图 5-2 内核处理流程

对于科学计算与系统建模仿真内核分为三个层次，分别是算法层、主控层和应用层。

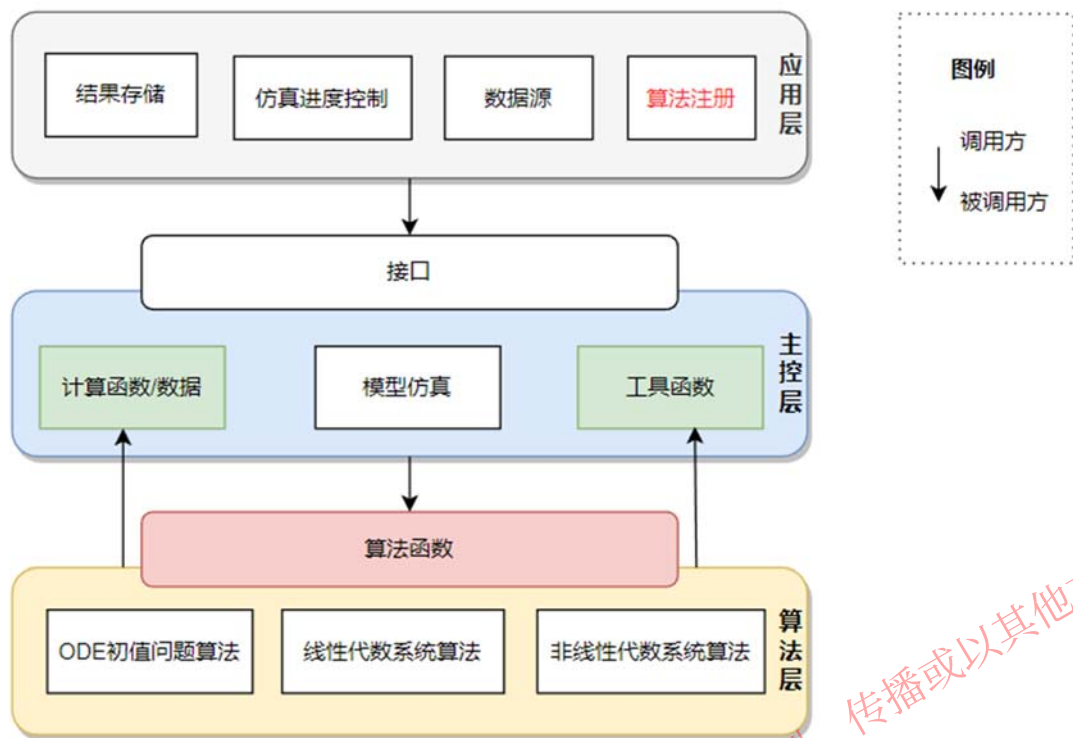


图 5-3 模型求解器架构

算法层实现 ODE 初值问题、线性代数系统、非线性代数系统的求解算法，分别对 ODE 初值问题、线性代数系统、非线性代数系统进行解算。扩展模型求解算法就在本层实现。

主控层是模型求解的核心，依据平台的求解策略，调用算法层的算法函数对模型进行仿真。同时，主控层提供模型方程系统中的 ODE 初值问题的计算函数、非线性代数系统的计算函数、线性代数系统的数据供算法层使用，也提供堆内存分配、日志输出等工具函数由算法层调用。

应用层调用主控层接口进行模型仿真，并提供模型仿真结果的保存、仿真进度控制、数据源等功能。算法层的求解算法在应用层通过主控层的接口进行注册以与求解器集成。

计算函数、工具函数、算法函数都是以回调函数的形式提供。回调函数是 C 语言函数，符合规范 C99。

5.2 平台层

平台层是科学计算与系统建模仿真的集成开发环境，为函数、模型、APP 等应用提供开发、调试、集成、测试、部署等全生命周期支持，主要由科学计算环境和系统建模仿真环境组成，科学计算环境主要用于函数的开发，系统建模仿真环境主要用于模型的开发，平台层提供开放、规范的接口用于 APP 的开

发。

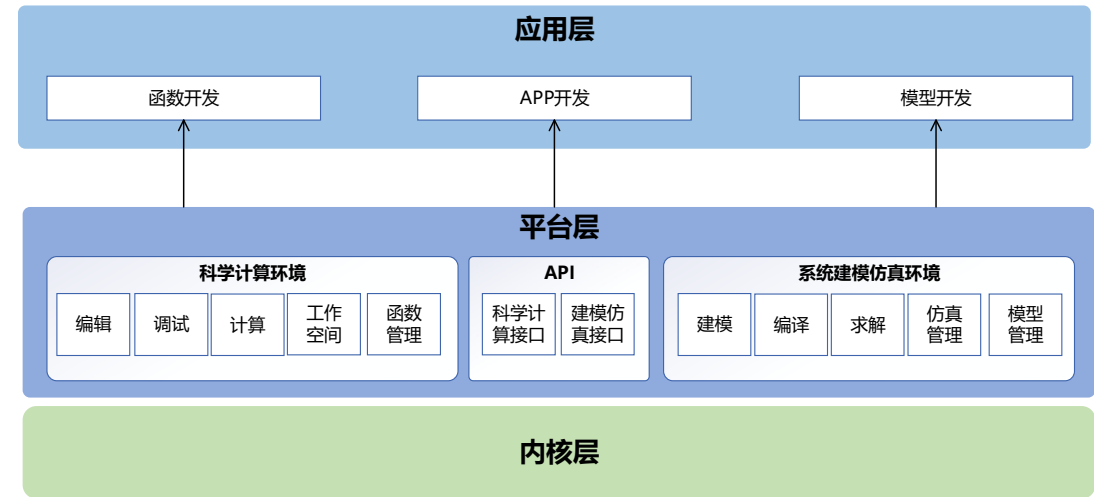


图 5-4 平台层框架

科学计算环境是同元面向科学计算的平台，通过 Julia 语言支持多范式统一编程，简约与性能兼顾。提供通用编程、科学计算、数据科学、机器学习、信号处理、通讯仿真、并行计算等功能，并可使用内置的图形进行数据可视化。科学计算环境实现了与工程建模仿真环境 MWORKS.Sysplorer 的双向融合，形成新一代科学计算与与工程建模仿真的一体化基础平台，满足各行业在设计、建模、仿真、分析、优化方面的业务需求。

系统建模仿真环境是面向多领域工业产品的系统级综合设计与仿真验证环境，完全支持多领域统一建模规范 Modelica，遵循现实中拓扑结构的层次化建模方式，支撑基于模型的系统工程应用。科学计算环境基于国际多领域统一建模规范 Modelica，支持工业设计知识的模型化表达和模块化封装，支持多方案优选及设计参数优化，以知识可重用、系统可重构方式，为工业企业的设计知识积累与产品创新设计提供了有效的技术支撑，对及早发现产品设计缺陷、快速验证设计方案、全面优化产品性能、有效减少物理验证次数等具有重要价值。

API 是指 MWORKS 内核层和平台层提供的供开发者和外部系统调用的开放、标准接口。API 包括科学计算算法环境 API 和系统建模仿真 API 两部分，每层 API 皆可独立使用，以满足不同层次的应用需求：

科学计算环境 API：包括基础 API 用于基本运算、数学 API 用于初等数学线性代数等基本数学方程、图形 API 用于生成二维和三维图等；

系统建模仿真环境 API：包括内核级 API 用于模型文件操作、模型参数操作、模型属性获取、编译仿真等，图形组件级 API 用于模型视图、模型浏览器、组件参数面板等图形接口。

5.3 应用层

应用层定义了一套函数库、模型库和 APP 的开发规范，支持以规范的方式开发科学计算与系统建模仿真平台资源，提供资源管理接口，支持函数库、模型库和 APP 的装载、驱动和卸载。

函数库开发规范：科学计算环境中的函数库由一系列函数组成，这些函数是科学计算环境的主要元素。除了科学计算环境中本身已提供的内置函数，用户也可根据函数库开发规范开发一套新的函数，并以函数库的方式集成到科学计算环境中，从而扩展科学计算环境的功能。

模型库开发规范：系统建模仿真环境中的模型库由一系列模块组成，模块是构建系统模型的主要元素。系统建模环境内置了库提供了多学科基本模块。并支持用户开发新的模块，并将其以模型库的方式集成到系统建模仿真环境中，从而扩展系统建模仿真环境的功能。

APP 开发说明：科学计算环境和系统建模仿真环境均支持用户开发一个带交互界面的应用程序以满足用户构建面向特点应用场景的 APP 开发。因此，定义了一套 APP 的开发规范，描述了 APP 的开发、运行及管理机制来规范用户开发 APP 的过程。

6. 开放架构和接口使用场景

6.1 内核级 API 使用场景

6.1.1 场景 1：扩展或替换基础数学算法

在科学计算与系统建模仿真平台上进行二次开发来扩展或替换平台的科学计算算法包，包括最底层的 BLAS、LAPACK 等基础算法包，以及上层的符号计算、曲线拟合等应用层数学包的替换和拓展。

6.1.2 场景 2：扩展或替换模型求解算法

在科学计算与系统建模仿真平台上进行二次开发来扩展平台的模型求解算法，包括初值问题（initial value problems——IVP）算法、线性问题求解算法、非线性问题求解算法。

6.1.3 场景 3：扩展或替换科学计算与系统建模求解内核

开放式架构支持采用自研的求解内核替换科学计算与系统建模仿真平台的原生内核，包括编译器、分析器、代码生成器等，为不同仿真问题提供更适合的求解策略和计算调度策略。

注：本场景将在开放系统架构规范后续版本中支持。

6.2 平台级 API 使用场景

6.2.1 场景 1：通过平台 API 开发 APP

提供一套标准科学计算与系统建模仿真平台 API，支持平台的界面、业务逻辑、数据等不同层次接口调用，支持 APP 的扩展开发和集成。

6.2.2 场景 2：通过平台 API 定制平台

提供一套标准科学计算与系统建模仿真平台 API，支持平台的界面、业务逻辑、数据等不同层次接口调用，支持对平台的定制和集成。

6.2.3 场景 3：通过平台 API 集成平台

外部系统通过命令行调用科学计算与系统建模仿真平台的计算能力，包括科学计算能力和建模仿真计算能力。外部系统完全独立于科学计算与系统建模仿真平台之外。

6.3 资源开发规范使用场景

6.3.1 场景 1：基于规范开发函数库

定义了一套函数库开发规范，描述了函数库开发、运行及管理机制，规范用户使用平台主语言（Julia）及外部语言（Python/C/C++）开发函数库的过程。基于该规范开发的函数（库）能在本平台导入，纳入平台管理，开展科学计算编程，执行计算，查看计算结果。

6.3.2 场景 2：基于规范开发模型库

定义了一套模型库的开发规范，描述了模型库开发、运行及管理机制，规范用户使用平台主语言（Modelica）及外部语言（Julia/Python/C/C++）开发模型库的过程。基于该规范开发的模型（库）能在本平台导入，纳入平台管理，开展可视化系统建模，执行仿真求解，查看仿真结果。

6.3.3 场景 3：基于规范开发 APP

定义了一套 APP 的开发规范，描述了 APP 开发、运行及管理机制，规范用户开发 APP 的过程。在本系统可视化应用程序集成开发环境中开发的 APP，会自动遵循 APP 接口规范，内置相关的 API 接口，自动支持 APP 的加载、驱动、卸载，并可以集成到 APP 商店中管理。外部 APP，按照 APP 开发接口规范，提供 APP 注册、APP 监听、APP 运行、APP 终止等 API 接口，即能在本平台加载、驱动、卸载，并可以集成到 APP 商店中管理。

7. 开放架构和接口详细说明

7.1 内核层

内核层定义了一套接口规范，包括底层算法接口和上层内核接口。底层算法接口对科学计算算法、模型求解算法的接口进行规约，符合算法接口标准即可接入科学计算与系统建模仿真平台；上层内核接口提供一组内核原子 API，支持模型编译、模型分析、模型求解、代码生成、仿真结果读写等操作。

7.1.1 科学计算算法

科学计算与系统建模仿真平台内置了大量学科计算算法包，从线性代数、插值、微积分到傅里叶变换等。平台提供了科学计算算法扩展功能，支持从底层数值算法到整体数学的替换和扩充功能。

科学计算函数库分为三个层次，数学应用层、数学层内核、算法内核。利用数学应用层通过数学层内核调用算法内核，如图所示：

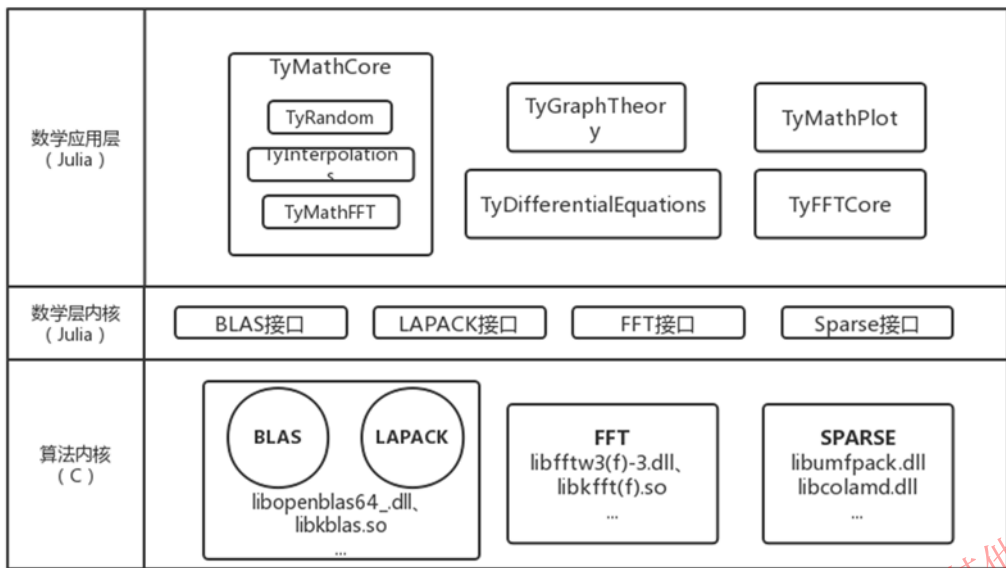


图 7-1 科学计算函数库架构

数学应用层包含直接对外暴露、用户可直接调用使用的函数集，其大致分类为初等数学、线性代数、插值、微积分、傅里叶变换、稀疏矩阵等板块函数。

数学层内核针对 BLAS、LAPACK、FFT、Sparse 算法封装内核 API，提供算法调用接口和算法内核替换接口。

算法内核包含 BLAS/LAPACK 等有关矢量、矩阵乘法、矩阵分解、线性方程组求解等底层核心算法工具集，作为基础数学工具箱内核，支撑整个基础数学算法。

7.1.1.1 BLAS 接口

BLAS（基本线性代数子程序）提供了一系列基本线性代数运算函数的标准接口，包括矢量线性组合、矩阵乘以矢量、矩阵乘以矩阵等功能。BLAS 已被广泛的应用于工业界和科学计算，成为业界标准。为提高性能，各软硬件厂商针对其产品对 BLAS 接口实现进行了高度优化。替代本平台科学计算的 BLAS 算法库，应符合 BLAS 标准的 C 接口形式。

BLAS 针对不同的数据类型定义了相应的函数接口，并通过函数名前缀字母进行区分，其中 S 表示单精度浮点型、D 表示双精度浮点型、C 表示单精度复数、Z 表示双精度复数。

7.1.1.1.1 Level-1 BLAS

Level-1 BLAS 提供了 15 种函数接口族。

表 7-1 Level-1 BLAS 函数接口族

函数族	简介
?rotg	生成 Givens 变换矩阵 具体分为以下两个函数： cblas_srotg:生成单精度浮点型 Givens 变换矩阵 cblas_drotg:生成双精度浮点型 Givens 变换矩阵
?rotmg	生成改进 Givens 变换矩阵 具体分为以下两个函数： cblas_srotmg:生成单精度浮点型改进 Givens 变换矩阵 cblas_drotmg:生成双精度浮点型改进 Givens 变换矩阵
?rot	执行 Givens 变换 $x_i = c * x_i + sy_i$; $y_i = c * y_i - sx_i$; x 和 y 是含有 n 个元素的向量, c 是旋转角的余弦, s 是旋转角的正弦。具体分为以下四个函数： cblas_srot:单精度浮点型 Givens 变换 cblas_drot:双精度浮点型 Givens 变换 cblas_crot:单精度复数型 Givens 变换 cblas_zrot:双精度复数型 Givens 变换
?rotm	执行改进 Givens 变换 $x_i = h_{11} * x_i + h_{12}y_i$; $y_i = h_{21} * x_i + h_{22}y_i$; x 和 y 是含有 n 个元素的向量, h 是变换矩阵元素。具体分为以下两个函数： cblas_srotm:单精度浮点型改进 Givens 变换 cblas_drotm:双精度浮点型改进 Givens 变换
?swap	交换两个向量中的元素。 $x \leftrightarrow y$ 具体分为以下四个函数： cblas_sswap:单精度浮点型向量交换 cblas_dswap:双精度浮点型向量交换 cblas_cswap:单精度复数型向量交换 cblas_zswap:双精度复数型向量交换
?scal	系数与向量乘积 $x \leftarrow ax$, 其中 a 是乘法系数, x 是含有 n 个元素的向量。具体分为以下六个函数： cblas_sscal:单精度浮点型系数与向量乘积 cblas_dscal:双精度浮点型系数与向量乘积 cblas_cscal:单精度复数型系数与向量乘积 cblas_zscal:双精度复数型系数与向量乘积 cblas_csscal:单精度浮点型系数与单精度复数型向量乘积 cblas_zdscal:双精度浮点型系数与双精度复数型向量乘积
?copy	向量复制 $y \leftarrow x$, x 是源向量, y 是目标向量。 具体分为以下四个函数： cblas_scopy:单精度浮点型向量复制 cblas_dcopy:双精度浮点型向量复制 cblas_ccopy:单精度复数型向量复制 cblas_zcopy:双精度复数型向量复制

函数族	简介
?axpy	向量缩放后的乘积与另一向量的加和 $y \leftarrow ax + y$ a 是乘法系数，x 和 y 是长度为 n 的向量。 具体分为以下四个函数： cblas_saxpy:单精度浮点型向量缩放后的乘积加和 cblas_daxpy:双精度浮点型向量缩放后的乘积加和 cblas_caxpy:单精度复数型向量缩放后的乘积加和 cblas_zaxpy:双精度复数型向量缩放后的乘积加和
?dot	向量点积 $dot \leftarrow x^T y$ x 和 y 是含有 n 个元素的向量。 具体分为以下两个函数： cblas_sdot:单精度浮点型向量点积 cblas_ddot:双精度浮点型向量点积
?dotc	共轭向量与另一向量的点积 $dotc \leftarrow \sum_{i=1}^n conjg(x_i) * y_i$ x 和 y 是长度为 n 的复数向量。 具体分为以下两个函数： cblas_cdotc:单精度复数型向量点积 cblas_zdotc:双精度复数型向量点积
?dotu	复数向量的点积。 cblas_cdotu:单精度复数型向量点积 cblas_zdotu:双精度复数型向量点积
?sdot	向量点积（输入向量为单精度，以双精度计算） $dot \leftarrow x^T y$ 具体分为以下两个函数： cblas_sdsdot:输出单精度复数型向量点积 cblas_dsdot:输出双精度复数型向量点积
?nrm2	向量 2 范数 $nrm2 \leftarrow \ x\ _2$，x 是含有 n 个元素的向量。 具体分为以下四个函数： cblas_snrm2:单精度浮点型向量 2 范数 cblas_dnrm2:双精度浮点型向量 2 范数 cblas_scnrm2:单精度复数型向量 2 范数 cblas_dznrm2:双精度复数型向量 2 范数
?asum	向量 1 范数 $asum \leftarrow \ x\ _1$ 具体分为以下四个函数： cblas_sasum:单精度浮点型向量 1 范数 cblas_dasum:双精度浮点型向量 1 范数 cblas_scasum:单精度复数型向量 1 范数 cblas_dzasum:双精度复数型向量 1 范数
i?amax	实数向量返回最大绝对值的索引值，复数向量返回实部绝对值和虚部绝对值之和的最大值索引值。 具体分为以下四个函数： cblas_isamax:单精度浮点型 cblas_idamax:双精度浮点型 cblas_icamax:单精度复数型 cblas_izamax:双精度复数型

7.1.1.1.2 Level-2 BLAS

Level-2 BLAS 提供了 16 种函数接口。

表 7-2 Level-2 BLAS 函数接口

函数族	简介
?gemv	$y \leftarrow \alpha * A * x + \beta * y$ or $y \leftarrow \alpha * A^T * x + \beta * y$ A 为常规矩阵; 具体分为以下四个函数: cblas_sgemv: 单精度浮点型 cblas_dgemv: 双精度浮点型 cblas_cgemv: 单精度复数型 cblas_zgemv: 双精度复数型
?gbmv	$y \leftarrow \alpha * A * x + \beta * y$ or $y \leftarrow \alpha * A^T * x + \beta * y$ A 为带状矩阵; 具体分为以下四个函数: cblas_sgbmv: 单精度浮点型 cblas_dgbmv: 双精度浮点型 cblas_cgbmv: 单精度复数型 cblas_zgbmv: 双精度复数型
?symv	$y \leftarrow \alpha * A * x + \beta * y$ A 为对称矩阵; 具体分为以下两个函数: cblas_ssylv: 单精度浮点型 cblas_dsylv: 双精度浮点型
?sbmv	$y \leftarrow \alpha * A * x + \beta * y$ A 为带状对称矩阵; 具体分为以下两个函数: cblas_ssbmv: 单精度浮点型 cblas_dsbmv: 双精度浮点型
?spmv	$y \leftarrow \alpha * A * x + \beta * y$ A 为 packed 对称矩阵; 具体分为以下两个函数: cblas_sspmv: 单精度浮点型 cblas_dspmv: 双精度浮点型
?trmv	$x \leftarrow A * x$ or $x \leftarrow A^T * x$ A 为三角矩阵; 具体分为以下四个函数: cblas_strmv: 单精度浮点型 cblas_dtrmv: 双精度浮点型 cblas_ctrmv: 单精度复数型 cblas_ztrmv: 双精度复数型

函数族	简介
?tbmv	$x \leftarrow A * x$ or $x \leftarrow A^T * x$ A 为带状三角矩阵; 具体分为以下四个函数: cblas_stbmv: 单精度浮点型 cblas_dtbmv: 双精度浮点型 cblas_ctbmv: 单精度复数型 cblas_ztbmv: 双精度复数型
?tpmv	$x \leftarrow A * x$ or $x \leftarrow A^T * x$ A 为 packed 三角矩阵; 具体分为以下四个函数: cblas_stpmv: 单精度浮点型 cblas_dtpmv: 双精度浮点型 cblas_ctpmv: 单精度复数型 cblas_ztpmv: 双精度复数型
?trsv	$x \leftarrow A^{-1} * x$ or $x \leftarrow A^{-T} * x$ A 为三角矩阵; 具体分为以下四个函数: cblas_strsv: 单精度浮点型 cblas_dtrsv: 双精度浮点型 cblas_ctrsv: 单精度复数型 cblas_ztrsv: 双精度复数型
?tbsv	$x \leftarrow A^{-1} * x$ or $x \leftarrow A^{-T} * x$ A 为带状三角矩阵; 具体分为以下四个函数: cblas_stbsv: 单精度浮点型 cblas_dtbsv: 双精度浮点型 cblas_ctbsv: 单精度复数型 cblas_ztbsv: 双精度复数型
?tpsv	$x \leftarrow A^{-1} * x$ or $x \leftarrow A^{-T} * x$ A 为 packed 三角矩阵; 具体分为以下四个函数: cblas_stpsv: 单精度浮点型 cblas_dtpsv: 双精度浮点型 cblas_ctpsv: 单精度复数型 cblas_ztpsv: 双精度复数型
?ger	$A \leftarrow \alpha * x * y^T + A$ A 为常规矩阵; 具体分为以下两个函数: cblas_sger: 单精度浮点型 cblas_dger: 双精度浮点型

函数族	简介
?syr	$A \leftarrow \alpha * x * x^T + A$ A 为对称矩阵; 具体分为以下两个函数: cblas_ssyr: 单精度浮点型 cblas_dsyr: 双精度浮点型
?spr	$A \leftarrow \alpha * x * x^T + A$ A 为 packed 对称矩阵; 具体分为以下两个函数: cblas_sspr: 单精度浮点型 cblas_dspr: 双精度浮点型
?syr2	$A \leftarrow \alpha * x * y^T + \alpha * y * x^T + A$ A 为对称矩阵; 具体分为以下两个函数: cblas_ssyr2: 单精度浮点型 cblas_dsyr2: 双精度浮点型
?spr2	$A \leftarrow \alpha * x * y^T + \alpha * y * x^T + A$ A 为 packed 对称矩阵; 具体分为以下两个函数: cblas_sspr2: 单精度浮点型 cblas_dspr2: 双精度浮点型
?hpr	$A \leftarrow \alpha * x * conj(x^T) + A$ A 为三角展开埃尔米特矩阵; 具体分为以下两个函数: cblas_chpr: 单精度复数型 cblas_zhpr: 双精度复数型
?her	$A \leftarrow \alpha * x * conj(x^T) + A$ A 为 n 阶埃尔米特矩阵; 具体分为以下两个函数: cblas_cher: 单精度复数型 cblas_zher: 双精度复数型
?hpr2	$A \leftarrow \alpha * x * conj(y^T) + conj(\alpha) * y * conj(x^T) + A$ A 为三角展开埃尔米特矩阵; 具体分为以下两个函数: cblas_chpr2: 单精度复数型 cblas_zhpr2: 双精度复数型
?her2	$A \leftarrow \alpha * x * conj(y^T) + conj(\alpha) * y * conj(x^T) + A$ A 为 n 阶埃尔米特矩阵; 具体分为以下两个函数: cblas_cher2: 单精度复数型 cblas_zher2: 双精度复数型

函数族	简介
?hpmv	$y \leftarrow \alpha * A * x + \beta * y$ A 是三角展开的埃尔米特矩阵; 具体分为以下两个函数: cblas_chpmv: 单精度复数型 cblas_zhpmv: 双精度复数型
?hemv	$y \leftarrow \alpha * A * x + \beta * y$ A 是 n 阶埃尔米特矩阵; 具体分为以下两个函数: cblas_chemv: 单精度复数型 cblas_zhemv: 双精度复数型
?hbmvm	$y \leftarrow \alpha * A * x + \beta * y$ A 是 n 阶埃尔米特带状矩阵; 具体分为以下两个函数: cblas_chbmvm: 单精度复数型 cblas_zhbmvm: 双精度复数型
?gerc	$A \leftarrow \alpha * x * y^T + A$ A 为常规矩阵; 具体分为以下两个函数: cblas_cgerc: 单精度复数型 cblas_zgerc: 双精度复数型
?geru	$A \leftarrow \alpha * x * conj(y^T) + A$ A 为常规矩阵; 具体分为以下两个函数: cblas_cgeru: 单精度复数型 cblas_zgeru: 双精度复数型

7.1.1.1.3 Level-3 BLAS

Level-3 BLAS 提供了 9 种函数接口。

表 7-3 Level-3 BLAS 函数接口

函数族	简介
?gemm	$C \leftarrow \alpha * op(A) * op(B) + \beta * C$, and $op(X) = X \text{ or } X^T$ A/B/C 为常规矩阵; 具体分为以下四个函数: cblas_sgemm: 单精度浮点型 cblas_dgemm: 双精度浮点型 cblas_cgemm: 单精度复数型 cblas_zgemm: 双精度复数型

函数族	简介
?hemm	$C \leftarrow \alpha * A * B + \beta * C$ or $C \leftarrow \alpha * B * A + \beta * C$ A 为埃尔米特矩阵，B/C 为常规矩阵； 具体分为以下两个函数： cblas_chemm: 单精度复数型 cblas_zhemm: 双精度复数型
?symm	$C \leftarrow \alpha * A * B + \beta * C$ or $C \leftarrow \alpha * B * A + \beta * C$ A 为对称矩阵，B/C 为常规矩阵； 具体分为以下四个函数： cblas_ssymm: 单精度浮点型 cblas_dsymm: 双精度浮点型 cblas_csymm: 单精度复数型 cblas_zsymm: 双精度复数型
?syrk	$C \leftarrow \alpha * A * A^T + \beta * C$ or $C \leftarrow \alpha * A^T * A + \beta * C$ A 为对称矩阵，C 为常规矩阵； 具体分为以下四个函数： cblas_ssyrk: 单精度浮点型 cblas_dsyrk: 双精度浮点型 cblas_csyrk: 单精度复数型 cblas_zsyrk: 双精度复数型
?syr2k	$C \leftarrow \alpha AB^T + \alpha BA^T + \beta C$ or $C \leftarrow \alpha A^T B + \alpha B^T A + \beta C$ C 为对称矩阵，A/B 为常规矩阵； 具体分为以下四个函数： cblas_ssyr2k: 单精度浮点型 cblas_dsyr2k: 双精度浮点型 cblas_csyr2k: 单精度复数型 cblas_zsyr2k: 双精度复数型
?trmm	$B \leftarrow \alpha * op(A) * B$ or $B \leftarrow \alpha * B * op(A)$ and $op(A) = A \text{ or } A^T$ A 为三角矩阵，B 为常规矩阵； 具体分为以下四个函数： cblas_strmm: 单精度浮点型 cblas_dtrmm: 双精度浮点型 cblas_ctrmm: 单精度复数型 cblas_ztrmm: 双精度复数型

函数族	简介
?trsm	$B \leftarrow \alpha * op(A^{-1}) * B$ or $B \leftarrow \alpha * B * op(A^{-1})$, and $op(A) = A \text{ or } A^T$ A 为三角矩阵, B 为常规矩阵; 具体分为以下四个函数: cblas_strsm: 单精度浮点型 cblas_dtrsm: 双精度浮点型 cblas_ctrsm: 单精度复数型 cblas_ztrsm: 双精度复数型
?herk	$C \leftarrow \alpha * A * A^H + \beta * C$ or $C \leftarrow \alpha A^H A + \beta C$ C 为埃尔米特矩阵; 一种情况矩阵 A 为 n*k, 第二种情况矩阵 A 为 k*n; 具体分为以下两个函数: cblas_cherk: 单精度复数型 cblas_zherk: 双精度复数型
?her2k	$C \leftarrow \alpha * A * B^H + conjg(\alpha) * B * A^H + \beta * C$ or $C \leftarrow \alpha A^H B + conjg(\alpha) * B^H * A + \beta * C$ C 为埃尔米特矩阵; 第一种情况矩阵 A, B 为 n*k, 第二种情况矩阵 A, B 为 k*n; 具体分为以下两个函数: cblas_cher2k: 单精度复数型 cblas_zher2k: 双精度复数型

7.1.1.2 LAPACK 接口

LAPACK 是一个实现了高级的线性运算功能的接口标准, 例如矩阵分解, 求逆等, 底层是调用的 BLAS 代码库。要替代本平台科学计算 LAPACK 算法库, 应符合 LAPACK 标准的 C 接口形式。

LAPACK 能够用于计算多种线性代数问题, 针对每种问题又提供不同的求解方法, 其程序组织结构分为三层, 如下表所示:

表 7-4 LAPACK 程序组织结构

driver routines	每个 driver 程序能够求解一个完整问题, 如线性方程组求解、计算矩阵特征值, 其作为顶层程序被用户直接调用。driver 程序主要分为如下 5 类: 1) Linear Equations 2) Linear Least Squares(LLS) Problems 3) Generalized Linear Least Squares(LSE and GLM) Problems 4) Standard Eigenvalue and Singular Value Problems
------------------------	--

	5) Generalized Eigenvalue and Singular Value Problems
computational routines	不同 computational 程序执行不同计算任务，如 LU 分解、正交分解，其作为中间层程序被 driver 调用，通常一个 driver 程序需要调用一系列 computational 程序。另外，用户也可以直接调用 computational 程序实现目标任务求解。 computational 程序主要分为如下 9 类： 1) Linear Equations 2) Orthogonal Factorizations and Linear Least Squares Problems 3) Generalized Orthogonal Factorizations and Linear Least Squares 4) Symmetric Eigenproblems 5) Nonsymmetric Eigenproblems 6) Singular Value Decomposition 7) Generalized Symmetric Definite Eigenproblems 8) Generalized Nonsymmetric Eigenproblems 9) Generalized (or Quotient) Singular Value Decomposition
auxiliary routines	auxiliary 程序则是最下层程序，通常由 computational 程序调用，如计算矩阵范数、矩阵缩放，基本上等于 BLAS 的功能，且往往就是调用 BLAS 中的函数。

下方根据函数族的功能区分进行介绍。

7.1.1.2.1 矩阵分解函数

表 7-5 矩阵分解函数

函数族	简介
?getrf	计算矩阵 A 的 LU 分解，允许行交换（partial pivoting）。分解结果为 $A = PLU$，其中 P 为置换矩阵、L 为下三角矩阵或下阶梯矩阵且对角线为 1，U 是上三角矩阵或上阶梯矩阵。 具体分为以下四个函数： sgetrf_：单精度浮点型 dgetrf_：双精度浮点型 cgetrf_：单精度复数型 zgetrf_：双精度复数型
?geqrf	计算矩阵的 QR 分解，即 $A=Q*R$ 具体分为以下四个函数： sgeqrf_：单精度浮点型 dgeqrf_：双精度浮点型 cgeqrf_：单精度复数型 zgeqrf_：双精度复数型

函数族	简介
?gerqf	计算矩阵的 RQ 分解，即 $A=R*Q$。 具体分为以下四个函数： sgerqf_：单精度浮点型 dgerqf_：双精度浮点型 cgerqf_：单精度复数型 zgerqf_：双精度复数型
?geqlf	计算矩阵的 QL 分解，即 $A=Q*L$。 具体分为以下四个函数： sgeqlf_：单精度浮点型 dgeqlf_：双精度浮点型 cgeqlf_：单精度复数型 zgeqlf_：双精度复数型
?gelqf	计算矩阵的 LQ 分解，即 $A=L*Q$。 具体分为以下四个函数： sgelqf_：单精度浮点型 dgelqf_：双精度浮点型 cgelqf_：单精度复数型 zgelqf_：双精度复数型
?potrf	计算对称正定矩阵或者 Hermite 正定矩阵的 Cholesky 分解，即 $A = LL^T$ 或者 $A = UU^T$, U 上三角矩阵，L 下三角矩阵。 具体分为以下四个函数： spotrf_：单精度浮点型 dpotrf_：双精度浮点型 cpotrf_：单精度复数型 zpotrf_：双精度复数型
?pttrf	计算实（共轭）对称正定三对角矩阵 A 的 LDL^H 或 $U^H DU$ 分解，其中 D 为对角块矩阵，L 为下三角矩阵，U 为上三角矩阵。 具体分为以下四个函数： spttrf_：单精度浮点型 dpttrf_：双精度浮点型 cpttrf_：单精度复数型 zpttrf_：双精度复数型
?gttrf	计算一般三对角矩阵 A 的 LU 分解。 具体分为以下四个函数： sgttrf_：单精度浮点型 dgttrf_：双精度浮点型 cgttrf_：单精度复数型 zgttrf_：双精度复数型

函数族	简介
?sptrf	计算压缩对称矩阵的LDL^H或$U^H DU$分解。 具体分为以下四个函数： ssptrf_：单精度浮点型 dsptrf_：双精度浮点型 csptrf_：单精度复数型 zsptrf_：双精度复数型

7.1.1.2.2 线性方程求解函数

表 7-6 线性方程求解函数

函数族	简介
?ppsv	求解线性方程组$A * X = B$。其中 A 为实对称或共轭对称的 $n*n$ 正定矩阵，以压缩方式存储；X 和 B 为 $n*nrhs$ 矩阵。计算矩阵 A 的 Cholesky 分解，并通过分解结果求解线性方程组。 具体分为以下四个函数： sppsv_：单精度浮点型 dppsv_：双精度浮点型 cppsv_：单精度复数型 zppsv_：双精度复数型
?gesv	求解线性方程组$A * X = B$。其中 A 为 $n*n$ 的矩阵，X 和 B 为 $n*nrhs$ 矩阵。求解过程中，对 A 使用具有部分旋转和行交换的 LU 分解，将其分解为$A = P * L * U$的形式，其中 P 是置换矩阵，L 是单位下三角矩阵，U 是上三角矩阵。并通过 LU 分解的结果来求解线性方程组。 具体分为以下四个函数： sgesv_：单精度浮点型 dgesv_：双精度浮点型 cgesv_：单精度复数型 zgesv_：双精度复数型
?ptsv	求解线性方程组，其中系数矩阵 A 为实（共轭）对称正定三角矩阵。 具体分为以下四个函数： sptsv_：单精度浮点型 dptsv_：双精度浮点型 cptsv_：单精度复数型 zptsv_：双精度复数型

函数族	简介
?gtsv	求解线性方程组 $A * X = B$ ，其中系数矩阵 A 为一般三对角矩阵。 具体分为以下四个函数： sgtsv_：单精度浮点型 dgtsv_：双精度浮点型 cgtsv_：单精度复数型 zgtsv_：双精度复数型

7.1.1.2.3 矩阵求逆函数

表 7-7 矩阵求逆函数

函数族	简介
?getri	根据?getrf得到的LU分解结果计算逆矩阵。 具体分为以下四个函数： sgetri_：单精度浮点型 dgetri_：双精度浮点型 cgetri_：单精度复数型 zgetri_：双精度复数型
?potri	计算对称正定矩阵的逆。 具体分为以下四个函数： spotri_：单精度浮点型 dpotri_：双精度浮点型 cpotri_：单精度复数型 zpotri_：双精度复数型

7.1.1.2.4 回代求解函数

表 7-8 回代求解函数

函数族	简介
?pttrs	求解三对角方程 $AX=B$ ，其中系数矩阵 A 由?pttrf分解而来。 具体分为以下四个函数： spttrs_：单精度浮点型 dpttrs_：双精度浮点型 cpttrs_：单精度复数型 zpttrs_：双精度复数型
?ptts2	求解三对角方程 $AX=B$ ，其中系数矩阵 A 由?pttrf分解而来。 具体分为以下四个函数： sptts2_：单精度浮点型 dptts2_：双精度浮点型 cptts2_：单精度复数型 zptts2_：双精度复数型

函数族	简介
?gttrs	求解三对角方程 $A * X = B$，或 $A^H * T * X = B$，或 $A^H * H * X = B$，其中系数矩阵 A 由?gttrf 分解而来。 具体分为以下四个函数： sgttrs_：单精度浮点型 dgttrs_：双精度浮点型 cgttrs_：单精度复数型 zgttrs_：双精度复数型
?trtrs	求解三角方程 $A * X = B$，或 $A^H * T * X = B$。 具体分为以下四个函数： strtrs_：单精度浮点型 dtrtrs_：双精度浮点型 ctrtrs_：单精度复数型 ztrtrs_：双精度复数型

7.1.1.2.5 特征值问题函数

表 7-9 特征值问题函数

函数族	简介
?sy(he)evd	计算对称（Hermite）矩阵的全部特征值和特征向量（可选）。其中特征向量通过分治算法计算。即矩阵 A 分解为：$A = Z\Lambda Z^T$，其中 Λ 为对角矩阵，对角线元素 λ_i 为特征值；Z 为正交矩阵，其每列向量 Z_i 为对应的特征向量。即 $Az_i = \lambda_i z_i$，其中 $i=1,2,\dots,n$。 具体分为以下四个函数： ssyevd_：单精度浮点型 dsyevd_：双精度浮点型 cheevd_：单精度复数型 zheevd_：双精度复数型
?sy(he)ev	计算对称（Hermite）矩阵的所有特征值与特征向量。 具体分为以下四个函数： ssyev_：单精度浮点型 dsyev_：双精度浮点型 cheev_：单精度复数型 zheev_：双精度复数型

7.1.1.2.6 其他函数

表 7-10 其他函数

函数族	简介
?(or,un)glq	生成具有正交行的实/复矩阵 Q ，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即： $Q=H(k)...H(2)H(1)$ ，其中 H 为?gelqf 返回的。 具体分为以下四个函数： sorglq_：单精度浮点型 dorglq_：双精度浮点型 cunglq_：单精度复数型 zunglq_：双精度复数型
?(or,un)gqr	生成具有正交行的实/复矩阵 Q ，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即： $Q=H(k)...H(2)H(1)$ ，其中 H 为?geqrf 返回的。 sorgqr_：单精度浮点型 dorgqr_：双精度浮点型 cungqr_：单精度复数型 zungqr_：双精度复数型
?(or,un)grq	生成具有正交行的实/复矩阵 Q ，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即： $Q=H(1)H(2)\cdots H(k)$ ，其中 H 为?gerqf 返回的。 sorgrq_：单精度浮点型 dorgrq_：双精度浮点型 cungrq_：单精度复数型 zungrq_：双精度复数型
?(or,un)gql	生成具有正交列的实/复矩阵 Q ，且该矩阵定义为 K 个 M 阶基本反射器的乘积的前 N 列，即： $Q=H(k)...H(2)H(1)$ ，其中 H 为?geqlf 返回的。 sorgql_：单精度浮点型 dorgql_：双精度浮点型 cungql_：单精度复数型 zungql_：双精度复数型
?(or,un)mlq	计算 $C = Q^* * C$ 或 $C = C * Q^*$ ，其中 Q 是由?gelqf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T ，复数时 Q^* 表示 Q^H ；当 Q 不转置， Q^* 表示 Q 。 sorglq_：单精度浮点型 dorglq_：双精度浮点型 cunglq_：单精度复数型 zunglq_：双精度复数型

函数族	简介
?(or,un)mqr	计算 $C = Q^* * C$ 或 $C = C * Q^*$，其中 Q 是由?geqrf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T，复数时 Q^* 表示 Q^H；当 Q 不转置，Q^* 表示 Q。 sormqr_：单精度浮点型 dormqr_：双精度浮点型 cunmqr_：单精度复数型 zunmqr_：双精度复数型
?(or,un)mql	计算 $C = Q^* * C$ 或 $C = C * Q^*$，其中 Q 是由?geqlf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T，复数时 Q^* 表示 Q^H；当 Q 不转置，Q^* 表示 Q。 sormql_：单精度浮点型 dormql_：双精度浮点型 cunmql_：单精度复数型 zunmql_：双精度复数型
?(or,un)mrq	计算 $C = Q^* * C$ 或 $C = C * Q^*$，其中 Q 是由?gerqf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T，复数时 Q^* 表示 Q^H；当 Q 不转置，Q^* 表示 Q。 sormrq_：单精度浮点型 dormrq_：双精度浮点型 cunmrq_：单精度复数型 zunmrq_：双精度复数型
?sy(he)trd	将对称矩阵或 Hermite 矩阵通过相似变换成对称三对角 T，即 $Q^* * H * A * Q = T$。 ssytrd_：单精度浮点型 dormrq_：双精度浮点型 cunmrq_：单精度复数型 zunmrq_：双精度复数型
?lasr	对矩阵 A 做平面旋转操作。 slasr_：单精度浮点型 dlasr_：双精度浮点型 clasr_：单精度复数型 zlasr_：双精度复数型
?sy(he)trd_2stage	将对称矩阵或 Hermite 矩阵 A 转为对称或 Hermite 三对角矩阵 T，即 $A^* * H * A * Q = T$，其中 Q 为正交矩阵。 ssytrd_2stage：单精度浮点型 dsytrd_2stage：双精度浮点型 csytrd_2stage：单精度复数型 zsytrd_2stage：双精度复数型

函数族	简介
?laset	初始化 $m \times n$ 的矩阵，且将对角线元素设置为 β ，非对角线元素设置为 α 。 slaset_：单精度浮点型 dlaset_：双精度浮点型 claset_：单精度复数型 zlaset_：双精度复数型

7.1.1.2.7 Support 函数

表 7-11 Support 函数

函数	简介
KLAPACKGetVersion	获取 LAPACK 产品版本信息。

7.1.1.3 FFT 接口

FFT 用于计算一维或多维、任意输入大小以及实数和复数数据（以及偶数/奇数数据，即离散余弦/正弦变换）的离散傅里叶变换，包含 C2C 变换、R2C 变换、C2R 变换、R2R 变换。要替代本平台科学计算 FFT 算法库，应符合类 FFTW 的 C 接口形式。

关于 FFT 更多介绍请参考：

https://en.wikipedia.org/wiki/Fast_Fourier_transform

FFT 算法函数事实接口详情：

https://www.hikunpeng.com/document/detail/zh/kunpengaccel/math-lib/devg-kml/kunpengaccel_kml_16_0126.html

7.1.1.3.1 C2C 变换

表 7-12 C2C 变换

函数名	简介
fft(f)_plan_dft	建立单个连续数据序列 n 维 C2C 变换的 plan。
fft(f)_plan_dft_1d	建立单个连续数据序列 1 维 C2C 变换的 plan。
fft(f)_plan_dft_2d	建立单个连续数据序列 2 维 C2C 变换的 plan。
fft(f)_plan_dft_3d	建立单个连续数据序列 3 维 C2C 变换的 plan。
fft(f)_plan_guru_dft	建立多组数据序列 n 维 C2C 变换的 plan。
fft(f)_plan_guru_split_dft	建立多组数据序列 n 维 C2C 变换的 plan。
fft(f)_plan_guru64_dft	建立多组数据序列 n 维 C2C 变换的 plan。

函数名	简介
fft(f)_plan_guru64_split_dft	建立多组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
fft(f)_plan_many_dft	建立 howmany 组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

7.1.1.3.2 R2C 变换

表 7-13 1.1.1.1.1 R2C 变换

函数名	简介
fft(f)_plan_dft_r2c	建立单个连续数据序列 n 维 R2C 变换的 plan。
fft(f)_plan_dft_r2c_1d	建立单个连续数据序列 1 维 R2C 变换的 plan。
fft(f)_plan_dft_r2c_2d	建立单个连续数据序列 2 维 R2C 变换的 plan。
fft(f)_plan_dft_r2c_3d	建立单个连续数据序列 3 维 R2C 变换的 plan。
fft(f)_plan_guru_dft_r2c	建立多组数据序列 n 维 R2C 变换的 plan。
fft(f)_plan_guru_split_dft_r2c	建立多组数据序列 n 维 R2C 变换的 plan。
fft(f)_plan_guru64_dft_r2c	建立多组数据序列 n 维 R2C 变换的 plan。
fft(f)_plan_guru64_split_dft_r2c	建立多组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
fft(f)_plan_many_dft_r2c	建立 howmany 组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

7.1.1.3.3 C2R 变换

表 7-14 C2R 变换

函数名	简介
fft(f)_plan_dft_c2r	建立单个连续数据序列 n 维 C2R 变换的 plan。

函数名	简介
fft(f)_plan_dft_c2r_1d	建立单个连续数据序列 1 维 C2R 变换的 plan。
fft(f)_plan_dft_c2r_2d	建立单个连续数据序列 2 维 C2R 变换的 plan。
fft(f)_plan_dft_c2r_3d	建立单个连续数据序列 3 维 C2R 变换的 plan。
fft(f)_plan_guru_dft_c2r	建立多组数据序列 n 维 C2R 变换的 plan。
fft(f)_plan_guru_split_dft_c2r	建立多组数据序列 n 维 C2R 变换的 plan。
fft(f)_plan_guru64_dft_c2r	建立多组数据序列 n 维 C2R 变换的 plan。
fft(f)_plan_guru64_split_dft_c2r	建立多组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
fft(f)_plan_many_dft_c2r	建立 howmany 组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

7.1.1.3.4 R2R 变换

表 7-15 R2R 变换

函数名	简介
fft(f)_plan_r2r	建立单个连续数据序列 n 维 R2R 变换的 plan。
fft(f)_plan_r2r_1d	建立单个连续数据序列 1 维 R2R 变换的 plan。
fft(f)_plan_r2r_2d	建立单个连续数据序列 2 维 R2R 变换的 plan。
fft(f)_plan_r2r_3d	建立单个连续数据序列 3 维 R2R 变换的 plan。
fft(f)_plan_guru_r2r	建立多组数据序列 n 维 R2R 变换的 plan。
fft(f)_plan_guru64_r2r	建立多组数据序列 n 维 R2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
fft(f)_plan_many_r2r	建立 howmany 组数据序列 n 维 R2R 变换的 plan。

7.1.1.3.5 变换执行函数

表 7-16 变换执行函数

函数名	简介
fft(f)_execute	执行之前建立的 FFT 变换 plan。
fft(f)_execute_dft	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_dft_r2c	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_dft_c2r	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_r2r	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_split_dft	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_split_dft_r2c	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_split_dft_c2r	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

7.1.1.3.6 内存函数

内存函数的调用约束：plan 使用完毕后需要通过调用 fft(f)_destroy_plan 函数来释放；fft(f)_malloc 函数所申请的内存空间只能通过调用 fft(f)_free 函数释放。

表 7-17 内存函数

函数名	简介
fft(f)_malloc	fft(f)_malloc 用于分配所需的内存空间。所申请的内存空间只能通过调用 fft(f)_free 函数释放。
fft(f)_free	fft(f)_free 释放由 fft(f)_malloc 函数申请的内存。

函数名	简介
fft(f)_destroy_plan	fft(f)_destroy_plan 释放 FFT 变换 plan 的所有内存。调用此函数之后，此 plan 不再可用。

7.1.1.3.7 线程函数

线程函数的调用顺序约束：

1. 首先调用 fft(f)_init_threads 初始化线程的使用。
2. 然后调用 fft(f)_plan_with_nthreads 指定线程数。
3. 最后调用 fft(f)_cleanup_threads 释放多线程框架相关的资源。

表 7-18 线程函数

函数名	简介
fft(f)_init_threads	fft(f)_init_threads 用于初始化线程的使用。（此函数不可重入。）
fft(f)_plan_with_nthreads	fft(f)_plan_with_nthreads 用于指定 FFT 库接口函数执行的线程数。（此函数不可重入。）
fft(f)_cleanup_threads	fft(f)_cleanup_threads 用于释放多线程框架相关的资源，与 fft(f)_init_threads 相对应，在整个系统退出时调用。调用后不能再使用多线程功能。（此函数不可重入。）

7.1.1.3.8 获取 FFT 产品版本信息

表 7-19 获取 FFT 产品版本信息

函数名	简介
FFTGetVersion	获取 FFT 产品版本信息

7.1.1.4 SPARSE 接口

7.1.1.4.1 Sparse Level 1 函数

表 7-20 Sparse Level 1 函数

函数名	简介
<code>sparse_?axpyi</code>	标量-向量乘，并加到另一向量上：$y = a * x + y$。其中，a 是缩放系数，x 是稀疏向量，y 是稠密向量。具体分为以下四个函数： <code>sparse_saxpyi</code>: 单精度浮点类型标量-向量乘 <code>sparse_daxpyi</code>: 双精度浮点类型标量-向量乘 <code>sparse_caxpyi</code>: 单精度复数类型标量-向量乘 <code>sparse_zaxpyi</code>: 双精度复数类型标量-向量乘
<code>sparse_?doti</code>	实数域的点积（欧几里得空间）。 $res = x[0] * y[indx[0]] + x[1] * y[indx[1]] + ... + x[nz-1] * y[indx[nz-1]]$ 具体分为以下两个函数： <code>sparse_sdoti</code>: 单精度浮点类型点积 <code>sparse_ddoti</code>: 双精度浮点类型点积
<code>sparse_?dotci_sub</code>	共轭点积（希尔伯特空间）；$x \cdot y = x^H * y$，即 x 向量中的元素取其共轭复数后与 y 向量中对应元素相乘然后累加（复数 $a + bi$ 的共轭为 $a - bi$），计算公式表示如下： $conjg(x[0]) * y[indx[0]] + ... + conjg(x[nz-1]) * y[indx[nz-1]]$ 具体分为以下两个函数： <code>sparse_cdotci_sub</code>: 单精度浮点类型共轭点积 <code>sparse_zdotci_sub</code>: 双精度浮点类型共轭点积

函数名	简介
<code>sparse_?dotui_sub</code>	复数域的非共轭点积（欧几里得空间）；$\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^T * \mathbf{y}$，即对应元素相乘然后累加。 $\text{res} = \mathbf{x}[0] * \mathbf{y}[\text{indx}[0]] + \mathbf{x}[1] * \mathbf{y}[\text{indx}[1]] + \dots + \mathbf{x}[\text{nz} - 1] * \mathbf{y}[\text{indx}[\text{nz} - 1]]$ 具体分为以下两个函数： <code>sparse_cdotui_sub</code>：单精度浮点类型非共轭点积 <code>sparse_zdotui_sub</code>：双精度浮点类型非共轭点积
<code>sparse_?gthr</code>	将 full-storage 格式的稀疏向量 $\mathbf{y}$ 中指定的元素加载到 compressed 格式的向量中。即 $\mathbf{x}[i] = \mathbf{y}[\text{indx}[i]]$，$i = 0, 1, \dots, \text{nz} - 1$。 具体分为以下四个函数： <code>sparse_sgthr</code>：单精度浮点类型元素加载 <code>sparse_dgthr</code>：双精度浮点类型元素加载 <code>sparse_cgthr</code>：单精度复数类型元素加载 <code>sparse_zgthr</code>：双精度复数类型元素加载
<code>sparse_?gthrz</code>	将 full-storage 格式的稀疏向量 $\mathbf{y}$ 中指定的元素加载到 compressed 格式的向量中。即 $\mathbf{x}[i] = \mathbf{y}[\text{indx}[i]]$，$i = 0, 1, \dots, \text{nz} - 1$，并将 $\mathbf{y}$ 中对应元素清零。 具体分为以下四个函数： <code>sparse_sgthrz</code>：单精度浮点类型元素加载并清零 <code>sparse_dgthrz</code>：双精度浮点类型元素加载并清零 <code>sparse_cgthrz</code>：单精度复数类型元素加载并清零 <code>sparse_zgthrz</code>：双精度复数类型元素加载并清零
<code>sparse_?roti</code>	对 2 个实数类型的向量进行旋转操作。 $\mathbf{x}[i] = \mathbf{c} * \mathbf{x}[i] + \mathbf{s} * \mathbf{y}[\text{indx}[i]]$ $\mathbf{y}[\text{indx}[i]] = \mathbf{c} * \mathbf{y}[\text{indx}[i]] - \mathbf{s} * \mathbf{x}[i]$ 具体分为以下两个函数： <code>sparse_sroti</code>：单精度浮点类型向量旋转 <code>sparse_droti</code>：双精度浮点类型向量旋转

函数名	简介
sparse_?sctr	将 compressed 格式的向量写入 full-storage 格式的稀疏向量 y 指定位置。即 $y[\text{indx}[i]] = x[i]$, $i=0,1,\dots,nz-1$。具体分为以下四个函数： sparse_ssctr: 单精度浮点类型向量写入 sparse_dsctr: 双精度浮点类型向量写入 sparse_csctr: 单精度复数类型向量写入 sparse_zsctr: 双精度复数类型向量写入

7.1.1.4.2 Sparse Level 2 和 Level 3 函数

表 7-21 Sparse Level 2 和 Level 3 函数

函数名	简介
sparse_?csrgemv	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x$ 或 $y = A^T * x$ 或 $y = A^H * x$, 其中，x 和 y 为向量，A 是稀疏矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 1 开始的矩阵。具体分为以下四个函数： sparse_scsrgemv: 单精度浮点类型矩阵与向量计算 sparse_dcsrgemv: 双精度浮点类型矩阵与向量计算 sparse_ccsrgemv: 单精度复数类型矩阵与向量计算 sparse_zcsrgemv: 双精度复数类型矩阵与向量计算

函数名	简介
sparse_?csrsv	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x$ 其中，x 和 y 为向量，A 是的上三角或者下三角的对称矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 1 开始的矩阵。 具体分为以下四个函数： sparse_scsrsv：单精度浮点类型矩阵与向量计算 sparse_dcsrsv：双精度浮点类型矩阵与向量计算 sparse_ccsrsv：单精度复数类型矩阵与向量计算 sparse_zcsrsv：双精度复数类型矩阵与向量计算
csparse_?csrsv	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x \text{ 或 } y = A^T * x \text{ 或 } y = A^H * x,$ 其中，x 和 y 为向量，A 是稀疏矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 0 开始的矩阵。具体分为以下四个函数： csparse_scsrsv：单精度浮点类型矩阵与向量计算 csparse_dcsrsv：双精度浮点类型矩阵与向量计算 csparse_ccsrsv：单精度复数类型矩阵与向量计算 csparse_zcsrsv：双精度复数类型矩阵与向量计算

函数名	简介
csparse_?csrsv	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x$ 其中，x 和 y 为向量，A 是的上三角或者下三角的对称矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 0 开始的矩阵。具体分为以下四个函数： csparse_ scsrsv: 单精度浮点类型矩阵与向量计算 csparse_ dcsrsv: 双精度浮点类型矩阵与向量计算 csparse_ ccscrsv: 单精度复数类型矩阵与向量计算 csparse_ zcsrsv: 双精度复数类型矩阵与向量计算
sparse_?csrsv	矩阵与向量乘积，矩阵是 CSR 格式的稀疏矩阵。 $y = \alpha * A * x + \beta * y \text{ 或 } y = \alpha * A^T * x + \beta * y \text{ 或 } y = \alpha * A^H * x + \beta * y$ 其中，alpha 和 beta 是缩放系数，x 和 y 是向量，A 是 CSR 格式的稀疏向量。具体分为以下四个函数： sparse_ scsrsv: 单精度浮点类型矩阵与向量乘积 sparse_ dcsrsv: 双精度浮点类型矩阵与向量乘积 sparse_ ccscrsv: 单精度复数类型矩阵与向量乘积 sparse_ zcsrsv: 双精度复数类型矩阵与向量乘积

函数名	简介
sparse_?csrsv	求解三角矩阵方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $A * y = \alpha * x \text{ 或 } A^T * y = \alpha * x \text{ 或 } A^H * y = \alpha * x$ 其中，x 和 y 为向量，A 是 $m * m$ 的稀疏矩阵，采用 CSR 格式存储。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： sparse_scsrsv: 单精度浮点类型求解三角矩阵方程组 sparse_dcsrsv: 双精度浮点类型求解三角矩阵方程组 sparse_ccsrsv: 单精度复数类型求解三角矩阵方程组 sparse_zcsrsv: 双精度复数类型求解三角矩阵方程组
sparse_?csrmm	矩阵与矩阵计算，其中一个稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $C = \alpha * A * B + \beta * C$ ● $C = \alpha * A^T * B + \beta * C$ ● $C = \alpha * A^H * B + \beta * C$ 其中，B 和 C 为稠密矩阵，A 是采用 CSR 格式存储的 $m \times k$ 的稀疏矩阵。 具体分为以下四个函数： sparse_scsrmm: 单精度浮点类型矩阵与矩阵计算 sparse_dcsrmm: 双精度浮点类型矩阵与矩阵计算 sparse_ccsrmm: 单精度复数类型矩阵与矩阵计算 sparse_zcsrmm: 双精度复数类型矩阵与矩阵计算

函数名	简介
<code>sparse_?csradd</code>	计算 2 个 CSR 格式的稀疏矩阵之和。 $C = A + \text{beta} * \text{op}(B)$ A、B、C 为 3 数组 CSR 格式稀疏矩阵，仅支持基 1 索引。 $\text{op}(B)$可能的操作为： ● $\text{op}(B) = B$ ● $\text{op}(B) = B^T$ ● $\text{op}(B) = B^H$ 具体分为以下四个函数： <code>sparse_scsradd</code>: 单精度浮点类型稀疏矩阵之和 <code>sparse_dcsradd</code>: 双精度浮点类型稀疏矩阵之和 <code>sparse_ccsradd</code>: 单精度复数类型稀疏矩阵之和 <code>sparse_zcsradd</code>: 双精度复数类型稀疏矩阵之和
<code>sparse_?csrmultcsr</code>	矩阵与矩阵计算，其中 3 个稀疏矩阵采用 3-arrays CSR 格式存储，为基 1 索引，具体执行操作为： $C = \text{op}(A) * B$。 其中，A、B、C 是采用 CSR 格式存储的稀疏矩阵。 $\text{op}(A)$根据参数是以下 3 种情况之一： ● $\text{op}(A) = A$ ● $\text{op}(A) = A^T$ ● $\text{op}(A) = A^H$ 具体分为以下四个函数： <code>sparse_scsrmultcsr</code>: 单精度浮点类型矩阵与矩阵计算 <code>sparse_dcsrmultcsr</code>: 双精度浮点类型矩阵与矩阵计算 <code>sparse_ccsrmultcsr</code>: 单精度复数类型矩阵与矩阵计算 <code>sparse_zcsrmultcsr</code>: 双精度复数类型矩阵与矩阵计算

函数名	简介
sparse_?csrmultd	矩阵与矩阵计算，其中 2 个稀疏矩阵采用 CSR 格式存储，为基 1 索引，具体执行操作为：$C := \text{op}(A) * B$。 其中，C 为稠密矩阵，A 和 B 是采用 CSR 格式存储的稀疏矩阵。 $\text{op}(A)$ 根据参数是以下 3 种情况之一： ● $\text{op}(A) = A$ ● $\text{op}(A) = A^T$ ● $\text{op}(A) = A^H$ 具体分为以下四个函数： sparse_scsrmultd: 单精度浮点类型矩阵与矩阵计算 sparse_dcsrmultd: 双精度浮点类型矩阵与矩阵计算 sparse_ccsrmultd: 单精度复数类型矩阵与矩阵计算 sparse_zcsrmultd: 双精度复数类型矩阵与矩阵计算
sparse_?csrtrsv	求解稀疏三角线性方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $x = A * y$ ● $x = A^T * y$ ● $x = A^H * y$ A 为 3 数组 CSR 格式稀疏矩阵，x、y 为向量，仅支持 One-based indexing。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： sparse_scsrtrsv: 单精度浮点类型求解稀疏三角线性方程组 sparse_dcsrtrsv: 双精度浮点类型求解稀疏三角线性方程组 sparse_ccsrtrsv: 单精度复数类型求解稀疏三角线性方程组 sparse_zcsrtrsv: 双精度复数类型求解稀疏三角线性方程组

函数名	简介
<code>csparse_?csrtrsv</code>	求解稀疏三角线性方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $x = A * y$ ● $x = A^T * y$ ● $x = A^H * y$ A 为 3 数组 CSR 格式稀疏矩阵，x、y 为向量，仅支持 ze-based indexing。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： <code>csparse_scsrtrsv</code>：单精度浮点类型求解稀疏三角线性方程组 <code>csparse_dcsrtrsv</code>：双精度浮点类型求解稀疏三角线性方程组 <code>csparse_ccsrtrsv</code>：单精度复数类型求解稀疏三角线性方程组 <code>csparse_zcsrtrsv</code>：双精度复数类型求解稀疏三角线性方程组
<code>sparse_?cscsv</code>	求解三角矩阵方程组计算，稀疏矩阵采用 CSC 格式存储，具体执行操作如下： ● $A * y = \alpha * x$ ● $A^T * y = \alpha * x$ ● $A^H * y = \alpha * x$ 其中，x 和 y 为向量，A 是 $m * m$ 的稀疏矩阵，采用 CSC 格式存储。矩阵 A 中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。 具体分为以下四个函数： <code>sparse_scscsv</code>：单精度浮点类型求解三角矩阵方程组 <code>sparse_dcscsv</code>：双精度浮点类型求解三角矩阵方程组 <code>sparse_ccscsv</code>：单精度复数类型求解三角矩阵方程组 <code>sparse_zcscsv</code>：双精度复数类型求解三角矩阵方程组

函数名	简介
<code>sparse_?cscmv</code>	矩阵与向量乘积，矩阵是 CSC 格式的稀疏矩阵。 ● $y = \alpha * A * x + \beta * y$ ● $y = \alpha * A^T * x + \beta * y$ ● $y = \alpha * A^H * x + \beta * y$ 其中，α 和 β 是缩放系数，x 和 y 是向量，A 是 CSC 格式的稀疏向量。具体分为以下四个函数： <code>sparse_scscmv</code>: 单精度浮点类型矩阵与向量乘积 <code>sparse_dcscmv</code>: 双精度浮点类型矩阵与向量乘积 <code>sparse_ccscmv</code>: 单精度复数类型矩阵与向量乘积 <code>sparse_zcscmv</code>: 双精度复数类型矩阵与向量乘积
<code>sparse_?cscmm</code>	矩阵与矩阵计算，其中一个稀疏矩阵采用 CSC 格式存储。具体执行操作如下： $C = \alpha * A * B + \beta * C$ $C = \alpha * A^T * B + \beta * C$ $C = \alpha * A^H * B + \beta * C$ 其中，B 和 C 为稠密矩阵，A 是采用 CSC 格式存储的 $m \times k$ 的稀疏矩阵。具体分为以下四个函数： <code>sparse_scscmm</code>: 单精度浮点类型矩阵与矩阵计算 <code>sparse_dcscmm</code>: 双精度浮点类型矩阵与矩阵计算 <code>sparse_ccscmm</code>: 单精度复数类型矩阵与矩阵计算 <code>sparse_zcscmm</code>: 双精度复数类型矩阵与矩阵计算

函数名	简介
<code>sparse_?cscsm</code>	求解三角矩阵方程组计算，其中一个稀疏矩阵采用 CSC 格式存储，具体执行操作如下： ● $A * y = \alpha * x$ ● $A^T * y = \alpha * x$ ● $A^H * y = \alpha * x$ 其中，x 和 y 为 $m*n$ 的稠密矩阵，A 是采用 CSC 格式存储的 $m*m$ 的稀疏矩阵。矩阵 A 中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。 具体分为以下四个函数： <code>sparse_scscsm</code>：单精度浮点类型求解三角矩阵方程组 <code>sparse_dcscsm</code>：双精度浮点类型求解三角矩阵方程组 <code>sparse_ccscsm</code>：单精度复数类型求解三角矩阵方程组 <code>sparse_zcscsm</code>：双精度复数类型求解三角矩阵方程组
<code>sparse_?csrsm</code>	求解三角矩阵方程组计算，其中一个稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $A * y = \alpha * x$ ● $A^T * y = \alpha * x$ ● $A^H * y = \alpha * x$ 其中，x 和 y 为 $m \times n$ 的稠密矩阵，A 是采用 CSR 格式存储的 $m \times m$ 的稀疏矩阵。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： <code>sparse_scsrsm</code>：单精度浮点类型求解三角矩阵方程组 <code>sparse_dcscrsm</code>：双精度浮点类型求解三角矩阵方程组 <code>sparse_ccscrsm</code>：单精度复数类型求解三角矩阵方程组 <code>sparse_zcsrsm</code>：双精度复数类型求解三角矩阵方程组

7.1.1.4.3 Support 函数

表 7-22 Support 函数

函数名	简介
set_thread_num	设置 Sparse BLAS 库接口函数执行的线程数（此函数不可重入）。
SPBLASGetVersion	获取 SPBLAS 产品版本信息

7.1.2 模型求解算法

科学计算与系统建模仿真平台模型翻译后的方程系统包括 ODE 初值问题、线性代数系统、非线性代数系统。模型仿真需要对这些方程系统进行解算，因此模型求解用到 ODE 初值问题求解算法、线性系统求解算法、非线性系统求解算法。科学计算与系统建模仿真平台内置有很多这类求解算法，除此之外，平台还提供扩展模型求解算法的功能。

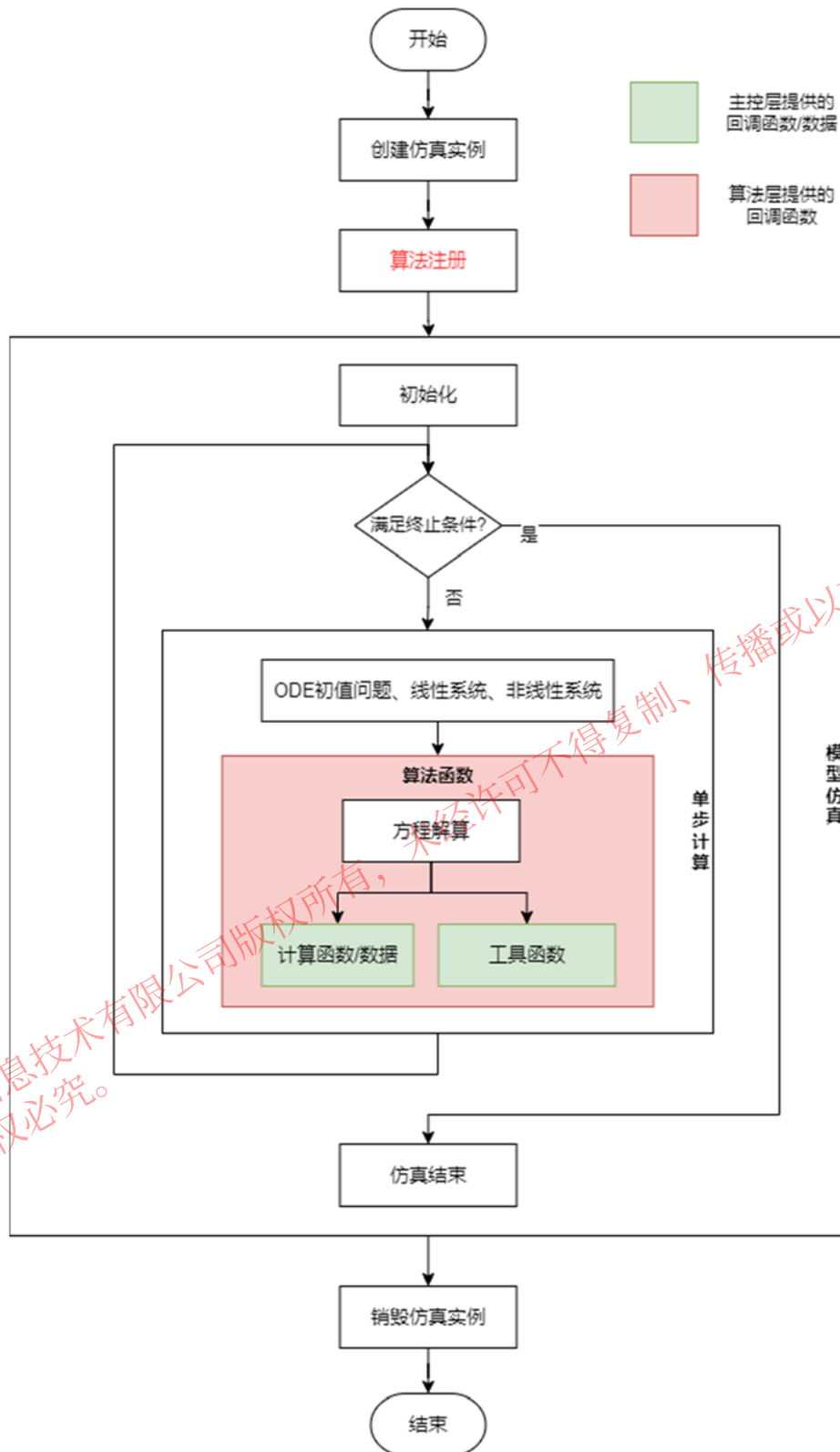
对科学计算与系统建模仿真平台模型求解在主控层的对外接口包括创建仿真实例、算法注册、模型仿真、销毁实例。其中，算法注册用于集成算法层扩展的求解算法。

模型仿真在主控层内部分为初始化、单步计算和仿真结束三个步骤。单步计算循环执行，使仿真在时间轴上推进，直至满足仿真终止条件而结束仿真。

ODE 初值问题、线性代数系统、非线性代数系统的求解算法函数在单步计算过程中被调用。当模型中有此类方程时，主控层调用算法层的算法函数对方程进行解算。算法层的算法函数在解算过程中，为获得方程的描述而调用主控层提供的计算函数（初值问题和非线性问题）或从主控层获取数据（线性问题）。

科学计算与系统建模仿真平台内核层的三个层次中，算法层、主控层中的算法注册由算法扩展方实现，其余部分均由平台提供。

后续将详细说明扩展科学计算与系统建模仿真平台模型求解算法涉及到的主控层求解算法注册接口、方程的计算函数原型或数据格式、工具函数的原型，以及算法层算法函数原型。



模型求解算法扩展的步骤是相同的：

- (1). 首先，按规范实现算法函数。
- (2). 然后在应用层将这些算法函数通过算法注册/设置接口集成到求解器。

(3). 模型方程和算法函数一起构建出模型的求解器，用于对模型进行仿真。

ODE 初值问题求解算法是注册到系统，每次仿真时在界面中选取指定名字的算法，即 ODE 初值问题求解算法注册后具备被选作求解算法的资格。

线性代数系统和非线性代数系统的求解算法是设置到系统，成为每次仿真时使用的算法。

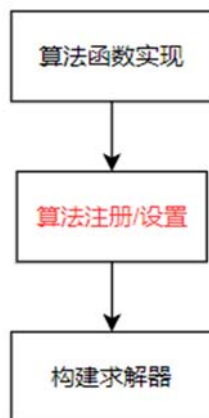


图 7-3 模型求解算法扩展过程

求解源文件 `mws_user_algo.c` 中有 `MwsRegisterUserAlgorithm1`、`MwsRegisterUserAlgorithm2` 函数，在前一函数中注册和设置线性系统求解算法、非线性系统求解算法，在后一函数中注册 ODE 初值问题求解算法。模型求解算法扩展案例见附录 2.2。

7.1.2.1 ODE 初值问题求解算法

模型的 ODE 初值问题通用形式为：

$$M\dot{y} = f(t, y)$$

$$y(t_0) = y_0$$

其中， t 是自变量， y 是因变量， $\dot{y}$ 是 $\frac{dy}{dt}$ 。 M 是变换矩阵，大多数情况下是单位阵。

另外，一些求解算法还将右端表示为两部分。例如：

$$M\dot{y} = f^E(t, y) + f^I(t, y)$$

$$y(t_0) = y_0$$

其中， $f^E(t, y)$ 表示非刚性（nonstiff）部分，使用显式方法求解； $f^I(t, y)$ 表示刚性（stiff）部分，使用隐式方法求解。

显式问题：

$$\dot{y} = f(t, y)$$

隐式问题：

$$f(t, y, \dot{y}) = 0$$

初值问题的算法本文也称之为积分算法，正在运行的积分算法对象称之为积分器。

由上所述可知，计算函数返回的是上述显示问题中 $\dot{y}$ 或隐式问题中 $f(t, y, \dot{y})$ 计算得到的值，由主控层提供。算法函数是对 ODE 初值问题进行求解的一系列函数，由算法扩展方提供，注册后与求解器集成。

工具函数由主控层提供，由算法函数使用，用于动态内存分配、日志输出等。数据结构由主控层定义，用于在函数之间传递数据。

ODE 初值问题求解算法扩展步骤参见 7.1.2 中图 7-3。

工具函数功能和算法扩展步骤所有算法都相同，下文不再赘述。

7.1.2.1.1 数据结构

数据结构由规范定义，算法扩展时可直接使用。

表 7-23 数据结构

函数名	简介
typedef void* MwsIVPSolverObj;	积分算法对象
typedef void* MwsIVPObj;	求解问题对象
typedef enum MwsIVPStatus	积分算法函数返回状态
typedef struct MwsIVPOptions	积分算法选项
typedef enum MwsIVPType;	积分算法求解问题类型
typedef struct MwsIVPSolverFcns	积分算法使用的算法函数
typedef struct MwsIVPUtilFcns	积分算法使用的工具函数
typedef struct MwsIVPCallback	积分算法使用的计算函数

7.1.2.1.2 算法函数

算法函数由算法扩展方实现，即实现 ODE 初值问题求解算法，其中必须实现的函数由主控层按如下流程进行调用。

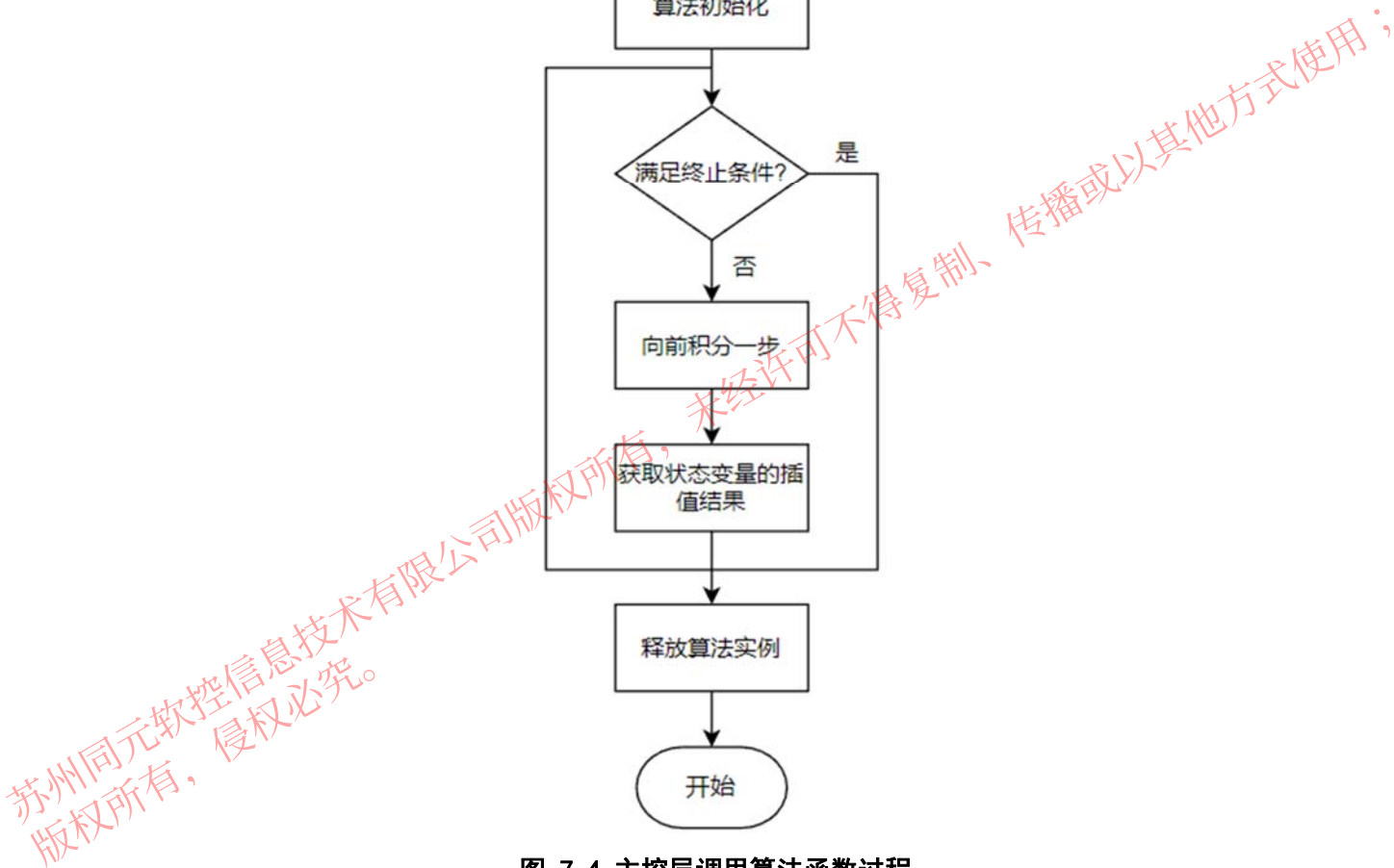


图 7-4 主控层调用算法函数过程

表 7-24 算法函数

函数名	简介
MwsIVPSolverCreatePtr	积分算法对象创建函数
MwsIVPCreatePtr	求解问题对象创建函数
MwsIVPInitPtr	积分算法初始化
MwsIVPSolvePtr	求解函数

MwsIVPInterpolatePtr	插值函数
MwsIVPDestroyPtr	求解问题对象销毁函数
MwsIVPSolverDestroyPtr	释放积分算法实例

7.1.2.1.3 工具函数

工具函数由主控层提供，在算法层由算法函数调用。

表 7-25 工具函数

函数名	简介
m_logger	日志函数
m_allocMemory	内存分配函数
m_freeMemory	内存释放函数
m_allocDataMemory	数据内存分配函数
m_freeDataMemory	数据内存释放函数

7.1.2.1.4 计算函数

计算函数由主控层提供，在算法层由算法接口函数调用。

计算函数由主控层提供，其中 ODE 模型函数值表示下面两种形式中的 f 函数值，在算法层由算法函数调用。

显式问题：

$$\dot{y} = f(t, y)$$

隐式问题：

$$f(t, y, \dot{y}) = 0$$

表 7-26 计算函数

函数名	简介
MwsIVPRshFcnPtr	ODE 通用形式右端计算函数 $y' = f(t, y)$
MwsIVPResFcnPtr	DAE 残余量计算函数

MwsIVPJacFcnPtr	Jacobian 计算函数
MwsIVPStepFinishedPtr	积分步完成回调函数

7.1.2.1.5 算法注册接口

注册接口通常在 mws_user_algo.c 文件中调用，对实现的算法对象进行注册。

表 7-27 算法注册接口

函数名	简介
isimRegisterIVPSolver	注册非线性求解算法
isimUnregisterIVPSolver	移除注册的积分算法

7.1.2.2 线性代数系统

线性代数系统的形式为：

$$Ax = b$$

其中 A 是系数矩阵，b 是常数项，x 是未知量。

线性代数系统的系数矩阵 A 和常数项 b 由主控层提供，调用算法函数时传入。算法函数是对线性代数系统问题进行求解的一系列函数，由算法扩展方提供，注册后与求解器集成。

7.1.2.2.1 数据结构

数据结构由规范定义，算法扩展时可直接使用。

表 7-28 数据结构

函数名	简介
typedef void* MwsLSSolverObj;	线性代数系统求解算法对象
typedef void* MwsLSPObj;	线性代数系统求解问题对象
typedef enum MwsLSStatus	线性代数系统求解算法函数返回状态
typedef enum MwsLSFactorType	线性代数系统系数类型
typedef struct MwsLSSolverProp	线性代数系统求解算法属性
typedef struct MwsLSOptions	线性代数系统求解选项

typedef struct MwsLSSolverFcns	线性代数系统求解算法函数
typedef struct MwsLSUtilFcns	线性代数系统求解算法工具函数

7.1.2.2.2 算法函数

线性系统求解算法的算法函数由扩展方实现，其中必须实现的函数由主控层调用的主体流程如下：

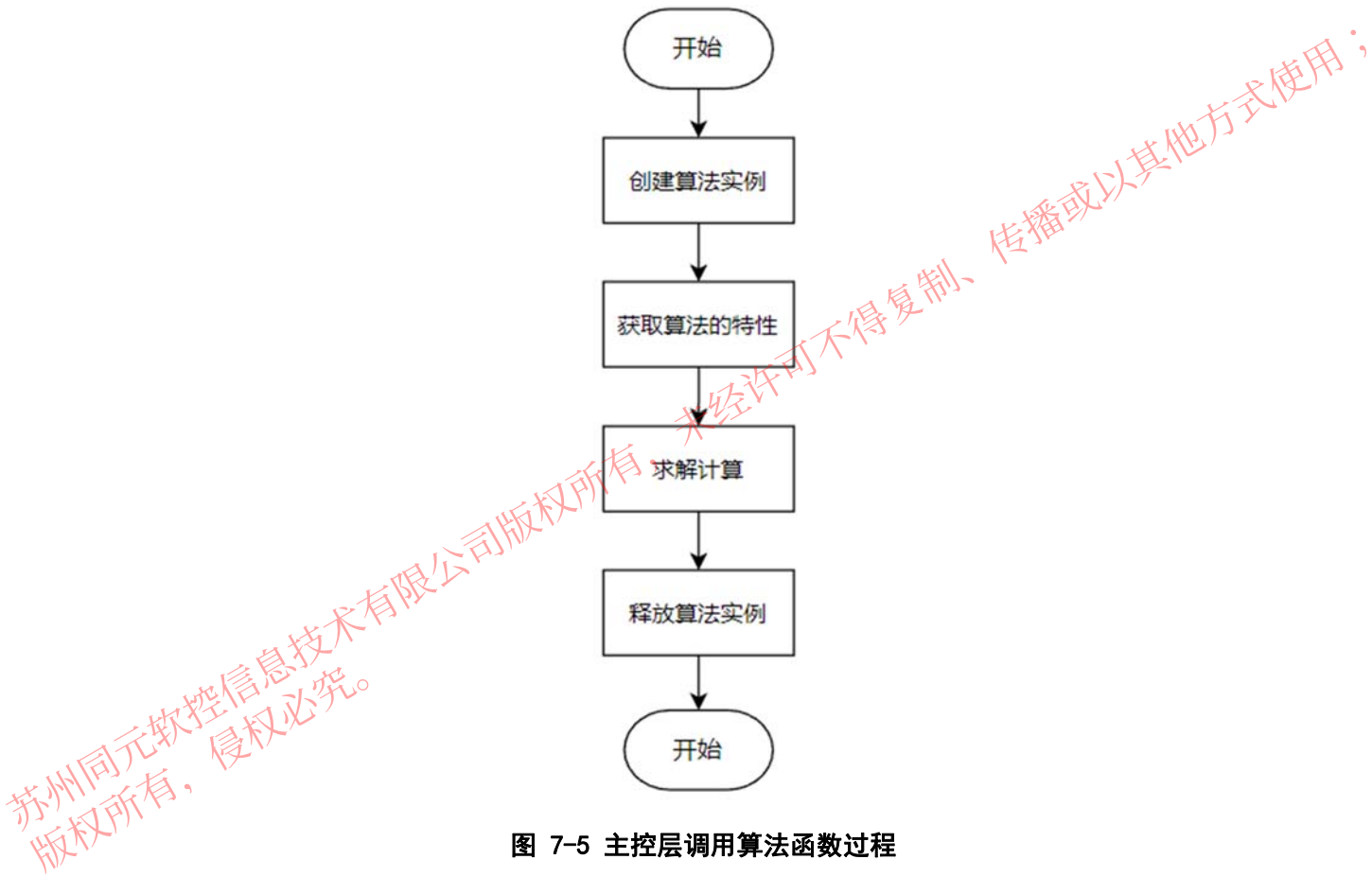


图 7-5 主控层调用算法函数过程

表 7-29 算法函数

函数名	简介
MwsLSSolverCreatePtr	线性求解算法对象创建函数
MwsLSProblemCreatePtr	线性求解算法对象创建函数
MwsLSSolvePtr	求解函数

MwsLSProblemDestroyPtr	求解问题对象销毁函数
MwsLSSolverDestroyPtr	求解对象销毁函数

7.1.2.2.3 工具函数

工具函数由主控层提供，在算法层由算法函数调用。

表 7-30 工具函数

函数名	简介
m_logger	日志函数
m_allocMemory	内存分配函数
m_freeMemory	内存释放函数
m_allocDataMemory	数据内存分配函数
m_freeDataMemory	数据内存释放函数

7.1.2.2.4 算法注册接口

注册接口通常在 mws_user_algo.c 文件中调用，对实现的算法对象进行注册。

表 7-31 注册接口

函数名	简介
imdlRegisterLSSolver	注册线性求解算法
imdlUnregisterLSSolver	移除注册的线性求解算法

7.1.2.3 非线性代数系统

非线性代数系统的形式为：

$$F(x) = 0$$

其中 x 是未知量。

计算函数返回的是上述 $F(x)$ 计算得到的值，由主控层提供。算法函数是对非线性代数系统进行求解的一系列函数，由算法扩展方提供，注册后与求解器集成。

7.1.2.3.1 数据结构

数据结构由规范定义，算法扩展时可直接使用。

表 7-32 数据结构

函数名	简介
typedef void* MwsNLSSolverObj;	非线性代数系统求解算法对象
typedef void* MwsNLSPObj;	非线性代数系统求解问题对象
typedef enum MwsNLSStatus	非线性代数系统求解算法函数返回状态
typedef struct MwsNLSSolverProp	非线性代数系统求解算法属性
typedef struct MwsNLSOptions	非线性代数系统求解选项
typedef struct MwsNLSSolverFcns	非线性代数系统求解算法函数
typedef struct MwsNLSUtilFcns	非线性代数系统求解算法工具函数

7.1.2.3.2 算法函数

非线性代数系统求解算法的算法函数由扩展方实现，其中必须实现的函数由主控层调用的主体流程如下：

苏州同元软控信息技术有限公司 版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。



图 7-6 主控层调用算法函数过程

表 7-33 算法函数

函数名	简介
MwsNLSSolverCreatePtr	非线性代数系统求解算法对象创建函数
MwsNLSPProblemCreatePtr	非线性代数系统求解问题对象创建函数
MwsNLSSolvePtr	求解函数
MwsNLSPProblemDestroyPtr	求解问题对象销毁函数
MwsNLSSolverDestroyPtr	求解对象销毁函数

7.1.2.3.3 计算函数

计算函数由主控层提供，其中，非线性代数系统函数值计算函数表示非线性方程

$$F(x) = 0$$

中的函数 F 。

表 7-34 计算函数

函数名	简介
MwsNLSSysFnPtr	计算 $F(u)$ 的值, $F(u) = 0$
MwsNLSJacFnPtr	计算 $F(u)$ 的 Jacobian

7.1.2.3.4 工具函数

工具函数由主控层提供，算法层根据需要调用。

表 7-35 工具函数

函数名	简介
m_logger	日志函数
m_allocMemory	内存分配函数
m_freeMemory	内存释放函数
m_allocDataMemory	数据内存分配函数
m_freeDataMemory	数据内存释放函数

7.1.2.3.5 算法注册接口

注册接口通常在 mws_user_algo.c 文件中调用，对实现的算法进行注册。

表 7-36 注册接口

函数名	简介
imdlRegisterNLSSolver	注册非线性代数系统求解算法
imdlUnregisterNLSSolver	移除注册的非线性代数系统求解算法

7.2 平台层

平台层提供一套平台级 API 用于开放科学计算与系统建模仿真平台的计算能力。科学计算平台的计算能力包括表达式计算、函数调用等，系统建模仿真平台的计算能力包括模型编译、模型分析、模型求解、代码生成。

平台提供云服务和命令行两大类应用接口。在云平台应用中，用户可以在自己的 web 页面中调用平台提供的接口。

7.2.1 科学计算环境

科学计算环境平台层 API，支持对平台的界面、业务逻辑、数据等不同层次接口调用，支持 App 的扩展开发和集成。按功能划分为基础 API、数学 API、图形 API、App 构建 API，如下图所示。

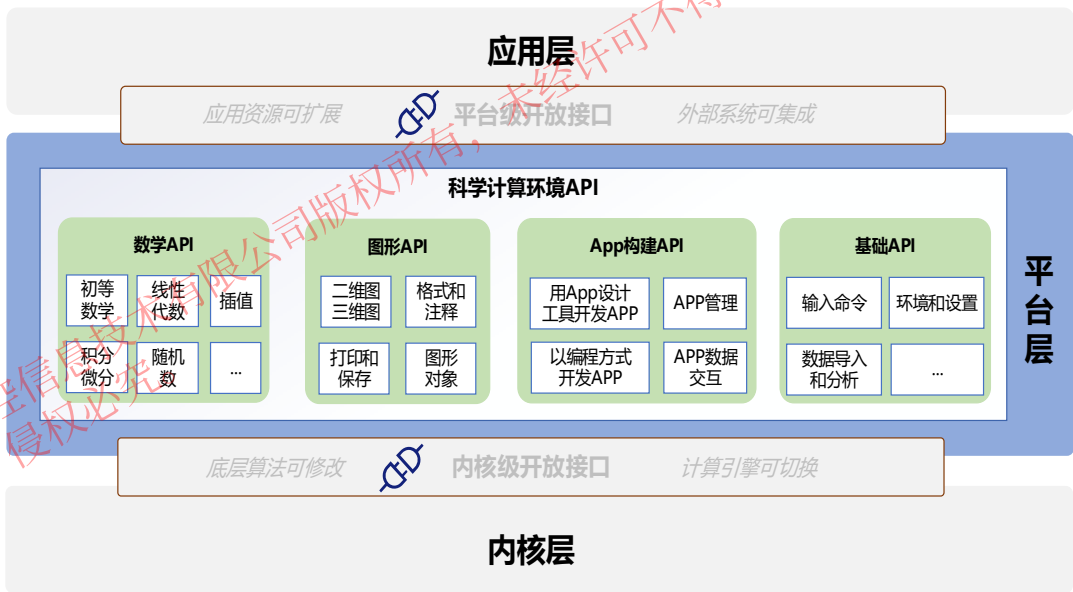


图 7-7 科学计算环境 API

- 基础 API：提供了科学计算最基础的功能，包括命令行控制，科学计算语言基础、平台环境和设置操作、数据导入导出和分析的功能、以及外部语言接入和调用的功能。
- 数学 API：提供科学计算核心的专业数学计算函数。
- 图形 API：提供可视化绘图的功能。
- App 构建 API 提供 App 开发、打包、部署、运行相关的功能。

基础 API、数学 API 和图形 API 都是 Julia 语言的函数，用户可以在自己的编程环境中，通过调用 Julia 脚本的形式来使用这些 Julia 函数。App 构建 API 中，App 管理 API 是 Syslab 提供的一套 App 管理的函数，可以在 Syslab 交互式命令环境中实现对 App 的安装、启动、卸载等管理操作；App 通信 API 支持用户在多种图形应用环境中开发的 APP 与 Syslab 平台通信互操作。

7.2.1.1 基础 API

基础 API 是编译和运行 Syslab 的函数，在 Syslab 中工作时，您可以发出创建变量和调用函数的命令。

7.2.1.1.1 输入命令

在 Syslab 中工作时，您可以发出创建变量和调用函数的命令。

表 7-37 编译和运行 Syslab 语句

函数名	简介
ans	最近计算的答案
clc	清空命令行窗口
clipboard	在目标与系统剪贴板之间复制和粘贴文本
Ctrl + D	终止 Syslab 程序（与 exit() 等效）
Ctrl + l	清空命令行窗口
Ctrl+R	打开命令历史记录窗口
diary	将命令行窗口文本记录到日志文件中
exit	终止 Syslab 程序
gethostname	获取本地计算机的主机名
import	导入整个库或者库中的模块
iskeyword	确定输入是否为 Syslab 关键字
ty_format	设置命令行窗口输出显示格式
using	加载整个库或者库中的模块，并使其可直接使用
varinfo	展示模块中的变量及大小和类型
VersionNumber	用于将字符串解析为版本号

@show	显示表达式和结果，返回结果
@time	用于执行表达式的宏，打印执行所需的时间、分配数及其字节总数

7.2.1.1.2 环境和设置

预设和设置

系统命令

表 7-38 系统命令

函数名	简介
ispc	确定版本是否适用于 Windows (PC) 平台
pause	暂时停止执行 Syslab
perl	使用操作系统可执行文件调用 Perl 脚本
system	执行操作系统命令并返回输出

平台和许可

表 7-39 平台和许可

函数名	简介
ver	Syslab 产品的版本信息

7.2.1.1.3 数据导入和分析

导入和导出数据，包括大文件；预处理数据、可视化和浏览。

数据导入和导出

文本文件和其他文件格式

低功耗 Bluetooth 通信

表 7-40 低功耗 Bluetooth 通信

函数名	简介
read_serial_port	从串行端口设备读取数据
write_serial_port	将数据写入串行端口设备

低级文件 I/O

表 7-41 低级文件 I/O

函数名	简介
fclose	关闭一个或所有打开的文件
feof	检测文件末尾
fgetl	读取文件中的行，并删除换行符
fgets	读取文件中的行，并保留换行符
fopen	打开文件或获得有关打开文件的信息
fprintf	将数据写入文本文件
fread	读取二进制文件中的数据
fscanf	读取文本文件中的数据
fwrite	将数据写入二进制文件

标准文件格式

表 7-42 标准文件格式

函数名	简介
h5info	有关 HDF5 文件的信息
h5read	从 HDF5 数据集读取数据
h5readatt	从 HDF5 文件中读取属性
importdata	从文件加载数据
textscan	从文本文件或字符串读取格式化数据

工作区变量

表 7-43 工作区变量

函数名	简介
clear	从工作区中删除项目、释放系统内存
load	将文件变量加载到工作区中
save	将工作区变量保存到文件中

描述性统计量

范围、集中趋势、标准差、方差、相关性

表 7-44 描述性统计量

函数名	简介
maxk	计算数组的 k 个最大元素
median	数组的中位数值
mink	计算数组的 k 个最小元素
mode	数组中出现次数最多的值
movsum	移动总和

大型文件和大数据

访问和处理文件集合以及大型数据集

表 7-45 大型文件和大数据

函数名	简介
add	向 KeyValue 中添加单个键-值对组

数据的预处理

表 7-46 数据的预处理

函数名	简介
fillmissing	填充缺失值
rmmissing	删除缺失的条目
standardizemissing	插入标准缺失值

7.2.1.2 数学 API

7.2.1.2.1 初等数学

三角学、指数和对数、复数值、舍入、余数、离散数学 初等数学函数包括支持算术运算（+、-、*、...）的功能、数学常量函数（Inf、pi、...）、多项式运算函数（poly、roots、...）以及特殊的数学函数（如 gamma 和 beta）。

算术运算

加、减、乘、除、幂、四舍五入 算术函数包括用于简单运算（如加法和乘法）的运算符，以及用于常见计算（如求和、移动和、取模运算和舍入）的函

数。

表 7-47 算术运算

函数名	简介
+	添加数字，追加字符串
sum	数组元素总和
cumsum	累积和
-	减法
diff	差分和近似导数
.*	乘法
*	矩阵乘法
prod	数组元素的乘积
cumprod	累积乘积
pagetimes	按页矩阵乘法
./	数组右除
.\	数组左除
^	矩阵幂
'	复共轭转置
transpose	转置向量或矩阵
pagetranspose	按页转置
pagectranspose	按页复共轭转置

三角学

结果以弧度或度为单位的正弦、余弦和相关函数。

Syslab 中的三角函数计算以弧度或度为单位的标准三角函数值、以弧度为单位的的双曲三角函数值以及每个函数的反函数。

表 7-48 三角学

函数名	简介
sin	参数的正弦，以弧度为单位
sind	参数的正弦，以度为单位
sinpi	准确地计算 $\sin(X \cdot \pi)$

asin	反正弦（以弧度为单位）
asind	反正弦（以度为单位）
sinh	双曲正弦
asinh	反双曲正弦
cos	以弧度为单位的参数的余弦
cosd	以度为单位的参数的余弦
cospi	准确计算 $\cos(X*\pi)$
acos	反余弦（以弧度为单位）
acosd	反余弦（以度为单位）
cosh	双曲余弦
acosh	反双曲余弦
tan	以弧度表示的参数的正切
tand	以度表示的参数的正切
atan	反正切（以弧度为单位）
atand	反正切（以度为单位）
tanh	双曲正切
atanh	反双曲正切
csc	输入角的余割（以弧度为单位）
cscd	以度为单位的参数的余割
acsc	反余割（以弧度为单位）
acscd	反余割（以度为单位）
csch	双曲余割
acsch	反双曲余割
sec	角的正割（以弧度为单位）
secd	参数的正割，以度为单位

asec	反正割（以弧度为单位）
sech	双曲正割
asech	反双曲正割
cot	角的余切（以弧度为单位）
cotd	以度为单位的参数的余切
acot	反余切（以弧度为单位）
acotd	反余切（以度为单位）
coth	双曲余切
acoth	反双曲余切
hypot	平方和的平方根（斜边）
deg2rad	将角从以度为单位转换为以弧度为单位
rad2deg	将角的单位从弧度转换为度
cart2pol	将笛卡尔坐标转换为极坐标或柱坐标
cart2sph	将笛卡尔坐标转换为球面坐标
pol2cart	将极坐标或柱坐标转换为笛卡尔坐标
sph2cart	将球面坐标转换为笛卡尔坐标

指数和对数

指数、对数、幂和根函数。

除了 `exp` 和 `log` 等常用函数，Syslab 还提供了其他几个相关函数，可以进行灵活的数值计算。`expm1` 和 `log1p` 函数补偿小参数中的数值舍入误差，而 `reallog`、`realpow` 和 `realsqrt` 函数将这些函数的范围限制为实数。`nthroot` 计算任意次方根，而专用函数 `pow2` 和 `nextpow2` 计算 2 的幂。

表 7-49 指数和对数

函数名	简介
<code>exp</code>	指数
<code>expm1</code>	针对较小的 x 值正确计算 $\exp(x)-1$
<code>exponent</code>	标准化浮点数的指数

log	自然对数
log10	常用对数（以 10 为底）
log1p	针对较小的 x 值正确计算 $\log(1+x)$
log2	以 2 为底的对数和浮点数分解
nextpow2	2 的更高次幂的指数
nthroot	实数的第 n 次实根
pow2	浮点数的以 2 为底的幂运算和缩放
reallog	非负实数数组的自然对数
realpow	仅实数输出的数组幂
realsqrt	非负实数数组的平方根
sqrt	平方根
cbrt	立方根

复数

实部和虚部、相位角度。

在 Syslab 中，im 表示基本虚数单位。您可以使用它们来创建复数，例如 $2im+5$ 。您还可以确定复数的实部和虚部，并计算相位和角度等其他常用值。

表 7-50 复数

函数名	简介
abs	绝对值和复数的模
abs2	绝对值和复数的模的平方
angle	相位角
complex	创建复数数组
conj	复共轭
cplxpair	将复数排序为复共轭对组
im	虚数单位
imag	复数的虚部
isreal	确定数组是否使用复数存储
real	复数的实部
sign	Sign 函数（符号函数）
unwrap	平移相位角

离散数学

质因数、阶乘、排列、有理分式、最小公倍数、最大公约数。

离散数学函数对整数 (...、-2、-1、0、1、2、...) 执行运算，或以整数返回离散输出。您可以使用这些函数来分解大数、计算阶乘、计算排列组合或求解最大公分母。

表 7-51 离散数学

函数名	简介
factor	质因数
factorial	输入的阶乘
gcd	最大公约数
isprime	确定哪些数组元素为质数
lcm	最小公倍数
nchoosek	二项式系数或所有组合
perms	所有可能的排列
primes	小于等于输入值的质数
prevprime	小于等于输入值的质数
nextprime	大于等于输入值的质数
nthprime	返回第 n 个质数
binomial	二项式系数或所有组合
Rational	有理输出
rationalize	将浮点数 x 近似为具有给定整数类型分量的有理数。结果将与 x 相差不超过 tol
matchpairs	求解线性分配问题
rats	有理输出

多项式

曲线拟合、根、部分分式展开 多项式是包含非负整数指数的单个变量的方程。

表 7-52 多项式

函数名	简介
detrend	去除多项式趋势
poly	具有指定根的多项式或特征多项式

polyeig	多项式特征值问题
residue	部分分式展开（部分分式分解）
roots	多项式根
polyval	多项式计算
polyvalm	矩阵多项式计算
polyint	多项式积分
polyder	多项式微分
polyreduce	去除多项式前系数为 0 的无用项
polyfit polyfits2 polyfits3	多项式曲线拟合

特殊函数

Bessel、Legendre、椭圆、误差、gamma 和其他函数。

特殊函数是一组在实际应用中经常出现的著名数学函数。

Bessel 函数

表 7-53 Bessel 函数

函数名	简介
airy	Airy 函数
airyai	第一类 airy 函数
airybi	第二类 airy 函数
besselh	第三类 Bessel 函数（Hankel 函数）
besseli	第一类修正 Bessel 函数
besselj	第一类 Bessel 函数
besselk	第二类修正 Bessel 函数
bessely	第二类 Bessel 函数

Beta 函数

表 7-54 Beta 函数

函数名	简介
beta	Beta 函数
beta_inc	不完全 beta 函数

betainc	不完全 beta 函数
beta_inc_inv	beta 逆累积分布函数
betaincinv	beta 逆累积分布函数
logbeta	beta 函数的对数
betaln	beta 函数的对数

误差函数

表 7-55 误差函数

函数名	简介
erf	误差函数
erfc	补余误差函数
erfcinv	逆补余误差函数
erfcx	换算补余误差函数
erfinv	逆误差函数
fresnelc	菲涅尔余弦积分函数
fresnels	菲涅尔正弦积分函数

gamma 函数

表 7-56 gamma 函数

函数名	简介
gamma	Gamma 函数
gamma_inc	不完全 gamma 函数
gamma_inc_inv	逆不完全 gamma 函数
gamma_incinv	逆不完全 gamma 函数
loggamma	gamma 函数的对数
gammaln	gamma 函数的对数
digamma	digamma 函数
trigamma	trigamma 函数
polygamma	polygamma 函数
psi	digamma 和 polygamma 函数

椭圆积分

表 7-57 椭圆积分

函数名	简介
ellipj	Jacobi 椭圆积分
ellipke	第一类和第二类完全椭圆积分
Elliptic.E	第二类完全和不完全椭圆积分
Elliptic.F	第一类不完全椭圆积分
Elliptic.K	第一类完全椭圆积分
Elliptic.Pi	第三类完全和不完全椭圆积分
ellipticCE	第二类互补完全椭圆积分
ellipticCK	第一类互补完全椭圆积分

Jacobi 椭圆积分

表 7-58 Jacobi 椭圆积分

函数名	简介
Jacobi.cd	Jacobi CD 椭圆函数
Jacobi.cn	Jacobi CN 椭圆函数
Jacobi.cs	Jacobi CS 椭圆函数
Jacobi.dc	Jacobi DC 椭圆函数
Jacobi.dn	Jacobi DN 椭圆函数
Jacobi.ds	Jacobi DS 椭圆函数
Jacobi.nc	Jacobi NC 椭圆函数
Jacobi.nd	Jacobi ND 椭圆函数
Jacobi.ns	Jacobi NS 椭圆函数
Jacobi.sc	Jacobi SC 椭圆函数
Jacobi.sd	Jacobi SD 椭圆函数
Jacobi.sn	Jacobi SN 椭圆函数
Jacobi.am	雅克比振幅函数

其他特殊函数

表 7-59 其他特殊函数

函数名	简介
expint	指数积分函数
legendre	连带 Legendre 函数
lambertw	朗博 w 函数 (又称欧米茄函数)
heaviside	单位阶跃函数

常量和测试矩阵

Pi、非数字、无穷；Hadamard 矩阵、伴随矩阵、帕斯卡矩阵和其他专用矩阵。

表 7-60 常量和测试矩阵

函数名	简介
eps	浮点相对精度
Inf	创建所有值均为 Inf 的数组
pi	圆的周长与其直径的比率
NaN	创建所有值均为 NaN 的数组
isfinite	确定哪些数组元素为有限
isinf	确定哪些数组元素为无限值
isnan	确定哪些数组元素为 NaN
compan	伴随矩阵
TestArrays	测试矩阵生成模块
gallery	测试矩阵
hadamard	Hadamard 矩阵
hankel	Hankel 矩阵
hilb	Hilbert 矩阵
invhilb	Hilbert 矩阵的逆矩阵
magic	幻方矩阵
pascal	帕斯卡矩阵
rosser	典型对称特征值测试问题

toeplitz	托普利茨矩阵
vander	Vandermonde 矩阵
wilkinson	Wilkinson 的特征值测试矩阵

7.2.1.2.2 线性代数

线性方程、特征值、奇异值、分解、矩阵运算、矩阵结构。

Syslab 中的线性代数函数提供快速且数值稳健的矩阵计算。功能包括各种矩阵分解、线性方程求解、计算特征值或奇异值等。

线性方程

解算多种类型的线性方程。

表 7-61 线性方程

函数名	简介
inv	矩阵求逆
pinv	Moore-Penrose 伪逆
\	求解关于 x 的线性方程组 $Ax = B$
/	求解关于 x 的线性方程组 $xA = B$
linsolve	对线性方程组求解
lsconv	存在已知协方差的最小二乘解
lsqnonneg	求解非负线性二乘问题
sylvester	求解关于 X 的 Sylvester 方程 $AX + XB = C$

特征值和奇异值

特征值、特征向量和奇异值的计算。

表 7-62 特征值和奇异值

函数名	简介
eigvals	返回 A 的特征值
eigvecs	返回 A 的特征向量
eigen	计算 A 的特征值分解
balance	对角线缩放以提高特征值准确性
svd	计算 A 的奇异值分解 (SVD)

svdvals	按降序返回 A 的奇异值
svdsketch	计算低秩矩阵草图的 SVD
ordeig	拟三角矩阵的特征值
ordschur	矩阵 $A = ZTZ'$ 的 Schur 分解 F 重新排序
polyeig	多项式特征值问题
hessenberg	矩阵的 Hessenberg 形式
schur	Schur 分解
schur!	Schur 分解
rsf2csf	将实数 Schur 形式变换为复数 Schur 形式
cdf2rdf	拟三角矩阵的特征值

矩阵分解

分解矩阵

表 7-63 矩阵分解

函数名	简介
lu	LU 矩阵分解
ldlt	实对称三对角矩阵 S 的 LDLt 分解
cholesky	Cholesky 分解
cholupdate	Cholesky 分解的秩 1 更新
qrdelete	从 QR 分解中删除列或行
qrinsert	将列或行插入 QR 分解
qrupdate	QR 分解的秩 1 更新
planerot	Givens 平面旋转

矩阵运算

矩阵的计算与转置。

表 7-64 矩阵运算

函数名	简介
transpose	转置向量或矩阵
'	复共轭转置
adjoint	复共轭转置

*	矩阵乘法
^	矩阵幂
sqrt	矩阵平方根
exp	矩阵平方根
log	矩阵对数
funm	计算常规矩阵函数
kron	Kronecker 张量积
cross	叉积
dot	点积

矩阵结构

矩阵的带宽与结构。

表 7-65 矩阵结构

函数名	简介
bandwidth	矩阵的上下带宽
tril	矩阵的下三角形部分
triu	矩阵的上三角形部分
isbanded	确定矩阵是否在特定带宽范围内
isdiag	确定矩阵是否为对角矩阵
ishermitian	确定矩阵是 Hermitian 矩阵
issymmetric	确定矩阵是对称矩阵
istril	确定矩阵是否为下三角矩阵
istriu	确定矩阵是否为上三角矩阵

矩阵属性

矩阵的属性。

表 7-66 矩阵属性

函数名	简介
rref	简化的行阶梯形矩阵（Gauss-Jordan 消元法）
opnorm	矩阵范数
cond	逆运算的条件数

rcond	条件数倒数
condskeel	相对条件数
condeig	逆运算的条件数
det	矩阵行列式
null	矩阵的零空间
orth	适用于矩阵范围的标准正交基
rank	矩阵的秩
tr	对角线元素之和
subspace	两个子空间之间的角度
norm	计算范数

7.2.1.2.3 随机数生成

种子、分布、算法。

使用 rand 和 randn 函数创建伪随机数序列，使用 randperm 函数创建随机置换整数向量。

创建随机值

创建随机数数组。

表 7-67 创建随机值

函数名	简介
mt19937ar	mt19937ar 随机种子算法
rand	均匀分布的随机数
randi	均匀分布的伪随机整数
randn	标准正态分布的随机数
randg	标准高斯分布的随机数
randperm	随机排列
bitrand	生成一个随机布尔值的 BitArray
randpermk	整数的随机排列

7.2.1.2.4 插值

网格和散点数据插值、数据网格化、分段多项式

插值是在一组已知数据点的范围内添加新数据点的技术。您可以使用插值来填充缺失的数据、对现有数据进行平滑处理以及进行预测等。

一维插值和网格插值

表 7-68 一维插值和网格插值

函数名	简介
interp1	一维插值
interp2	二维插值
interp3	三维插值
interp _n	n 维插值
griddedInterpolant	网格插值
akima	Akima 分段三次 Hermite 插值
interpft	一维插值（FFT 方法）
makima	修正 Akima 分段三次 Hermite 插值
pchip	分段三次 Hermite 插值多项式 (PCHIP)
interpolate	插值
extrapolate	外插
LagrangeInterp	拉格朗日插值
NewtonInterp	牛顿插值
ConstantInterpolation	最邻近插值
LinearInterpolation	线性插值
CubicSplineInterpolation	三次样条插值
mkpp	生成分段多项式
unmkpp	提取分段多项式详细信息
ppval	计算分段多项式
padecof	时滞的 Padé 逼近

网格创建

网格数据集插值简介

表 7-69 网格创建

函数名	简介
-----	----

meshgrid2	创建二维网格
meshgrid3	创建三维网格
ndgrid	N 维空间中的矩形网格

散点插值

散点数据集插值

表 7-70 散点插值

函数名	简介
scatteredInterpolant	散点数据插值
evaluate	计算插值方法在数据点处的值
griddata	对二维或三维散点数据插值

7.2.1.2.5 数值积分和微分方程

开始你的第一个微分方程

简单一阶微分方程

表 7-71 简单一阶微分方程

函数名	简介
Begin Your First ODE	微分方程图像应用

微分方程定义

微分方程简介与用法

表 7-72 微分方程定义

函数名	简介
ODEProblem	定义 ODE 问题
DynamicalODEProblem	定义动力学常微分方程
SecondOrderODEProblem	定义 二阶 ODE 问题
HamiltonianProblem	相应的哈密顿量定义运动方程的简单方法
SplitODEProblem	定义 拆分 ODE 问题
DAEProblem	定义微分代数方程问题

积分控制

初始化微分方程问题

表 7-73 积分控制

函数名	简介
积分器结构体	积分器结构体具体状态
init	初始化微分方程问题
step!	在积分器上执行一个（成功）步骤,并覆盖原有数据
check_error	检查积分器的状态并返回返回代码之一
set_t!	将积分器的当前时间点设置为 t
set_u!	将积分器的当前状态设置为 u
savevalues!	尝试保存当前时间点的状态和时间变量，或者在适当的时候使用插值保存点
get_proposed_dt	获取下一个时间步长的建议 dt
set_proposed_dt!	为下一个时间步设置建议的 dt
add_tstop!	在时间 t 添加一个 tstop
add_saveat!	在 t 添加一个保存时间点
reinit!	以新值重新开始积分
get_du	返回 t 处的导数

重构

微分方程重构

表 7-74 微分方程重构

函数名	简介
remake	问题重构

求解

微分方程求解

表 7-75 微分方程求解

函数名	简介
solve	微分方程问题求解

数值积分和微分

求积、二重积分和三重积分以及多维导数

表 7-76 求积、二重积分和三重积分以及多维导数

函数名	简介
integral	数值积分
integral2	对二重积分进行数值计算
integral3	对三重积分进行数值计算
quad	以自适应 Simpson 积分法计算数值积分
quad2d	计算二重数值积分 - tiled 方法
quad3d	对三重积分进行数值计算
quadgk	计算数值积分 - 高斯-勒让德积分法
cumtrapz	累积梯形数值积分
trapz	梯形数值积分
del2	离散拉普拉斯算子
diff	差分
gradient	数值梯度
polyder	多项式微分
polyint	多项式积分

7.2 1.2.6 傅里叶分析和滤波

傅里叶变换

傅里叶变换简介

表 7-77 傅里叶变换

函数名	简介
fft	傅里叶变换 (一维、二维、多维)
fftshift	将零频分量移到频谱中心
ifft	傅里叶逆变换 (一维、二维、多维)
ifftshift	将零频分量移到频谱中心
nextpow2	2 的更高次幂的指数
nufft	非均匀快速傅里叶变换

nufftn	N 维非均匀快速傅里叶变换
ty_fft	傅里叶变换 (一维、二维、多维)
ty_ifft	傅里叶逆变换 (一维、二维、多维)

滤波

表 7-78 数字滤波器

函数名	简介
filter1	1 维数字滤波器
filter2	二维数字滤波器

卷积

表 7-79 卷积和多项式

函数名	简介
conv	卷积和多项式
conv2	二维卷积
convn	N 维卷积
deconv	去卷积和多项式除法

7.2.1.2.7 稀疏数组

初等稀疏矩阵、重新排序算法、迭代法、稀疏线性代数使用稀疏矩阵可极大地减少存储数据所需的内存量。

创建

表 7-80 创建稀疏数组

函数名	简介
spalloc	为稀疏矩阵分配空间
spzeros	创建长度为 m 的稀疏向量或大小为 m x n 的稀疏矩阵
speye	稀疏单位矩阵
sprandsym	稀疏对称随机矩阵
spdiags	创建稀疏带状对角矩阵
sprand	创建一个随机长度 m 稀疏向量或 m x n 稀疏矩阵
sparse	创建稀疏矩阵

sprandn	创建一个长度为 m 的稀疏正态分布随机向量或大小为 $m \times n$ 的稀疏正态分布的矩阵
spconvert	从稀疏矩阵外部格式导入

操作

表 7-81 操作稀疏数组

函数名	简介
issparse	确定输入是否为稀疏矩阵
nnz	非零矩阵元素的数目
nonzeros	非零矩阵元素
spones	将非零稀疏矩阵元素替换为一
find	查找非零元素的索引和值
full	将稀疏矩阵转换为满存储
permute	将矩阵元素进行置换
dropzeros	生成矩阵或向量的副本，并从该副本中删除存储的数字零
dropzeros!	删除矩阵或向量中存储的数字零
droptol!	删除 A 中绝对值小于或等于 tol 的存储值
nzrange	将索引范围返回到稀疏矩阵列的结构非零值
rowvals	返回 A 的行索引的向量
blockdiag	以块对角方式连接矩阵
findnz	返回一个元组包含 A 中存储的（“结构上非零”）值的索引和对应值的向量
map	将函数应用于非零稀疏矩阵元素
nzmax	为非零矩阵元素分配的存储量
spfun	将函数应用于非零稀疏矩阵元素

重排序算法

表 7-82 稀疏数组重排序算法

函数名	简介
amd	近似最小度置换
colamd	列近似最小度排列
colperm	基于非零项计数的稀疏列置换

symamd	对称近似最小度置换
symrcm	稀疏反向 Cuthill-McKee 排序

结构分析

表 7-83 稀疏数组结构分析

函数名	简介
sprank	结构秩
etree	消去树
spaugment	构造最小二乘增广方程组
DMDecomposition	Dulmage-Mendelsohn 分解模块
treelayout	设置树或森林的布局
treeplot	绘制树形图
etreeplot	绘制消去树
gplot	绘制邻接矩阵中的节点和边
unmesh	unmesh

迭代算法

表 7-84 稀疏矩阵迭代算法

函数名	简介
bicg	求解线性方程组 - 双共轭梯度法
bicgstab	求解线性方程组 - 稳定双共轭梯度法
bicgstabl	求解线性方程组 - 稳定双共轭梯度 (I) 法
cg	求解线性方程-预条件共轭梯度法
cgs	求解线性方程组-共轭梯度二乘法
qmr	求解线性方程组-拟最小残差法
lsqr	求解线性方程组-最小二乘法
jacobi	求解线性方程-Jacobid 迭代算法 (基础算法)
symmlq	求解线性方程组 - 对称的 LQ 方法
gmres	求解线性方程-广义最小残差法
tfqmr	求解线性方程组 - 无转置拟最小残差法

7.2.1.2.8 图和网络算法

表示网络连接的图形，该类图形广泛应用于各种物理、生物和信息系统。您可以使用图形表示大脑中的神经元、航空公司的飞行模式及更多领域的相关内容。图形的结构由“节点”和“边”组成。每个节点表示一个实体，每个边表示两个节点之间的连接。有关详细信息，请参阅有向图和无向图。

构造

表 7-85 构造

函数名	简介
Graph	具有无向边的图
DiGraph	具备有向边的图

修改节点和边

表 7-86 修改节点和边

函数名	简介
addnode	将新节点添加到图
rmnode	从图中删除节点
addedge	向图添加新边
rmedge	从图中删除边
flipedge	反转边的方向
numnodes	图中节点的数量
numedges	图中边的数量
findnode	定位图中的节点
findedge	定位图中的边
edgecount	两个节点之间的边数
subgraph	提取子图

分析结构

表 7-87 分析结构

函数名	简介
conncomp	图的连通分量
biconncomp	双连通图分量

condensation	图凝聚
bctree	块割点树图
toposort	有向无环图的拓扑顺序
isdag	确定图是否为无环
transreduction	传递归约
transclosure	传递闭包
ismultigraph	确定图是否具有多条边
simplify	将多重图简化为简单图

遍历、最短路径和循环

表 7-88 遍历、最短路径和循环

函数名	简介
dfsearch	深度优先图搜索

矩阵表示

表 7-89 矩阵表示

函数名	简介
conncomp	图的连通分量
biconncomp	双连通图分量
laplacian	图凝聚

节点信息

表 7-90 节点信息

函数名	简介
degree	图节点的度
neighbors	图节点的相邻节点
indegree	节点的入度
outdegree	节点的出度
predecessors	前趋节点
successors	后继节点
inedges	进入节点的入向边
outedges	节点的出向边

7.2.1.3 图形 API

二维和三维绘图、图像、动画

图形函数包括二维和三维绘图函数，用于以可视化形式呈现数据和通信的结果。以交互方式或编程方式自定义绘图。

7.2.1.3.1 二维和三维图

绘制连续、离散、曲面以及三维体数据图

使用绘图以可视化形式呈现数据。例如，您可以比较多组数据、跟踪数据随时间所发生的更改或显示数据分布。

线图

线图、对数图和函数图

在比较数据集或跟踪数据随时间变化方面，线图是一个非常有用的方法。您可以使用线性刻度或对数刻度在二维或三维视图中绘制数据。此外，还可以在特定区间上绘制表达式或函数。

线图

表 7-91 线图

函数名	简介
plot	二维线图
plot3	三维点或线图
stairs	阶梯图
errorbar	含误差条的线图
ezplot	易用的函数绘图函数
area	填充区二维绘图

对线图

表 7-92 对线图

函数名	简介
loglog	双对数刻度图
semilogx	半对数图(x 轴有对数刻度)
semilogy	半对数图(y 轴有对数刻度)

函数图

表 7-93 函数图

函数名	简介
fplot	绘制表达式或函数
fimplicit	绘制隐函数
fplot3	三维参数化曲线绘图函数

数据分布图

直方图、饼图、文字云等

使用直方图、饼图等将数据分布可视化。例如，使用直方图将数据分组到各个 bin 中并显示每个 bin 中的元素数量。

直方图

表 7-94 直方图

函数名	简介
hist	直方图
histogram	直方图
histogram2	二元直方图
morebins	增加直方图的 bin 数量
fewerbins	减少直方图 bin 数量
histcounts	直方图 bin 计数
histcounts2	二元直方图 bin 计数
boxchart	创建箱线图

散点图

表 7-95 散点图

函数名	简介
scatter	散点图
scatter3	三维散点图
spy	可视化矩阵的稀疏模式

plotmatrix	散点图矩阵
------------	-------

饼图和热图

表 7-96 饼图和热图

函数名	简介
pie	饼图
heatmap	创建热图
sortx	对热图行中的元素进行排序
sorty	对热图列中的元素进行排序
wordcloud	使用文本数据创建文字云图

离散数据图

条形图、散点图等

使用条形图或针状图等将离散数据可视化。例如，可以创建垂直或水平条形图，其中条形长度与它们所代表的值成比例。

条形图

表 7-97 条形图

函数名	简介
bar	条形图
barh	水平条形图
bar3	绘制三维条形图
bar3h	绘制水平三维条形图
pareto	帕累托图

针状图

表 7-98 针状图

函数名	简介
stem	针状图
stem3	三维针状图

散点图

表 7-99 散点图

函数名	简介
scatter	散点图
scatter3	三维散点图

阶梯图

表 7-100 阶梯图

函数名	简介
stairs	阶梯图

极坐标图

在极坐标中绘图

对数据绘图

表 7-101 对数据绘图

函数名	简介
polar	极坐标图
polarplot	在极坐标中绘制线条
polarscatter	极坐标中的散点图
polarhistogram	极坐标中的直方图
ezpolar	易用的极坐标绘图函数

自定义极坐标区

表 7-102 自定义极坐标区

函数名	简介
rlim	在极坐标中绘制线条
thetalim	极坐标中的散点图
rticks	极坐标中的直方图
thetaticks	易用的极坐标绘图函数
rticklabels	在极坐标中绘制线条

thetaticklabels	极坐标中的散点图
rtickformat	极坐标中的直方图
thetatickformat	易用的极坐标绘图函数
rtickangle	在极坐标中绘制线条
polaraxes	极坐标中的散点图

等高线图

二维和三维等值线图

表 7-103 等高线图

函数名	简介
contour	矩阵的等高线图
contourf	填充的二维等高线图
contour3	三维等高线图
clabel	为等高线图添加高程标签
fcontour	绘制等高线

图曲面、体积和多边形

网格曲面和三维体数据、非网格多边形数据

表 7-104 图曲面、体积和多边形

函数名	简介
surf	曲面图
surfc	曲面图下的等高线图
meshc	网格曲面图下的等高线图
surface	基本曲面图
mesh	网格曲面图
fsurf	绘制三维曲面
fmesh	绘制三维网格图
peaks	包含两个变量的示例函数

cylinder	创建圆柱
ellipsoid	创建椭圆体
sphere	创建球面
ribbon	条带图
flow	包含三个变量的简单函数
pcolor	伪彩图
plt_fill	填充的二维多边形
patch	绘制一个或多个填充多边形区域

向量场

罗盘状图、羽状图、箭状图和流线图

表 7-105 向量场

函数名	简介
feather	绘制速度向量
quiver	箭头图或速度图
compass	绘制从原点发射出的箭头
quiver3	三维箭头图或速度图
streamline	根据二维或三维向量数据绘制流线图

7.2.1.3.2 格式和注释

添加标签、调整颜色、定义坐标轴范围、应用光照或透明度、设置照相机视图。

自定义绘图的外观，以传递其他信息或增强数据的显示。例如，您可以添加标题和标签、更改坐标轴范围或添加网格线。此外，也可以通过在一个坐标区中显示所有数据或在单幅图窗中使用多个坐标区，将多组数据合并在一幅图窗中。

标签和注释

添加标题、轴标签、信息性文本以及其他图表注释。

添加标题、坐标轴标签或向图添加注释以帮助传达重要信息。可以创建图例来标记绘制的数据系列，或在数据点旁边添加描述性文本。此外，还可以创

建注释，例如突出显示特定区域数据的矩形、椭圆形、箭头、垂直线或水平线。

标签

表 7-106 标签

函数名	简介
title	添加标题
sgtitle	在子图网格上添加标题
xlabel	为 x 轴添加标签
ylabel	为 y 轴添加标签
zlabel	为 z 轴添加标签
legend	在坐标区上添加图例

注释

表 7-107 注释

函数名	简介
text	向数据点添加文本说明
gtext	使用鼠标将文本添加到图窗
xline	具有常量 x 值的垂直线
yline	具有常量 y 值的水平线
annotation	创建注释
datacursormode	开启数据提示
datatip	创建数据提示
line	创建基本线条
rectangle	创建带有尖角或圆角的矩形
ginput	标识坐标区坐标

坐标区外观

修改坐标轴范围和刻度值、添加网格线、合并多个绘图。
可以通过更改范围、控制刻度线的位置、格式化刻度标签或添加网格线来自定义坐标区。您也可以通过在同一图窗中使用独立的坐标区，或通过相同

的坐标区中合并绘图（可选择添加第二个 y 轴），来合并多个绘图。

坐标区范围和纵横比

表 7-108 坐标区范围和纵横比

函数名	简介
xlim	设置或查询 x 坐标轴范围
ylim	设置或查询 y 坐标轴范围
zlim	设置或查询 z 坐标轴范围
axis	设置坐标轴范围和纵横比
daspect	控制沿每个轴的数据单位长度
pbaspect	控制每个轴的相对长度

网格线、刻度值和标签

表 7-109 网格线、刻度值和标签

函数名	简介
grid	显示或隐藏坐标区网格线
xticks	设置或查询 x 轴刻度值
yticks	设置或查询 y 轴刻度值
zticks	设置或查询 z 轴刻度值
xticklabels	设置或查询 x 轴刻度标签
yticklabels	设置或查询 y 轴刻度标签
zticklabels	设置或查询 z 轴刻度标签
xtickformat	指定 x 轴刻度标签格式
ytickformat	指定 y 轴刻度标签格式
ztickformat	指定 z 轴刻度标签格式
xtickangle	旋转 x 轴刻度标签
ytickangle	旋转 y 轴刻度标签
ztickangle	旋转 z 轴刻度标签

多个绘图

表 7-110 多个绘图

函数名	简介
hold	添加新绘图时保留当前绘图
yyaxis	创建具有两个 y 轴的图
legend	在坐标区上添加图例
colororder	为可视化多个数据序列设置色序
subplot	在各个分块位置创建坐标区

清除或创建坐标区

表 7-111 清除或创建坐标区

函数名	简介
cla	清除坐标区
plt_axes	创建笛卡尔坐标区
figure	创建图窗窗口

颜色图

条形图、散点图等
使用条形图或针状图等将离散数据可视化。例如，可以创建垂直或水平条形图，其中条形长度与它们所代表的值成比例。

颜色图和颜色栏

表 7-112 颜色图和颜色栏

函数名	简介
colormap	查看并设置当前颜色图
colorbar	显示色阶的颜色栏

颜色空间

表 7-113 颜色空间

函数名	简介
hsv2rgb	将 HSV 颜色转换为 RGB

rgb2hsv	将 RGB 颜色转换为 HSV
---------	-----------------

预定义颜色图

表 7-114 预定义颜色图

函数名	简介
parula	parula 颜色图数组
hsv	HSV 颜色图数组
hot	hot 颜色图数组
cool	冷色颜色图数组
spring	Spring 颜色图数组
summer	Summer 颜色图数组
autumn	Autumn 颜色图数组
winter	Winter 颜色图数组
gray	gray 颜色图数组
bone	bone 颜色图数组
copper	copper 颜色图数组
pink	粉色颜色图数组
lines	线条颜色图数组
jet	Jet 颜色图数组
prism	Prism 颜色图数组
flag	flag 颜色图数组

三维场景控制

添加光源、设置对象透明度、控制相机视图

表 7-115 三维场景控制

函数名	简介
plt_view	相机视线
alpha	向坐标区中的对象添加透明度

7.2.1.3.3 打印和保存

表 7-116 打印和导出为标准文件格式

函数名	简介
plt_print	打印图窗或保存为特定文件格式
saveas	将图窗保存为特定文件格式

7.2.1.3.4 图形对象

图形对象属性

查看和设置图形对象属性、定义默认值

表 7-117 图形对象属性

函数名	简介
get	查询图形对象属性
set	设置图形对象属性

图形对象的标识

查找、复制和删除图形对象

表 7-118 图形对象的标识

函数名	简介
gca	当前坐标区或图
gcf	当前图窗的句柄
isgraphics	对有效的图形对象句柄为 True

图形对象编程

对对象数组进行比较、有效性测试、预分配和使用

表 7-119 图形对象编程

函数名	简介
clf	清空图窗
cla	清除坐标区
plt_close	删除指定图窗

指定图形输出的目标

控制目标图窗和轴以及如何更新这些对象

表 7-120 指定图形输出的目标

函数名	简介
hold	添加新绘图时保留当前绘图
ishold	当前保留状态
clf	清空图窗
cla	清除坐标区

7.2.1.4 App 构建 API

7.2.1.4.1 App 管理

定义了一套在科学计算环境中安装、卸载、运行 Apps 的 Julia 语言接口。

表 7-121 App 管理

序号	函数名	简介
1	init_syslabapp	初始化 App 环境
2	AppInfo	App 模型定义
3	install	注册并安装 App
4	uninstall	卸载名称为 name 的 App
5	get_apps	查询用户注册的所有 App 列表信息
6	get_app	查询名称为 name 的 App 的信息
7	start	启动名称为 name 的 App
8	disable	禁用名称为 name 的 App
9	enable	启用名称为 name 的 App

7.2.1.4.2 App 通信

平台提供与 Syslab 通信的 Sdk，支持 App 从 Syslab 工作空间中获取数据、将数据写入 Syslab 工作空间、运行 julia 脚本并获取结果。

表 7-122 App 通信

序号	函数名	简介
----	-----	----

1	MwGetVariables	初始化 App 环境
2	MwGetValue	查询工作区变量 var 的值
3	MwRunScript	在工作区运行脚本，生成结果
4	MwDeleteVar	删除工作区单个变量 var

7.2.2 系统建模仿真环境

系统建模仿真环境平台层 API 是 MWORKS.Sysplorer 供开发者和外部系统调用的标准接口。按照 workflow 分为模型文件操作、模型参数操作、模型属性获取、元素及属性判定、模型属性查找、编译仿真、结果查询、图形组件类和系统配置共 9 类 API。

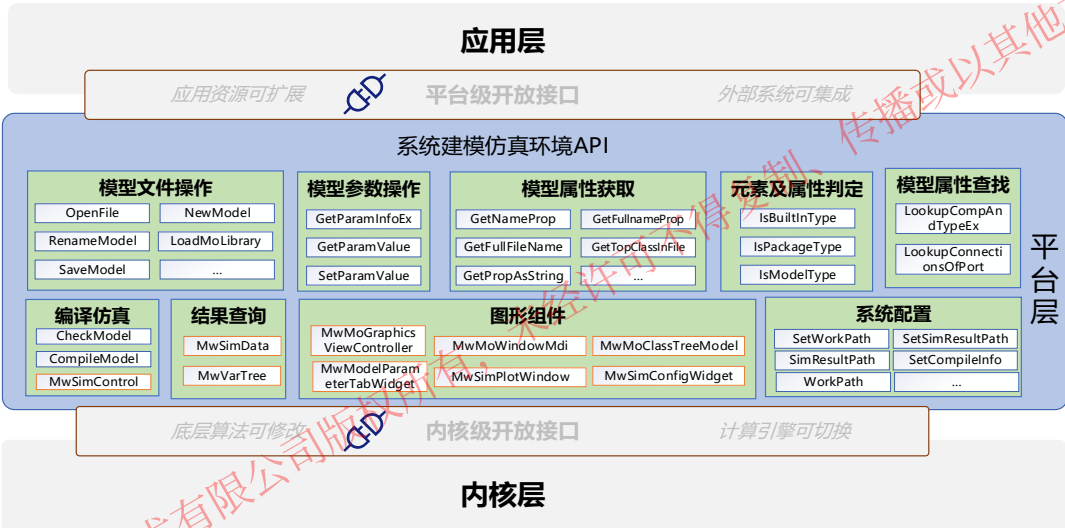


图 7-8 平台层-系统建模仿真环境接口模块

其中除图形组件模块外，其他八大模块均可在无界面情况下对模型进行操作、编译、仿真、数据后处理。

图形组件模块，为用户提供了模型可视化界面，包括显示模型视图、建模操作、仿真设置界面、后处理数据显示界面、曲线显示界面，用户可结合其他几大模块搭建一个完整的带界面的模型处理软件。

7.2.2.1 模型文件操作

模型文件操作，主要为对模型文件进行新建、打开、加载、重命名、保存、卸载、复制模型等操作。

表 7-123 模型文件操作

函数名	简介
OpenFile	打开模型文件

NewModel	新建模型
RenameModel	为模型重命名
LoadMoLibrary	加载模型库(mo)
SaveModel	将修改内容保存到模型文件中
UnloadModel	卸载已加载或打开的模型
DuplicateModel	复制模型

7.2.2.2 模型参数操作

模型参数操作主要为获取模型参数相关信息和值，并可修改模型相关参数值。

表 7-124 模型参数操作

函数名	简介
GetParamInfoEx	获取参数信息
GetParamValue	获取模型参数值
SetParamValue	设置模型参数值

7.2.2.3 模型属性获取

对模型内部属性信息进行获取如获取模型的键值，名称，全名，以及模型的父类，以及模型中的注解等属性获取。

表 7-125 模型属性获取

函数名	简介
GetKeyByTypeName	根据模型的名称获取模型键值
GetNestedCompKeys	获取模型下的所有组件键值
GetFullnameProp	获取元素的全名
GetNameProp	获取模型或元素的名称
GetFullFileName	获取模型所在文件路径

GetPropAsString	获取元素描述
GetTopClassInFile	获取文件中的顶层类键值
GetTopClassInFileByKey	获取顶层父类的键值

7.2.2.4 元素及属性判定

元素及属性判定为对模型的类型进行判定，从而确定模型的类型。

表 7-126 元素及属性判定

函数名	简介
IsBuiltInType	判断是否是内置类型
IsPackageType	判断是否是 package 类型
IsModelType	判断是否是 model 类型

7.2.2.5 模型属性查找

模型属性查找为在模型中查找相关属性，如查找模型中所有组件键值的列表等。

表 7-127 模型属性查找

函数名	简介
LookupCompAndTypeEx	查找模型中组件类型的键值
LookupConnectionsOfPort	查找端口连线

7.2.2.6 编译仿真

对模型实现底层检查模型文本、编译模型操作。

表 7-128 编译仿真

函数名	简介
CheckModel	检查模型文本
CompileModel	编译模型

7.2.2.6.1 仿真控制类

仿真控制类，负责控制仿真的开始、暂停、继续、单步执行、停止等。

表 7-129 仿真控制类

函数名	简介
RebindSimData	绑定仿真数据
GetSimData	获取仿真数据
IsSimIdle	判断仿真是否空闲
IsSimPaused	判断仿真是否暂停
IsSimRunning	判断仿真是否正在运行中
IsSimPausing	判断仿真是否正在暂停中
IsSimStarting	判断仿真是否正在启动中
IsSimLocked	判断仿真是否占用
StartSimulate	开始仿真
PauseSimulate	暂停仿真
ResumeSimulate	继续仿真
SimulateNextStep	仿真执行一步
StopSimulate	停止仿真
KillSimulate	强行终止仿真
SigBeforeSimStart	仿真启动信号
SigSimPaused	仿真暂停信号
SigSimStopped	仿真停止信号
SigSimResumed	仿真继续信号
SigSimTime	仿真时间点信号
SigSimLog	仿真日志信号
SigSimStep	仿真单步信号

7.2.2.7 结果数据查询

7.2.2.7.1 仿真结果类

模型仿真结果类，包含模型所有变量的仿真结果数据。

表 7-130 仿真结果类

函数名	简介
ApplyExperimentData	应用仿真设置
GetVarTreeRoot	获取根节点
InitializeSimInst	初始化仿真实例
GetVarData	读取结果变量

7.2.2.7.2 结果变量树

结果变量树节点类，用于获取仿真结果变量树结构，一般不主动创建该对象实例。

表 7-131 结果变量树

函数名	简介
Name	变量名字
GetFullName	获取变量全名
Parent	获取父节点
Description	获取描述
Unit	单位
Children	孩子节点

7.2.2.8 图形组件

7.2.2.8.1 模型视图管理

模型视图管理类，负责显示模型的图标视图，组件视图和文本视图，并提

供模型编辑功能。

表 7-132 模型视图管理

函数名	简介
CloseMoWindow	关闭模型窗口
OpenMoWindow	打开模型窗口
CloseCurrentWindow	关闭当前窗口
CloseAllWindows	关闭所有窗口
SetMdiInterface	设置视图接口
SetClassDirty	设置脏标
GetCurrentClassKey	获取当前模型 key
SaveCurrentWindow	保存当前模型
SigUpdate	模型视图更新信号
SigClassDirtyChanged	脏标变化信号
SigAppendClass	添加模型信号
SigRemoveClass	移除模型信号
SigReplaceClass	替换模型信号

7.2.2.8.2 中央视图控件

中央视图控件，负责管理模型视图窗口，创建后设置到 MwMoGraphicsViewController 中。该类只负责提供视图，用户无需对该窗口有其他操作，所有对窗口操作可通过 MwMoGraphicsViewController 完成。

7.2.2.8.3 模型树数据

模型树数据类，用于将内核的模型数据操作同步到界面模型中，利用 QT 所提供的 MVD 设计模式，提供了一个现成的 Model 自动管理数据，用户只需实现自己的 View 视图用于展示数据并将 Model 设置到 View 视图中。

表 7-133 模型树数据

函数名	简介
-----	----

SetClassifyName	设置分类
AppendTopClass	增加顶层模型
GetTopItems	获取所有顶层节点
InsertClass	插入模型
RemoveClass	移除模型

7.2.2.8.4 模型参数面板

模型参数面板类，能够显示选中模型或组件参数，并且支持对各种类型的参数进行编辑，能够与中央视图进行联动。

表 7-134 模型参数面板

函数名	简介
GetParamEditMode	获取参数编辑模式
SlotUpdate	更新面板

7.2.2.8.5 仿真视图曲线

仿真曲线视图类，用于显示仿真变量曲线。

表 7-135 仿真视图曲线

函数名	简介
GetPlotWinIndex	获取曲线唯一 ID
AddCurveToCurrentView	添加变量到曲线图
SigWindowClosed	窗口关闭信号

7.2.2.8.6 仿真设置控件

模型仿真设置控件，用于显示和修改模型仿真设置。

表 7-136 仿真设置控件

函数名	简介
GetSimConfig	获取仿真设置

7.2.2.9 系统配置

对软件或模型进行相关系统环境设置，如可设置系统工作的路径设置编译平台的信息。

表 7-137 系统配置

函数名	简介
SetWorkPath	设置软件工作路径
WorkPath	获取软件工作路径
SetSimResultPath	设置仿真结果路径
SimResultPath	获取仿真结果路径
SetCompileInfo	设置编译器信息
GetCompileInfo	获取编译器信息
SwitchSolverPlatform	切换求解器平台

7.3 应用层

应用层定义了一套函数库、模型库和 APP 的开发规范，支持以规范的方式开发科学计算与系统建模仿真平台资源，提供资源管理接口，支持函数库、模型库和 APP 的装载、驱动和卸载。

7.3.1 函数库开发规范

科学计算环境内置了数学、图形、图像、符号数学、曲线拟合、信号处理、通信、DSP 系统、控制系统、优化、统计等多维度的函数库，同时也允许用户开发新的函数库并集成到科学计算环境中，从而扩展科学计算环境功能。

函数库是指存放可重用功能代码的项目，主要以 Julia 语言实现。函数库的物理形态是一个文件夹，其中包含了一系列的 Julia 源代码和配置文件。

本规范包含开发函数库的相关约束和标准，包含三个部分：函数库的开发规范、外部语言的封装规范、M 语言接口规范。

7.3.1.1 函数库开发规范

函数库的开发过程中需要遵循相应的开发规范，以确保函数库的高质量和可扩展性，并方便其在 Syslab 环境中进行集成和管理。同时为了保证 Julia 函数库的可维护性，开发函数库需要遵守 Julia 编码规范。

7.3.1.1.1 目录结构规范

一个标准的包（如 Revise），其目录结构如下图所示：

- docs: 帮助文档文件夹
- examples: 可选，测例文件夹
- images: 可选，资源文件夹
- src: 库源码文件夹
- test: 单元测试文件夹
- Project.toml: 项目文件
- LICENSE.md: 许可证文件
- README.md: 库说明文件
- ...: 用户还可以添加其它文件夹

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。



图 7-9 函数库的目录结构

7.3.1.1.2 函数定义规范

导出列表

函数、类型、全局变量和常量等，可以通过 `export` 添加到模块的导出列表。通常，导出接口列表位于或靠近模块定义的顶部，以便读者可以轻松找到它们。

首先，可以看下典型 Julia 包的做法，如下图所示：

```
1 if isdefined(Base, :Experimental) && isdefined(Base.Experimental, Symbol("@optlevel"))
2   @eval Base.Experimental.@optlevel 1
3 end
4
5 using FileWatching, REPL, Distributed, UUIDs
6 import LibGit2
7 using Base: PkgId
8 using Base.Meta: isexpr
9 using Core: CodeInfo
10
11 export revise, includet, entr, MethodSummary
```

图 7-10 Revise 库的导出列表

例如，MyExample 包的导出列表，定义如下：

```
module MyExample

export greet

greet() = print("Hello World!")

...

end # module
```

函数定义

Julia 包中最常用的就是函数，函数定义由两部分组成：一是函数注释，包括函数原型和函数功能说明；二是函数的算法实现。

首先，可以看下典型 Julia 包的做法，如下图所示：

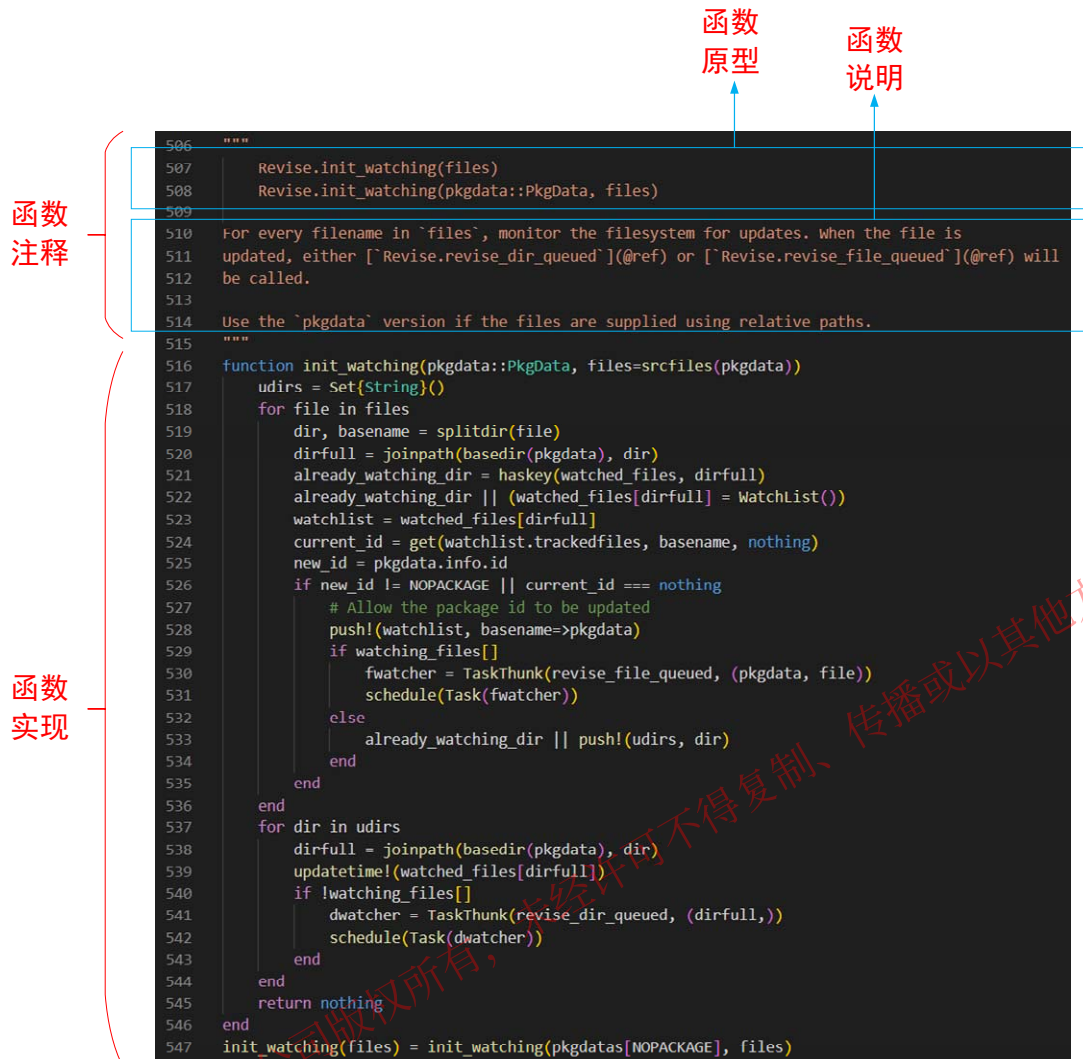


图 7-11 Revise 库的函数定义

例如，为 MyExample 包添加 “domath” 和 “pythagoras” 函数，如下所示：

```

module MyExample
export greet, domath, pythagoras

greet() = print("Hello World!")

"""
    domath(x::Number)

Return `x + 5`.
"""
domath(x::Number) = x + 5

include("math.jl")

end # module

```

其中，MyExample/src/math.jl 中的函数定义如下：

```

"""
    pythagoras(a,b)

```

勾股定理，英文名为 **Pythagoras** 也称为毕达哥拉斯定理。

在平面上的一个直角三角形中，两个直角边边长的平方加起来等于斜边长的平方。
如果设直角三角形的两条直角边长度分别是`a`和`b`，斜边长度是`c`，那么数学公式为：

```
``{\rm c = }\sqrt {{a^2} + {b^2}}``
```

返回斜边长`c`

```
"""
```

```
function pythagoras(a, b)
```

```
    c = sqrt(a^2 + b^2)
```

```
end
```

7.3.1.1.3 工程管理规范

每个函数库包含一个项目文件 `project.toml`，用于对函数库进行工程管理，其中包括函数的信息、函数库的依赖信息等，如下图所示：

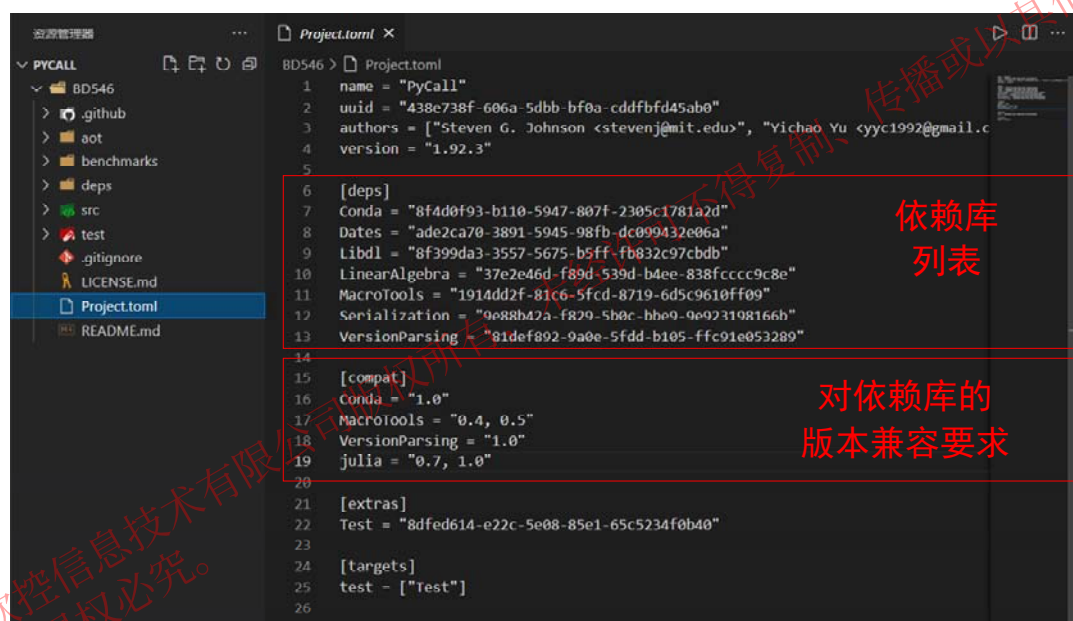


图 7-12 依赖库及其版本兼容要求

包的项目文件 `project.toml`，内容解释如下：

- **name**: 包的名称；
- **uuid**: 包的唯一标识；
- **authors**: 包的作者，书写规则为 “[NAME <EMAIL>, NAME <EMAIL>]”；
- **version**: 包的版本。必须遵守 SemVer 语义化版本，简而言之：
 - 破坏性更新：主版本 major release
 - 新特性：小版本 minor release
 - bug 修复：补丁版本 patch release
- **[deps]**: 该库依赖的其它函数库，书写规则为 “name = uuid”；

- [compat]: 该库对依赖库的版本兼容要求。关于 compat 字段的规则，请参考 <http://pkgdocs.julialang.org/v1/compatibility/>。典型的写法是：

```
[compat]
# 指 D 的所有 0.1.* 和 0.2.* 的版本都兼容
D = "0.1, 0.2"
```

- [extras]: 单元测试规定的依赖库，与 “[targets]” 一起使用；
- [targets]: 单元测试规定的依赖库。

注，extras 和 targets 请参考 <https://pkgdocs.julialang.org/v1/creating-packages/#Test-specific-dependencies-1>。

7.3.1.1.4 单元测试规范

单元测试

对于绝大多数代码开发来说，测试是保障代码质量和可靠性的最有力的工具。在这里我们介绍测试的基本内容和手段，以单元测试为主。

Julia 的测试代码存放在 “<包根目录>/test” 文件夹内并通过 “pkg> test” 调用 “test/runtests.jl” 文件。该文件可以理解为 Julia 单元测试的 main 文件。

(1). @test: 检查表达式的结果是否为 true。如果为 true 则测试通过。

```
using Test
@test 1 + 1 == 2
@test ones(2, 2) == [1 1; 1 1],

julia> using Test

julia> @test 1 + 1 == 2
Test Passed
Expression: 1 + 1 == 2
Evaluated: 2 == 2

julia> @test ones(2, 2) == [1 1; 1 1]
Test Passed
Expression: ones(2, 2) == [1 1; 1 1]
Evaluated: [1.0 1.0; 1.0 1.0] == [1 1; 1 1]
```

图 7-13 @test 的运行结果

(2). @testset: 用于将各个测试组织在一起，例如：

```
@testset "math" begin
    @test 1 + 1 == 2
    @test 1 - 1 == 0
    @test 1 / 0 == 1
end
```

其中，不通过的测试会被标记为 "Fail"，此时要么是测试代码没写对，要么是对应的功能存在 bug。


```

math: Test Failed at REPL[9]:4
  Expression: 1 / 0 == 1
  Evaluated: Inf == 1
Stacktrace:
 [1] macro expansion
   @ C:\Program Files\MWorks.Syslab 2022\Tools\julia-1.7.3\share\julia\stdlib\v1.7\Test\src\Test.jl:445 [inlined]
 [2] macro expansion
   @ REPL[9]:4 [inlined]
 [3] macro expansion
   @ C:\Program Files\MWorks.Syslab 2022\Tools\julia-1.7.3\share\julia\stdlib\v1.7\Test\src\Test.jl:1283 [inlined]
 [4] top-level scope
   @ REPL[9]:2
Test Summary: | Pass  Fail  Total
math          |    2     1     3
ERROR: Some tests did not pass: 2 passed, 1 failed, 0 errored, 0 broken.

```

图 7-14 @testset 的运行结果

常用工具

Julia 开发生态最常用的测试工具如下：

- [Test](#)：为 Julia 标准库；
- [ReferenceTests](#)：将结果与参考文件绑定做交叉对比测试，常用于图片的测试；
- [Suppressor](#)：用于抑制或捕获 stdout/stderr 的输出结果，经常与 Test 联合使用；
- Random.seed!：用于重置随机数种子，对于某些不定的情况会有用。

7.3.1.2 外部语言封装规范

在 Syslab 平台，我们提供了基于 Julia 的函数库。对于那些使用 C/C++ 或 Python 开发的外部语言函数库，我们可以通过封装成 Julia 函数库来快速集成到 Syslab 平台中。这种封装的方式使得外部语言函数库能够与 Syslab 平台进行无缝集成。通过将外部语言函数库进行封装，我们可以提供统一的接口，使得用户可以直接在 Julia 中调用这些外部语言函数库的功能，而无需关注底层的实现细节和语言差异。

封装外部语言函数库的过程往往涉及到对函数接口的映射和数据类型的转换，以确保外部语言函数库在 Syslab 平台中的正常运行。

7.3.1.2.1 C/C++ 语言封装规范

对 C/C++ 库的要求

为了满足跨平台的要求，对于使用 C/C++ 开发的函数库，我们需要同时提供适用于 Windows 和 Linux 系统的库文件，即动态链接库（DLL）和共享对象库（SO）。

通过提供 DLL 和 SO 库，我们可以确保函数库在不同操作系统下的兼容性。DLL 是 Windows 平台上常见的库文件格式，而 SO 则是 Linux 平台上的标准库文件格式。通过同时提供这两种库文件，可以为不同的操作系统封装相应的函

数据库。

ccall 组件

ccall 组件是 Julia 语言内核提供的调用 C/C++动态库的模块。通过 ccall 组件可以将 C/C++函数封装成 Julia 函数。ccall 函数使用方法如下：

```
ccall((function_name, library), returntype, (argtype1, ...),
      argvalue1, ...)
ccall(function_name, returntype, (argtype1, ...), argvalue1, ...)
ccall(function_pointer, returntype, (argtype1, ...), argvalue1, ...)
```

其中，参数解释如下：

- function_name: 需要调用的 C 函数名
- library: 包含函数的 C 库文件名
- returntype: 函数的返回类型
- argtype: 函数的输入参数类型列表
- argvalue: 函数的输入参数值

数据类型转换

使用 ccall 函数调用 C++库函数时，需要遵循 C/C++数据类型与 Julia 数据类型之间的映射规则。目前，基本的 C/C++值类型可以转换为 Julia 类型，以 C 为前缀。Julia 数据类型转换为 C/C++数据类型的对应关系如下表所示。

表 7-138 Julia 转 C

C 类型	标准 Julia 别名	Julia 基本类型
unsigned char	Cuchar	UInt8
bool (Bool in C99+)	Cuchar	UInt8
short	Cshort	Int16
unsigned short	Cushort	UInt16
int, BOOL (C, typical)	Cint	Int32
unsigned int	Cuint	UInt32
long long	Clonglong	Int64
unsigned long long	Culonglong	UInt64
intmax_t	Cintmax_t	Int64
uintmax_t	Cuintmax_t	UInt64
float	Cfloat	Float32
double	Cdouble	Float64
complex float	ComplexF32	Complex{Float32}
complex double	ComplexF64	Complex{Float64}
ptrdiff_t	Cptrdiff_t	Int

ssize_t	Cssize_t	Int
size_t	Csize_t	UInt
void		Cvoid
void and [[noreturn]] or _Noreturn		Union{}
void*		Ptr{Cvoid} (或 Ref{Cvoid})
T* (where T represents an appropriately defined type)		Ref{T} (只有当 T 是 isbits 类型时, T 才可以安全地转变)
char* (or char[], e.g. a string)		Cstring if NUL-terminated, or Ptr{UInt8} if not
char** (or *char[])		Ptr{Ptr{UInt8}}
jl_value_t* (any Julia Type)		Any
jl_value_t* const* (一个 Julia 值的引用)		Ref{Any} (常量, 因为转变需要写屏障, 不可能正确插入)
va_arg		Not supported
... (variadic function specification)		T... (其中 T 是上述类型之一, 当使用 ccall 函数时)
... (variadic function specification)		; va_arg1::T、va_arg2::S 等 (仅支持 @ccall 宏)

7.3.1.2.2 Python 语言封装规范

PyCall 组件

PyCall 是 Julia 语言调用 Python 库的组件, 这个组件提供了直接调用 Python 库并与 Python 完全互操作的能力。通过 PyCall 组件可以将 Python 函数封装成 Julia 函数。

PyCall 组件可以从 Julia 导入任意的 Python 模块, 调用 Python 函数(在 Julia 和 Python 之间自动转换类型), 从 Julia 方法定义 Python 类, 并在 Julia 和 Python 之间共享大型数据结构。

数据类型转换

在使用 PyCall 调用 Python 函数或库时, 需要使用 Python object interfaces 来表示和操作 Python 对象, 这些接口如下表所示

表 7-139 Python object interfaces

Python 对象	功能描述
-----------	------

PyObject	一个表示 Python 对象的 Julia 类型。通常情况下，Python object 的返回值都是 PyObject 类型
PyNone	表示 Python 中的 None 对象
PyCall.PyString	表示 Python 中的字符串对象。可以通过 string 函数将 Julia 的字符串转换为 PyCall.PyString 对象
PyCall.PyDict	表示 Python 中的字典对象。可以通过 Dict 函数将 Julia 字典转换为 PyCall.PyDict 对象，并且可以使用 get, setindex 和 index 等函数进行访问
PyCall.PyTuple	表示 Python 中的元组对象。可以通过 Tuple 函数将 Julia 元组转换为 PyCall.PyTuple 对象，并且可以使用 getindex 和 setindex 等函数进行访问
PyCall.PyArray	表示 Python 中的数组对象。可以通过传递 NumPy 数组到 PyCall.PyArray 函数，得到等效于 python 的 ndarray 的 Julia 对象，PyArray 还支持广播
PyCall.PyAny	表示 Python 中的任何对象。可以在函数的参数列表中使用 PyAny，以便接受任何 Python 对象或类型

这些 Python object interfaces 提供了一些基本的 API，使得 Julia 可以与 Python 交互，并且可以方便地访问和操作 Python 对象。同时，PyCall 还提供了一些额外的函数和工具，可以帮助用户更方便地使用 Python 库。

7.3.1.3 M 语言接口规范

TyMLang 是同元 M 语言兼容工具，通过该工具可以实现 M 语言调用 Julia 函数，具体调用过程如下：

- 1.将 M 对象转换成 Julia 对象。
- 2.调用 Julia 函数，用到第 1 步的 Julia 对象，获取到 Julia 类型的结果。
- 3.把 Julia 类型的结果转换成 M 类型结果。

Julia 函数库开发要遵循 M 语言兼容规范，主要包括 Julia 函数的语法、功能定义、函数输入和输出要和 M 语言函数保持一致。

M 语言兼容工具提供了 M 语言调用 Julia 函数的 API，具体详情如下。

7.3.1.3.1 API 列表

函数名	简介
Julia	将 M 类型对象转换为 Julia 类型对象
fromJulia	将 Julia 类型对象转换为 M 类型对象
jeval	执行 Julia 代码，返回 Julia 对象
jcall	调用 Julia 函数返回 Julia 对象

jinterp	调用 Julia 函数返回 Julia 对象，支持结构体类型参数输入
jindex	获取 Julia 对象的索引，返回 Julia 对象
jisa	判断 Julia 对象数据类型

7.3.1.3.2 M 类型与 Julia 类型对应关系

M 类型转为 Julia 类型之间的对应关系如下表所示：

表 7-140 M 类型转为 Julia 类型

M 类型	支持转换的 Julia 类型
double	Int/ComplexF64/Float64，及它们构成的数组
string	String, Vector{String}及其他 Array{String}
logical	Bool, Vector{Bool}及 Array{Bool}
uint8	UInt8, Vector{UInt8}及 Array{UInt8}
uint32	UInt32, Vector{UInt32}及 Array{UInt32}
uint64	UInt64, Vector{UInt64}及 Array{UInt64}
cell	Tuple
自定义映射的 M 类	自定义映射 M 类绑定的 Julia 类型，以及它的 Array 类型

例如：M 类型为 double，则可以将其转为 Julia 的 Float64/Vector{Float64}/Array{Float64}类型。

若使用 Julia(o) 函数直接创建 Julia 对象时，遵循以下规则：

- 数值、数值数组类型默认转为 Array{Float64} 类型。
- string 类型默认转为 Array{String} 类型。
- struct 类型默认转为 NamedTuple 类型。
- cell 类型默认转为 Tuple 类型，cell 中的元素按照前述规则转换。

7.3.1.3.3 Julia 类型与 M 类型对应关系

Julia 类型转为 M 类型之间的对应关系如下表所示：

表 7-141 Julia 类型转为 M 类型

M 类型	支持转换的 Julia 类型
double	Int/ComplexF64/Float64，及它们构成的数组
string	String, Vector{String}及其他 Array{String}
logical	Bool, Vector{Bool}及 Array{Bool}
uint8	UInt8, Vector{UInt8}及 Array{UInt8}
uint32	UInt32, Vector{UInt32}及 Array{UInt32}
uint64	UInt64, Vector{UInt64}及 Array{UInt64}

cell	Tuple
自定义映射的 M 类	自定义映射 M 类绑定的 Julia 类型，以及它的 Array 类型

7.3.2 模型库开发规范

系统建模仿真环境中的模型库由一系列模块组成，模块是构建系统模型的主要元素。系统建模仿真环境内置模型库提供了机、电、液、控、热、磁等多学科基本模块。如果内置库中没有合适的模块，系统建模仿真环境允许用户开发新的模块，并将其以模型库的方式集成到系统建模仿真环境中，从而扩展系统建模仿真环境的功能。为了保证各工程师开发的模型库具备统一的质量、稳定性和可读性，需要制定相应的规范。本规范包括模型库开发规范和外部库封装规范。

模型库开发规范包括以下内容：

模型库组织结构规范：规定了模型库的文件夹结构，以及各文件的作用，确保模型库的整体结构清晰且易于维护。

模型编码规范：定义了模型的命名规范、模型行为和方程编写规范规范、代码结构规范、注释规范等，以确保模型代码的可读性、可维护性和一致性。

模型图形布局规范：提供了模型图形化表示的规范，包括模块的排列方式、连线的表示方法等，使得模型图形直观且易于理解。

模型用户指南编写规范：规定了编写模型用户指南的格式、内容结构和语言风格，使用户能够快速了解并正确使用模型库中的模块。

模型工程化实践规范：指导模型库开发过程中的工程化实践，包括如何改善模型健壮性、如何提高仿真效率查等，以确保模型库的质量和稳定性。

模型测试规范：定义了模型测试大纲和模型测试流程，以保证模型的能够被充分测试。

模型库版本管理规范：规定了模型库版本号的命名规范、版本升级规范，以便于对模型库进行版本管理和追踪

外部函数封装规范包括以下内容：

C/C++语言及链接库封装规范：讲解了如何将 C 语言开发的函数和链接库封装为 Modelica 函数，包括外部函数机制、Modelica URI、注解填写等内容。

Julia 函数库集成封装规范：讲解了如何将 Julia 中的 Function 和 Object 封装集成到 Modelica 模型中，包括 Syslab Function 开发规范、Syslab Object 开发规范。

7.3.2.1 模型库开发规范

7.3.2.1.1 模型库组织结构规范

清晰的模型库层次架构，能够让模型库具有很强的可重用性和重构性，能够全面直观的展示模型库信息，方便用户在此基础上进行快速的应用、完善和拓展。

架构设计中引入 Package 的概念帮助组织 Modelica 模型中的 models、connectors 等元件，将相关的模型有机地组合在一起。将一个领域中的元件及接口组织在一起的 Package 就叫做“库”。这些库往往是以一个整体的形式被引用，但是库中总是包含若干个子模型库，就模型库的逻辑结构内容，可参考如下：

表 7-142 参考模型库逻辑结构

名称		描述	说明
UsersGuide		用户指导	模型库介绍、版本信息、作者信息及其他信息。
Common (可分开)	Types	类型	存放模型库中所有自定义类型
	Interfaces	接口	模型接口，存放 connector 连接器模型
	Functions	函数	存放模型库中所有函数
	Icons	图标	存放模型库中所有模型图标
	Resources	外部函数	包含 Include、Library 子目录存放外部函数用以集成 Fortran、C 代码，以及 lib、dll 等函数库
AeroPowerSystem (视开发内容而定)		航空动力装置模型库	基于子系统 SubSystems、组件 Components、基础元件 Basics 层次由高向低划分的系统模型库
ControlSystem (视开发内容而定)		控制系统模型库	基于子系统 SubSystems、组件 Components、基础元件 Basics 层次由高向低划分的系统模型库
HydraulicSystem (视开发内容而定)		液压系统模型库	基于子系统 SubSystems、组件 Components、基础元件 Basics 层次由高向低划分的系统模型库
...		...	...
Experiments		示例	系统模型示例集，主要包括各种仿真工况下的仿真模型
Tests		测试	模型测试用例，用于模型开发过程中的测试，在模型库交付时移除该内容

以航空发动机多物理领域模型库为示例模型库结构如下图 7-15 所示：

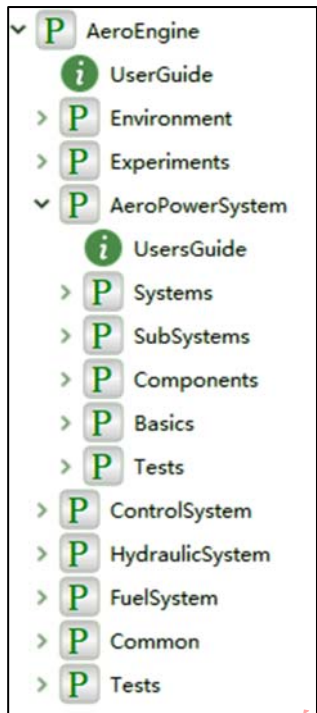


图 7-15 航空发动机多领域模型库

当模型依赖外部资源时，如图片、数据文件、C 代码等，需要保证模型对资源的引用路径为相对路径，建议在模型库 Package 文件夹下新建 Resources 文件，并按类别存放对应的文件，如下为外部动态库的存放示例。

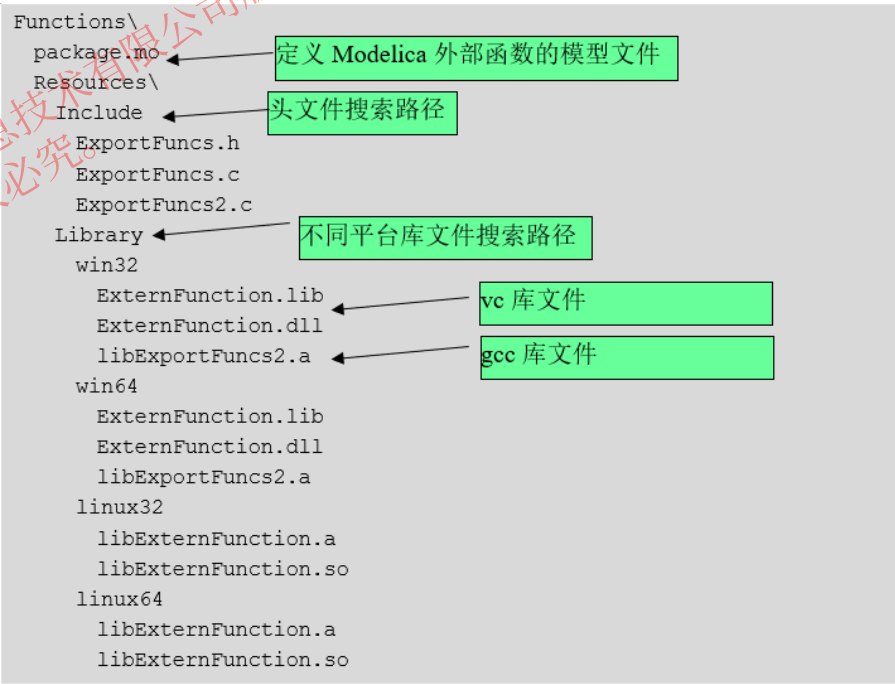


图 7-16 外部资源组织示例

7.3.2.1.2 模型编码规范

命名

命名需要在减少命名冲突的基础上增强模型代码的可读性。

减少命名冲突：建模过程中需要定义的参量、变量及系统自带的标识符和关键字之间应避免命名冲突。如：A 常用于表示面积、截面积等，因而在整个模型库中应该尽量减少字母 A 被用于其他含义的变量命名。

增强模型代码的可读性：对参数和变量的命名，除了可以方便地进行方程代码的书写，实现物理模型的行为外，还需要能够使读者快速理解该参数、变量的含义或者作用，更快地理解整个模型，因此其参数、变量的命名最好与其定义相对应，如长度的命名常常用单词 `length` 或者直接用大写字母 `L` 表示。

命名规则如下：

(1). 类命名

- 对于一个或多个单词全拼组成的类名，每个单词首字母应大写，如 `Exampe`、`ExanplePackeg`、`PartialModel` 等；
- 对于存在大写简写元素的类，为避免引起歧义，每个独立元素之间用下划线连接，如 `AD_Conversion`（数模转换器）等；
- 对于类的实例化。首字母应小写，其它遵循以上原则，如 `example`、`examplePackage` 等；

(2). 参数或变量命名

- 变量的名字尽量能显示出变量的含义；
- 对于一个单词的参数或变量名，一般均采用小写字母，如 `height`、`area` 等；
- 对于多个单词组成的参数或变量名，每个单词之间用下划线隔开，`angles_start`、`real_time` 等；
- 对于只有一个字母的参数或变量名，其命名需根据实际情况确定，如：`T`（温度）、`I`（转动惯量）、`t`（时间）等；

(3). 连接器命名

- 对于标准库（Modelica3.2 标准库及以上版本）中已存在的连接器，可直接继承使用，无需再另行定义，如：电气 `Pin`；机械 `Flange_a`、`Flange_b` 等；
- 对于需新定义的连接器和变量命名，须遵循类命名、参数和变量命名规则；

(4). 常用航空术语命名规范

航空领域有着自己的专业术语与缩写，收集此类名词并合理运用到

Modelica 建模中，可以避免命名过于冗长，使 Modelica 模型库更为专业化，参见《航空专业英语缩写索引》、《英汉航空航天工程词典》及相关设计规范文档，具体根据项目研发定制统一的命名规范。

表 7-143 常用符号命名规范

缩写	解释	全称
APU	辅助动力装置	Auxiliary Power Unit
ATS	自动油门系统	Autothrottle System
EEC	发动机电子控制器	Electronic Engine Control
EGT	排气温度	Exhaust Gas Temperature
.....		

模型行为和方程

对于一个物理系统而言，各组成部分内部及各组成部分之间的物理关系本身并没有因果性，因此，模型中应当是对模型行为的自然描述而无需考虑计算顺序。这种基于方程的建模方式称为陈述式建模，又称为非因果建模，与之相对的是过程式建模或因果性建模。

Modelica 是基于方程的建模语言，在上述的类型定义中可以看到，中间有明确的行为区域用于以方程来描述模型行为，作为对陈述式建模的补充，Modelica 同时支持过程式建模方法。

陈述式建模的特点之一是通过方程而非赋值来描述模型行为。Modelica 支持陈述式/过程式混合建模，分别以 `equation` 和 `algorithm` 来分别描述方程和赋值，区别在于，`equation` 定义的方程组间的求解顺序将由软件推断确定，而 `algorithm` 定义的则按照定义顺序进行求解。例如：

陈述式建模	过程式建模
<pre> model BasedOnEquation Real x; Real y; Real z; equation x + y + z = 1; x + 2 * z = y; x - y = time; end BasedOnEquation; </pre>	<pre> model BaseOnAlgorithm Real x; Real y; Real z; algorithm z := -time / 2; y := (1 - z - time) / 2; x := 1 - y - z; end BaseOnAlgorithm; </pre>

陈述式建模的最大优点是：用户建模时只需专注于物理问题的陈述，而无需考虑物理问题的错综复杂的求解过程怎样实现，因而建模更加简单，所建模型更加健壮。

对一个物理系统而言，其各组成部分内部及各组成部分之间的物理关系本

身就是非因果的，这与 Modelica 语言倡导的陈述式建模理念是完全吻合的。

代码结构

为了实现模型库编码风格的统一，模型库创建过程中应遵循如下的 Modelica 模型编码顺序：

- 第一部分：继承类语句，如 import、extend、outer 等；
- 第二部分：模型参数，parameter；
- 第三部分：模型变量；
- 第四部分：模型接口，FluidPort_a/flange_a；
- 第五部分：初始方程，initial equation；
- 第六部分：方程和算法，equation、algorithm。

具体代码结构示例如下形式：

```
model FixedDoubleActingCylinder "缸体固定式双作用液压缸"
// 第一部分：继承类语句如 import、extend、outer 等；
```

```
extends Utilities.Icons.FixedDoubleActingCylinder2;
extends Components.BasicModels.TwoPortsCylinder_piston;
```

```
// 第二部分：模型参数，parameter；
```

```
parameter SI.Diameter D = 0.05 "活塞直径"
    annotation (Dialog(tab = "结构参数",group = "基本参数"));
parameter SI.Diameter d = 0.024 "活塞杆直径"
    annotation (Dialog(tab = "结构参数",group = "基本参数"));
.....
```

```
// 第三部分：模型变量；
```

```
SI.Length s(start = initialPosition) "活塞位移";
SI.Velocity v "活塞运动速度";
SI.Acceleration a "活塞运动加速度";
.....
```

```
// 第四部分：模型的接口；
```

```
Modelica.Mechanics.Translational.Interfaces.Flange_a flange_a 固体接口
    annotation (...)
    .....
```

```
// 第五部分：初始方程，initial equation；
```

```
initial equation
    v = 0 "活塞初始速度";
```

```
// 第六部分：方程和算法，equation、algorithm；
```

```
equation
//速度、加速度方程
v = der(s);
a = der(v);
//左右腔等效容积
effVolume_L = activeArea_L * s + deadVolume;
effVolume_R = activeArea_R * (L - s) + deadVolume;
.....
end FixedDoubleActingCylinder;
```

注释

建立的所有类别（package, model, function）都需要相应的注释，一般类注释普遍采用下面的形式，一般用“'''”进行注释，如：

```
package AeroEngine_HyMo"航空发动机液压元部件模型库"
model Pipe "管道"
.....
end Pipe;
function ReynoldsNumber "雷诺数函数"
.....
end ReynoldsNumber;
end AeroEngine_HyMo;
```

(5). 参变量注释

定义每个参量，变量都要注释该参变量的含义，增强代码的可读性，注释语尽量简明扼要，一般用“'''”进行注释，如：

```
parameter SI.Length L = 1 "管道长度";
parameter SI.Diameter d = 0.02 "管道内径";
SI.Area A = d ^ 2 * Constants.pi / 4 "管截面积";
SI.Volume V = L * A "管道容积";
```

(6). 方程注释

在模型代码的方程区域，某一个方程的含义或某一部分方程的含义都要注释清楚，一般用“//”进行注释，如：

```
Equation
//流量方程
m_flow = sign(dp) * A * Cq * sqrt(2 * rho * noEvent(abs(dp)));
```

参数及参数框设计

B. 参数定义

定义模型参数包括参数数据类型、名称、默认值、量纲、必要的描述信息等，参数示例如下：

表 7-144 参数定义

参数类型	参数名称/默认值/量纲	参数说明
Modelica.SIunits.Length	Stroke = 0.03	换向阀行程
Boolean	Init=false	阀芯初始位置是否由输入决定
Real	Cq=0.7	流量系数
...	...	...

C. 参数框定义

参数框中的参数显示都是按照“先定义先显示”的原则，包括 Tab/Group 的显示，所以参数框的显示最好按照参数的“重要程度”进行有区别显示，进行参数分类显示时一般遵循这样几点原则：

- (1). 参数少，分 Group；
- (2). 参数多，分 Tab；
- (3). 重要共性参数（介质）单独分开，Tab/Group 统一；
- (4). 参数的一般分类：
 - a. 工质参数
 - b. 结构参数
 - c. 初始条件（变量初始化及初始参数）
 - d.

参数框设计代码：

```

parameter SI.Diameter D = 0.05 "活塞直径"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Diameter d = 0.024 "活塞杆直径"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Mass m = 0.5 "活塞质量"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Length L = 2 "活塞行程"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.TranslationalDampingConstant c = 2000 "活塞运动阻尼"
  annotation (Dialog(tab = "结构参数", group = "基本参数"));
parameter SI.Volume deadVolume = 1e-6 "死区容积"
  annotation (Dialog(tab = "限位参数"));
parameter SI.TranslationalSpringConstant c_c = 1e8 "接触刚度"
  annotation (Dialog(tab = "限位参数"));
  
```

参数框显示结果：

图 7-17 参数框设计参考

接口定义

为避免系统模型集成阶段由接口带来的问题，在系统分析设计初期就需要将接口定义清晰，以及各系统间的连接方式。

接口说明内容包括：模型输入/输出变量的数据类型、维数、名称、量纲、必要的描述信息，接口说明示例如下：

表 7-145 接口定义

接口	数据类型	名称	量纲	描述	其他说明
输入	Real[1,1]	H	m	飞行高度	高度表接口
	Real[1,1]	Ma	1	飞行马赫数	空速表接口
	Real[1,1]	TLADeg	deg	油门杆角度	
	...	...	...	...	...
输出	Real[1,1]	qm_f	kg/s	燃油流量	
	...	...	...	...	...

针对特定行业接口主要包含机械接口、电接口、热接口、信息接口和流体接口。

D. 机械特性接口

表 7-146 模型输入接口

	flow 变量	flow 变量	势变量	势变量
变量类型	Force	Torque	Position	Frames.Orientation
变量名称	f[3]	t[3]	r_0[3]	R
描述	剪切力	剪切力矩	从世界坐标系原点到连接点的位置向量	将世界坐标系转到连接点的方向向量

			(相对世界坐标系)	
功能	机械多体接口, 描述了剪切力、剪切力矩和两个方向向量			

表 7-147 模型输出接口

	flow 变量	flow 变量	势变量	势变量
变量类型	Force	Torque	Position	Frames.Orientation
变量名称	f[3]	t[3]	r_0[3]	R
描述	剪切力	剪切力矩	从世界坐标系原点到连接点的位置向量 (相对世界坐标系)	将世界坐标系转到连接点的方向向量
功能	机械多体接口, 描述了剪切力、剪切力矩和两个方向向量			

E. 电特性接口

表 7-148 模型输入接口

	flow 变量	势变量
变量类型	Current	Voltage
变量名称	i	v
描述	电流	电压
功能	用于基本模型中电流和电压的传递	

表 7-149 模型输出接口

	flow 变量	势变量
变量类型	Current	Voltage
变量名称	i	v
描述	电流	电压
功能	用于基本模型中电流和电压的传递	

F. 热特性接口

表 7-150 模型输入接口

	flow 变量	势变量
变量类型	HeatFlowRate	Temperature
变量名称	Q_flow	T
描述	接口热流量	接口温度
功能	用于基本模型中热流量和温度的传递	

表 7-151 模型输出接口

	flow 变量	势变量
变量类型	HeatFlowRate	Temperature
变量名称	Q_flow	T
描述	接口热流量	接口温度
功能	用于基本模型中热流量和温度的传递	

G. 信息特性接口

表 7-152 模型输入接口

	变量类型	变量名称	描述	功能
RealInput	Real	input	实型信号输入接口	实型数据传递
RealOutput	Real	output	型信号输出接口	
IntegerInput	Integer	input	整型信号输入接口	整型数据传递
IntegerOutput	Integer	output	整型信号输出接口	
BooleanInput	Boolean	input	布尔型信号输入接口	布尔型数据传递
BooleanOutput	Boolean	output	布尔型信号输出接口	

表 7-153 模型输出接口

	变量类型	变量名称	描述	功能
RealInput	Real	input	实型信号输入接口	实型数据传递
RealOutput	Real	output	型信号输出接口	
IntegerInput	Integer	input	整型信号输入接口	整型数据传递
IntegerOutput	Integer	output	整型信号输出接口	
BooleanInput	Boolean	input	布尔型信号输入接口	布尔型数据传递
BooleanOutput	Boolean	output	布尔型信号输出接口	

H. 流体特性接口

表 7-154 模型输入接口

	flow 变量	势变量	Stream 变量	Stream 变量
变量类型	VolumeFlowRate	AbsolutePressure	SpecificEnthalpy	MassFraction
变量名称	w	p	h	Xi
描述	接口质量流	接口压力	接口比焓	接口组分质量分数
功能	一维流体接口，用于基本模型中流动工质的质量流、压力、比焓和组分的传递			

表 7-155 模型输出接口

	flow 变量	势变量	Stream 变量	Stream 变量
变量类型	VolumeFlowRate	AbsolutePressure	SpecificEnthalpy	MassFraction
变量名称	w	p	h	Xi
描述	接口质量流	接口压力	接口比焓	接口组分质量分数
功能	一维流体接口，用于基本模型中流动工质的质量流、压力、比焓和组分的传递			

7.3.2.1.3 模型图形布局规范

基于组件连接构建系统模型时，应对系统模型进行合理布局，兼顾系统模型的可读性与美观性。注意以下几点：

- (1). 能显示出系统的层次结构，可用画图工具将重要区域标注，并附上简单

的文字说明；

- (2). 合理利用空间，各组件连接紧凑，互不干涉；
- (3). 尽量使系统模型结构整体看起来赏心悦目，并显示出其专业性。

可以参考如下模型的布局：

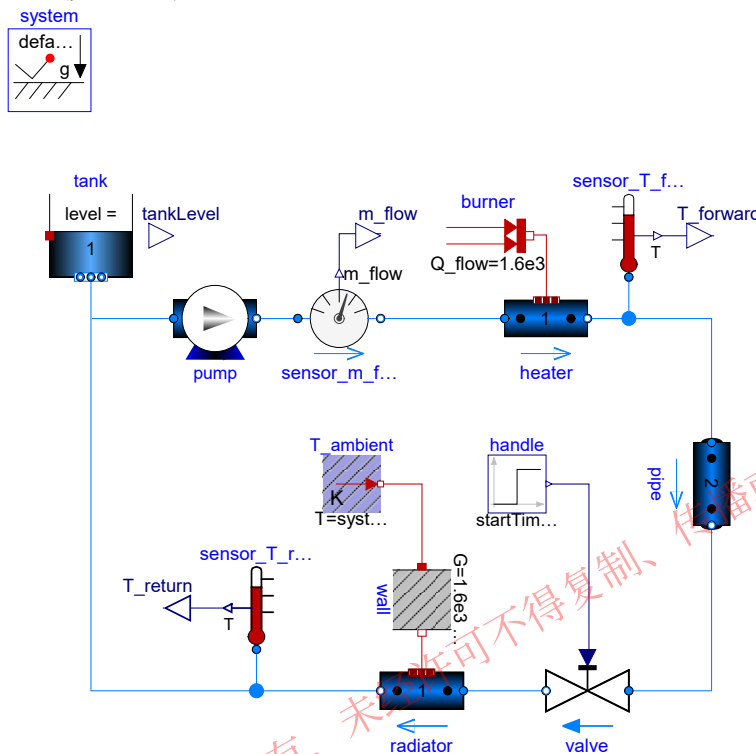


图 7-18 简单加热系统模型布局

7.3.2.1.4 模型用户指南编写规范

新建的模型库信息一般都是通过建立 UsersGuide 库进行相应地说明，一般模型库的 UsersGuide 主要分为这样几部分：

- (1). Overview（综述）——模型库特点，构架等；
- (2). UsersGuide（用户指导）——使用说明，开发说明等；
- (3). ReleaseNotes（版本日志）——版本修订日志；
- (4). Help（帮助）——关联用户说明手册或者 FAQ；
- (5). References（参考资料）——模型库建模主要参考资料；
- (6). Contant（联系方式）——开发者的相关信息。

对于任何一个组件模型及系统模型，应给出详尽的说明文档，清楚明了的说明该模型的功能、用法及注意事项。从使用者角度出发对模型简介、模型原理、模型接口、模型参数等方面进行描述，可参见附录 A。

- 简介，对模型的功能及用法进行简要描述；
- 原理，阐述模型的结构与原理，以及模型开发所应用到的理论基础、理论公式等信息；

- 接口，包括模型的输入、输出接口的类型、描述信息；
- 参数，包括模型参数的类型、量纲、描述信息。

7.3.2.1.5 模型工程化实践规范

改善模型健壮性

模型编码时需要注意模型的健壮性，不仅要每个元件进行充分的测试，也需要防止库被误用。

- 输出限幅，最大值、最小值限制等，可使用断言语句或加入限幅组件模型等，如：`assert(i*v<1e6," Maximum power exceeded" );`
- 使用 `final` 关键字和限制属性；
- 好的文档，给使用者意见和提醒，减少误用。

提高仿真效率

(1). 尽量使用 `equation`

如果没有特殊的目的，请尽量使用 `equation`。原因是在 `algorithm` 块中，变量可能被赋值多次，使得工具无法对其进行符号分析，提高仿真效率。例如，无法得到求解雅阁比矩阵的函数。

(2). 避免不必要的事件

事件会中断积分算法，从而影响仿真速度，因此避免不必要的事件可以较大地提供仿真速度。如：

```
der(x)= noEvent(if y<0 then 0 else y^2);
```

(3). 合适的积分算法

积分算法有自己的适用范围，因此需要模型的特点，采用合适的算法，例如模型是刚性的，则需要采用求解刚性问题的积分算法。

(4). 合适的误差

误差越小仿真时间越长，结果越可靠，而误差越大仿真速度越快，但结果越不可靠。可以使用不同的误差仿真，比较它们的结果，而确定合适的误差。

(5). 为函数提供雅阁比函数

对于隐式方程组，分析器使用符号推导，得到求解雅阁比矩阵的函数，避免数据求解。由于分析器不能推导用户自定义函数，因此如果用户为自定义函数提供导数函数，可以提高仿真效率。如：

```
model JacobianExample
  Real x,y;
  function f
    input Real x;
    output Real y;
    annotation (derivative=f_Jac);
  algorithm
    y := sin(x)+cos(x)+x;
```



```
end f;  
function f_Jac  
  input Real x;  
  input Real dx;  
  output Real dy;  
  algorithm  
    dy := dx * (cos(x)-sin(x)+1);  
  end f_Jac;  
equation  
  der(y) = 2.0;  
  y = f(x);  
end JacobianExample;
```

(6). 变量消除

分析器会进行符号处理，消除别名变量。因此用户手动消除别名变量对仿真速度没有影响，而且容易使代码变得不直观。除非有证据表明消除变量对仿真速度有影响，否则不建议手动消除别名变量。如：

```
model Resistor  
  import Modelica.SIunits;  
  import Modelica.Electrical;  
  SIunits.Voltage v "Voltage from pin p to n";  
  SIunits.Current i "Current entering at pin p";  
  Electrical.Analog.Interfaces.Pin p "Positive";  
  Electrical.Analog.Interfaces.Pin n "Negative";  
  parameter SIunits.Resistance R = 300 "Resistance";  
  
equation  
  v = p.v - n.v;  
  0 = p.i + n.i;  
  i = p.i;  
  i * R = v;  
end Resistor;
```

不建议消除

消除后的方程

$$R * p.i = p.v - n.v;$$
$$p.i + n.i = 0;$$

7.3.2.1.6 模型测试规范

模型测试规范包括模型测试验证流程和结果评价方法。模型测试验证流程用于规范模型测试过程，结果评价方法用于规范模型测试评价结果。

测试验证流程

模型验证的总体方法是模型仿真结果文件与可靠数据进行比较判断。其流程如图 7-19 所示：分别用模型库及系统的物理试验数据或（和）同类软件的仿真结果数据来和仿真结果数据作对比、分析，从而完成模型验证。

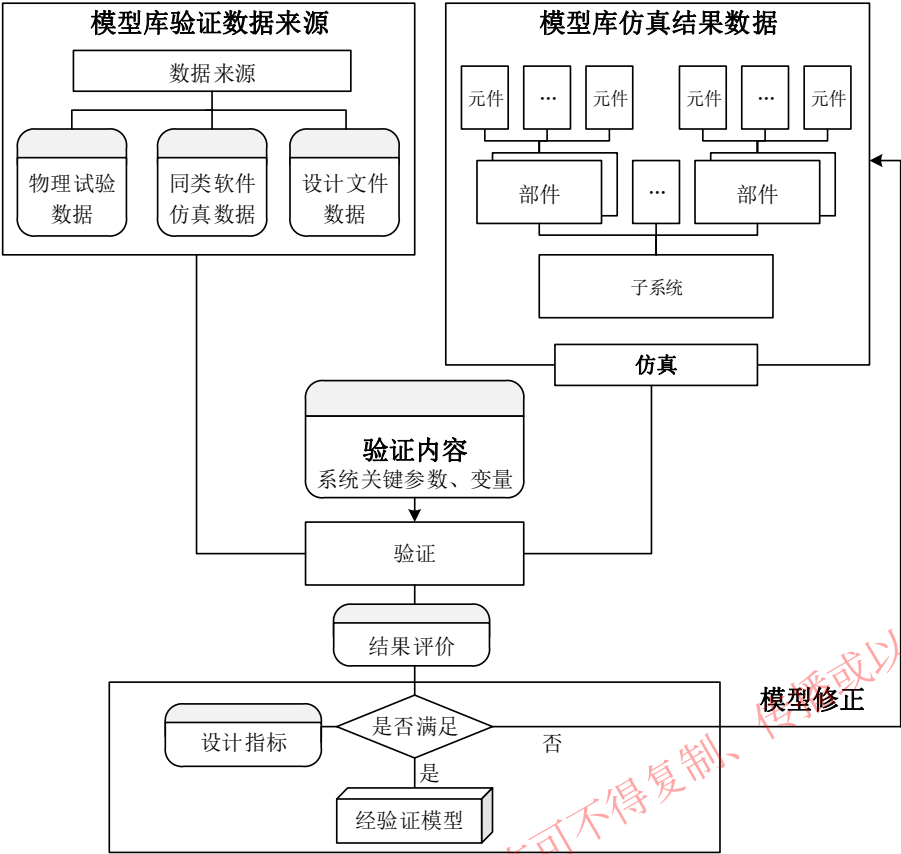


图 7-19 模型验证流程

对于以上模型验证流程，其数据来源包括三种，第一种为设计文件数据，第二种为同类软件仿真结果数据，第三种为物理试验数据。模型验证的层次分二层，第一层为模型库组件模型的验证，第二层为系统模型的验证。

结果评价

组件模型的测试需得到指定的关键数据结果，并且与对应物理试验结果进行比较。具体结果评价方法可参考静态误差与动态误差评价方式。

A. 稳态误差

1) 评价方式

对于稳态误差的评价方法，下面提供几种误差衡量指标：

- 平均偏差：根据每个采样时刻的仿真值与实际值之间差值的绝对值与仿真值之间的比值来计算误差百分比，最后通过加权的方式来获取整批数据的误差：

$$F = \frac{1}{n} \cdot \sum_{i=1}^n \left(\frac{|X_{\text{仿真}}(i) - X_{\text{试验}}(i)|}{|X_{\text{仿真}}(i)|} \times 100\% \right)$$

- 最大误差：即为所有采样时刻的仿真值与实际值的最大值。

$$F = \max_{i=1, \dots, n} (|X_{\text{Measurement}}(i) - X_{\text{Simulation}}(i)|)$$

2) 应用场景示例

静态误差的使用适用于稳态过程，即系统的输出数据达到一个固定的稳态值时进行评价。适用于大多数模型的仿真结果评价。

B. 动态误差

1) 评价方式

对于动态误差的评价方法，即当输入数据为连续变化时，系统的输出数据没有一个固定的稳态值时，无法计算系统输出的稳态误差，此时，可考虑以累计误差为评价标准进行衡量。

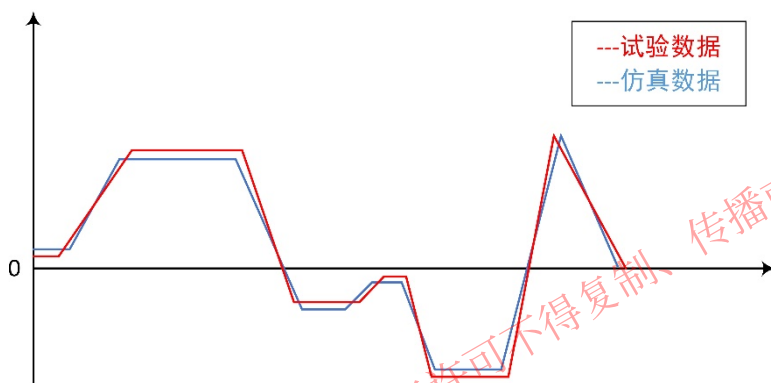


图 7-20 动态过程输入示意

- 累计误差：取试验、仿真曲线绝对值，通过积分的方式计算绝对值曲线面积与各采样点绝对误差曲线面积来反映误差大小，相当于将每个时刻的误差进行累计。累积误差计算方式如下：

$$F = \frac{S_{\text{误差}}}{S_{\text{试验}}} \times 100\% = \frac{\int |X_{\text{仿真}}(i) - X_{\text{试验}}(i)| \cdot \Delta t}{\int |X_{\text{试验}}(i)| \cdot \Delta t} \times 100\%$$

其中 $S_{\text{误差}}$ 为绝对误差曲线与时间轴所围成的面积， $S_{\text{试验}}$ 为试验曲线绝对值与时间轴所围成的面积， Δt 为采样时间， i 为对应每个采样的序号。

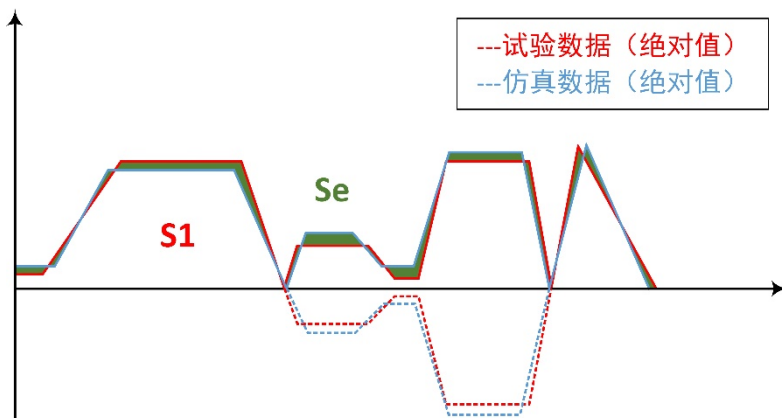


图 7-21 累积误差计算示意图

2) 应用场景示例

如在下列连续变化输入的动态仿真过程中，选择以累计误差评价动态误差的方式，比较合适。

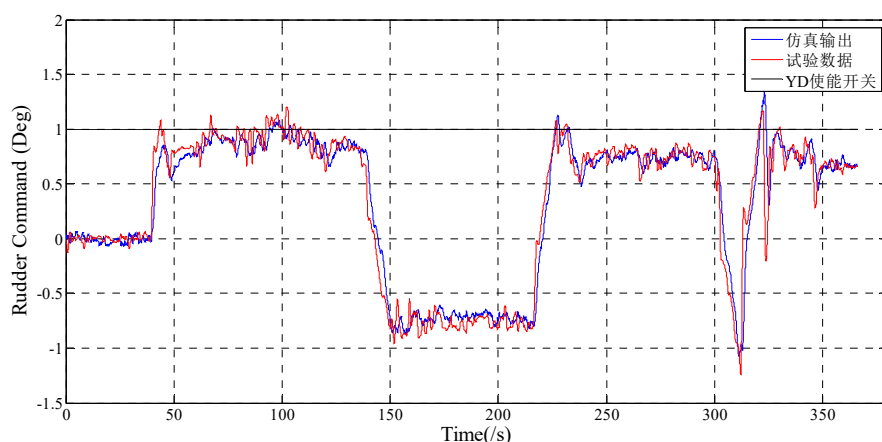


图 7-22 基于动态误差的结果评价应用场景

其误差计算结果为：

仿真绝对值曲线面积：239.7146

试验绝对值曲线面积：247.0526

误差计算：3.06%

7.3.2.1.7 模型库版本管理规范

在模型库构建以及后期维护过程中，存在大量的模型库版本迭代，应对模型库的版本进行管理，确保版本的准确性和可追溯性。

A. 版本命名

模型库版本命名主要由 5 部分组成，第一部分为模型库名称，第二部分为主版本号，第三部分为子版本号，第四部分为日期版本号和希腊字母版本号，第五部分为项目缩写名称。希腊字母版本号共有 4 种，分别为 Alpha、Beta、RC、Release。例如：HyMo_1_1_20180810_Release_BJHFYYJM。

- 模型库名称：指用户所构建的模型库名称，通常是系统物理属性等的英文全称或英文简写；
- 主版本号：当模型库由较大变动，如增加多个模块或这整体架构发生变化，此时可根据需要进行修改；
- 子版本号：当模型功能有一定增加或变化，如提高模型粒度等，可根据需要进行修改；
- 日期版本号：用于记录修改时间，每天对模型的修改都需要更改日期

版本号；

- Alpha 版：主要指模型以实现功能为主，该版本模型存在较多问题，需进行修改；
- Beta 版：较 Alpha 版修改了严重问题，但仍存在一定缺陷，需要经过多次测试来进一步消除；
- RC 版：该版本相对成熟，已修正了绝大部分错误，与最终版本接近；
- Release 版：经过多轮修改后的最终版本。

B. 版本升级

主要针对主版本号和子版本号，其基数均为 0 或 1，在每一次模型版本发生变更时，均在当前版本号上加 1，并以文本或其它形式标注每个版本号对应的变更内容。

7.3.2.2 外部语言封装规范

7.3.2.2.1 C/C++ 语言封装规范

用户可能采用 C/C++/Fortran/Python 等语言开发的模型，需要集成到本仿真平台中。Modelica 提供了外部函数机制，支持将外部语言开发的模块集成到 Modelica 模型库。

Modelica 中调用外部函数通过 Modelica function 进行，这种 Modelica 函数没有算法区域，取而代之的是外部函数接口声明语句“external”，以表示调用的是外部函数。

外部函数

模型中除了可以调用使用 Modelica 语言编写的函数外，还可以调用其它语言编写的函数，这些其它语言编写的函数称为外部函数。Modelica 中调用外部函数通过 Modelica 函数进行，这种 Modelica 函数没有算法（algorithm）区域，取而代之的是外部函数接口声明语句“external”，用以表示调用的是外部函数。

```
package myfuncs
function sin
  input Real x;
  output Real y;
  external "C" ;
end sin;
end myfuncs;
```

函数 myfuncs.sin 调用 C 语言函数 sin，调用形式为“y=sin(x)”。在 external 中可选地指定外部函数的语言、外部函数调用声明等。

注解用来指定数组布局（arrayLayout）是按行主存储（rowMajor）还是按

列主存储（columnMajor）、外部函数所需的头文件（Include）、外部函数所需的库文件（Library）、外部函数所需头文件所在的位置（IncludeDirectory）、外部函数所需库文件所在的位置（LibraryDirectory）。

```
package myfuncs
function exp "e 的指数"
  input Real u;
  output Real y;
  external "C" y = exp(u) annotation (Include = "#include<math.h>");
end exp;
end myfuncs;
```

如果没有 IncludeDirectory 注解，外部函数所需头文件默认位置是包含外部函数的包(package)文件(“.mo”)所在文件夹中的“Resources/Include”子文件夹。同样，若没有 LibraryDirectory 注解，外部函数所需库文件默认位置是包含外部函数的包(package)文件(“.mo”)所在文件夹中的“Resources/Library”子文件夹，不同平台的目标文件还可以存放在相应的平台文件夹中。

支持以下标准平台：

- win32 [32 位 Microsoft Windows]
- win64 [64 位 Microsoft Windows]
- linux32 [Intel 32 位 Linux]
- linux64 [Intel 64 位 Linux]

Modelica 模式 URI

Modelica 模式 URI 用于表示 Modelica 资源所在的位置，其形式如下：

```
modelica://[LibraryName]/[Path]
```

URI 由三部分组成：“modelica://”是 modelica 模式 URI 的专用标识，跟在其后两部分为 LibraryName 跟 Path：LibraryName 表示 Modelica 包的全名，path 部分用于表示资源相对于包的位置，是相对路径。如果 LibraryName 中有 Q-IDENT，则所有 URI 保留的字符都遵循 URI 表示的规则，即百分号编码。示例：

```
modelica://Modelica.Mechanics/Src/Include
```

LibraryName: Modelica.Mechanics, Path: Src/Include

```
modelica://myRec.%27%2b%27/Lib
```

LibraryName: myRec.'+', Path: Lib

Include 和 IncludeDirectory 注解

Include 注解中的内容直接嵌入至模型 C 代码中，因此，其中可以指定包含文件或者直接写一段 C 代码。示例：

指定包含文件

```
function exp "Exponential, base e"
```

```
input Real u;
output Real y;
external "C" y = exp(u) annotation(Include="#include <math.h>");
end exp;
```

直接嵌入 C 代码：

```
function IncTest1
input Real x;
output Real y;
external "C" y=myDouble(x) annotation(Include="double
myDouble(double x)
{return 2*x;}");
end IncTest1;
```

IncludeDirectory 注解指定包含文件所在的位置，以 URI 的 modelic 模式表示。

Library 和 LibraryDirectory 注解

Modelica 中外部函数声明时用 Library 注解指定链接库名（注意：不带扩展名）。下面示例中指定的连接库名是 Fac，那么完整的连接库名称是 Fac.lib（vc 编译生成）或 libFac.a（gcc 编译生成）。

```
function Fac "factorial"
input Integer n;
output Real res;
external "C" annotation (Library = "Fac",
LibraryDirectory="modelica://ExternFuncUseDll.ExternFunc");
end Fac;
```

LibraryDirectory 注解指定库文件和 dll（或 so）文件所在的位置。指定位置中可以使用不同的平台文件夹存放各平台的库文件和 dll（或 so）文件，：

- win32 [32 位 Microsoft Windows]
- win64 [64 位 Microsoft Windows]
- linux32 [Intel 32 位 Linux]
- linux64 [Intel 64 位 Linux]

7.3.2.2.2 Julia 函数库集成规范

本节说明如何将科学计算环境中开发的 Julia 函数库封装为 Modelica 函数库中的模块，以支持用户在系统仿真环境中重用 Julia 函数库，并以图形化的方式集成科学计算函数或模型。

Syslab Function 开发规范

Syslab Function 是一种将 Julia 函数封装为 Modelica 模型的机制。Syslab Function 通过 Modelica 外部函数实现。本节说明 Syslab Function 的功能机制和操作步骤。

Syslab Function 包含两个模型：

- SyslabGlobalConfig：用于全局声明，包括导入包及全局变量声明等。
- SyslabFunction：用于嵌入 Julia 函数，并将 Syslab Function 模块的输入和输出数据指定为参数和返回值。

SyslabGlobalConfig

用于为系统中的 Julia 函数提供全局声明，例如导入包或声明全局变量。当创建了 SyslabGlobalConfig 组件之后，点击右键菜单中的“Syslab 初始化配置...”可以在 Syslab 中打开编辑器，编写全局声明的 Julia 脚本。

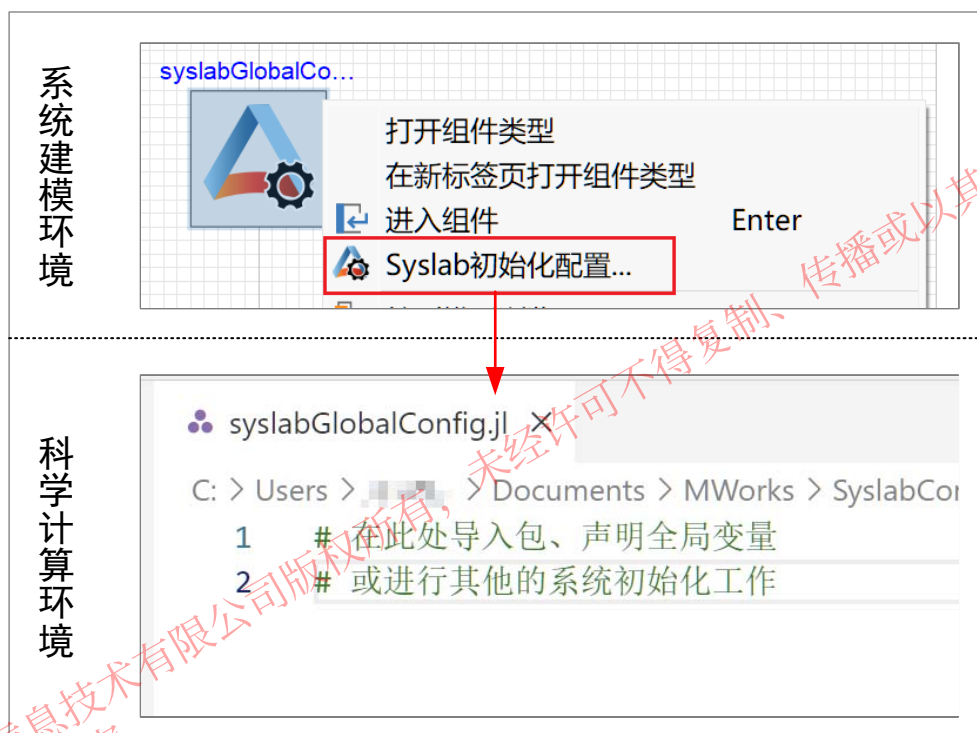


图 7-23 SyslabGlobalConfig

SyslabFunction

SyslabFunction 用于嵌入 Julia 函数，并将 Syslab Function 模块的输入和输出数据指定为参数和返回值。系统仿真每推进一步都会调用该 Julia 函数。

对于 SyslabFunction 组件而言，点击右键菜单中的“编辑 Syslab 脚本函数...”项，可以在 Syslab 中打开编辑器，编写 Julia 脚本，例如：


```
1 function func1(t)
2     x,y = get_xy(t)
3     return x,y
4 end
5
6 function get_xy(t)
7     a = [t, 2t]
8     b = [t 2t 3t; 4t 5t 6t]
9     return a,b
10 end
11
```

图 7-24 SyslabFunction

上图中的 Julia 脚本即完成了一个 Syslab Function 定义，组件将生成一个名为 in_t 输入端口和两个分别名为 out_x、out_y 的输出端口。

SyslabFunction 组件认为脚本中的第一个函数为本组件的主函数，其他函数均为服务于主函数的辅助函数。根据主函数的内容，组件从函数声明中的输入参数中，获取组件的输入端口的数量及名称；从返回值语句中获取组件的输出端口的数量及名称。Syslab 的主函数的定义规则如下：

- 模型的输入端口数量由 Julia 函数的声明决定，类型和维度在端口由用户在设置对话框中指定；
- 模型的输出端口数量由 Julia 函数的返回值决定，类型和维度在端口由用户在设置对话框中指定；
- 端口类型支持实型、整型、布尔型，维度字段如果空缺，则认为端口为标量端口；
- 主函数必须使用 function 定义；
- 主函数的输入不要指定类型，不要指定具名参数；
- 主函数的输出必须使用 return 指定，且必须为函数体中已经出现的变量符号；
- 在同一个系统模型中，函数名必须是唯一的；

Syslab Object 开发规范

Syslab Object 是一种 Julia 类，具有用于算法建模的特定方法和属性。System object 适用于为动态系统建模和处理流式数据。用户可以使用模型库产品中包含的预定义 Syslab Object，也可以定义自己的 Syslab Object。

Syslab Object Block 是系统建模环境将 Syslab Object 封装成的 Mo 模块，提供了一套结构化的模板流程，可在执行期间自动完成许多管理操作，包括初始化系统、验证数据、单步计算和终止等操作。

Syslab Object 的组件关系如下图所示。

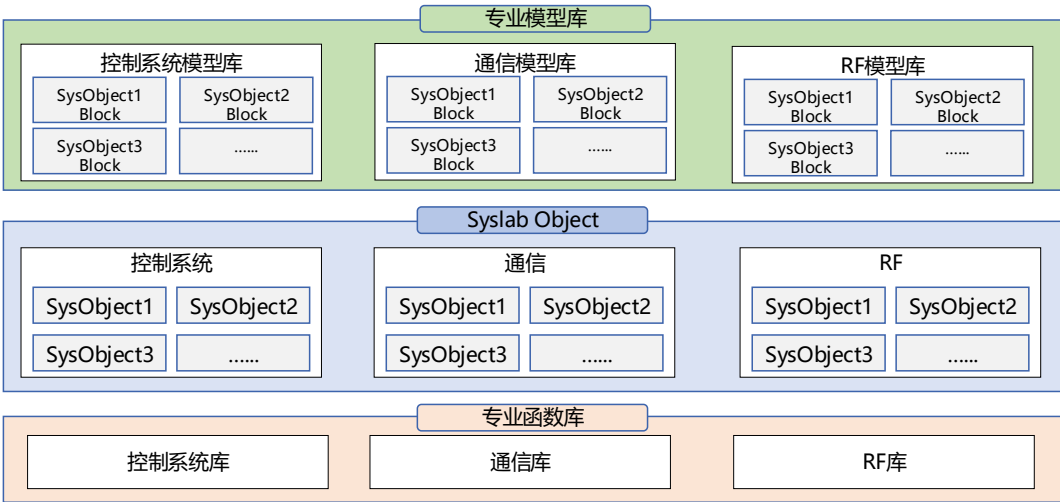


图 7-25 Syslab Object 组件关系图

Syslab Object 的开发规范

Syslab Object 是一个 Julia 类，包含类名、类属性、类方法等基本要素。Syslab Object 的开发需要遵循具体的开发规范，其模板文件如下所示。

```
using ObjectOriented
@ooodef mutable struct Untitled
    # Description (用于生成 Modelica 组件描述)
    # Template for syslab object block.

    # Parameter (用于生成 Modelica 组件参数，包括名称、类型、描述)
    # 格式: 参数名::参数类型 = 参数值    # 参数注释
    # 例如:
    gain::Real = 1.0                    # 增益

    # Private (Julia 内部变量)
    # 格式: 变量名::类型 = 值    # 变量说明
    # 例如:
    _count::Integer = -1                # 计数器

    # Methods (主要调用算法，包括 setupImpl, stepImpl, releaseImpl)

    # 初始化函数: 函数名固定，函数形参与 stepImpl 函数形参一致
    function setupImpl(self, u)
        self._count = 0
        # ...
        return nothing
    end #setupImpl

    # 单步计算函数: 函数名固定，第一个函数形参必须是 self，其余函数形参将作为
    # Modelica 组件的输入端口，函数返回值作为输出端口
    function stepImpl(self, u)
        self._count += 1
        # ...
        y = u * self.gain
    end #stepImpl
end
```

```

        return y
    end #stepImpl

    # 释放资源函数：函数名固定，且只能有一个函数参数 self
    function releaseImpl(self)
        # ...
        return nothing
    end #releaseImpl

    # 其它自定义函数，第一个函数形参数必须是 self
    # function xx(self)
    #     ...
    # end
end

```

编辑 Julia 对象文件时，请注意以下几点

- 请勿删除注释标签，如# Description、# Methods 等。
- 请勿删除初始化函数 setupImpl(), 单步计算函数 stepImpl(), 释放资源函数 releaseImpl()。
- 类名需要具有唯一性，且与文件名一致。

属性

Julia 对象文件中属性分为公共属性 Parameter 和私有属性 Private。属性对应的规则如下表所示。

表 7-156 Syslab Object 属性规则

属性分类	特点	支持的数据类型
Parameter	Sysplorer 中可以直接访问	特定的 Julia 数据类型
Private	只能通过 Syslab Object 的方法来访问，用户不可见	符合 Julia 语法的所有数据类型

可以在 Parameter 参数中使用以下数据类型：

```

String | Bool | Int64 | Integer | Float64 | Real | Number
Vector{Int64} | Vector{Integer} | Vector{Float64} | Vector{Real} |
Vector{Number}
Matrix{Int64} | Matrix{Integer} | Matrix{Float64} | Matrix{Real} |
Matrix{Number}

```

方法

Julia 对象文件中包含有三个固定方法：

表 7-157 Syslab Object 属性规则

方法	说明
setupImpl	执行设置和初始化任务
stepImpl	系统输出和状态更新方程
releaseImpl	释放使用的资源和执行必要的清理任务

Syslab Object Block 的开发规范

可以按照下图所示的流程，在系统建模仿真环境中开发 Syslab Object Block。

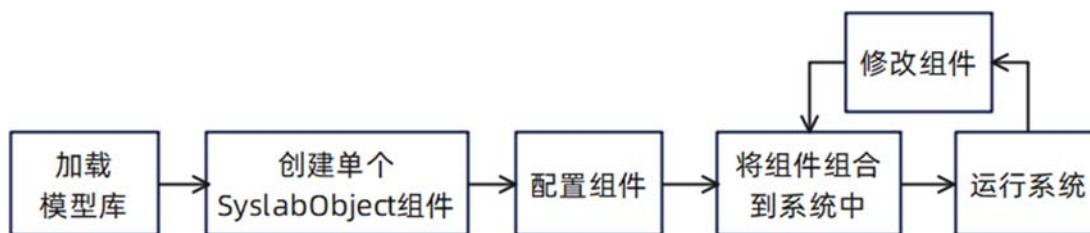


图 7-26 Syslab Object Block 开发流程

- 加载模型库：在 Syslab 中启动 Sysplorer，加载 SyslabWorkspace 模型库
- 创建单个组件：拖拽创建单个 SyslabObject 组件
- 配置组件：关联组件对应的 Julia 对象文件
 - 新建或编辑 SyslabObject 组件的 Julia 对象文件
 - 编辑 SyslabObject 组件的端口设置
- 将组件组合到系统中：编写一个包含这些 Syslab Object 的 Sysplorer 程序，完成其余部分模型的建模
- 运行系统：在 Sysplorer 中运行您的程序进行仿真
- 修改组件：对已关联 Julia 对象文件的组件进行修改

7.3.3 APP 开发规范

7.3.3.1 什么是 APP

APP 是带交互界面的应用程序，提供面向特定场景的专业应用，如控制系统设计与分析应用。APP 通常依赖函数库或模型库，具备 GUI 实现交互入口，通过专业算法调用底层函数。APP 作为专业工具，是基于平台层的基础能力之上，构建的面向特定使用场景的专业应用工具。

7.3.3.2 APP 的开发运行流程

APP 的开发运行流程包括以下过程：

- APP 开发：基于 APP 应用开发环境，编写代码开发 GUI 图形用户界面，开发具体的业务逻辑，实现与科学计算与系统建模仿真环境的数据交互，调用科学计算与系统建模仿真环境的函数库与模型库。
- APP 测试：APP 开发完成后的测试验证工作，包括开发者自测试和专业测试。
- APP 打包：APP 的代码开发完成后，将 APP 打包成独立可运行的程序（例如 exe 或 sh 脚本），对于依赖的动态链接库等文件也要一起打包。

- APP 安装：指 APP 打包好后，将 APP 安装和集成到科学计算与系统建模仿真环境中，实现 APP 的可查询、可运行、可管理。
- APP 使用：APP 安装成功后，可以在科学计算与系统建模仿真环境中使用统一的用法来使用 APP，包括查询 APP、启动 APP、使用 APP 等。

APP 的开发运行流程如下图所示。

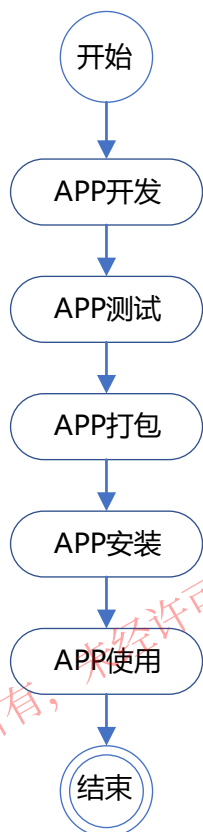


图 7-27 APP 的开发运行流程图

7.3.3.2.1 APP 开发

开发 APP 的具体过程包括编写代码开发 GUI 图形用户界面，开发具体的业务逻辑，实现与平台层的数据交互，调用平台层接口实现具体的业务逻辑。APP 的开发过程如下所示。

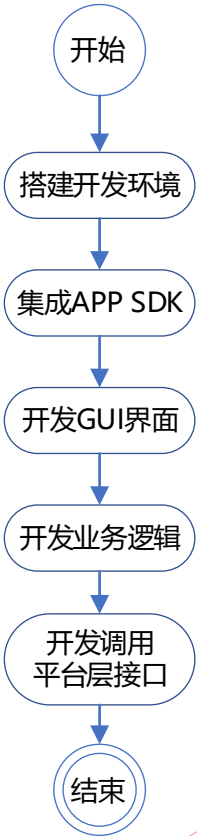


图 7-28 APP 开发流程图

7.3.3.2.2 APP 测试

APP 开发完成后的测试验证工作，包括开发者自测试和专业测试。本文侧重于开发者的自测试，包括 2 个测试场景：场景 1 为不依赖平台层通过打桩测试实现 APP 的独立测试，验证 APP 自身的功能；场景 2 为与平台层的集成联调测试。具体流程图如下所示。

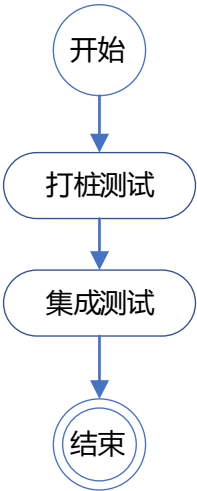


图 7-29 APP 测试流程图

7.3.3.2.3 APP 打包

APP 打包遵循具体 APP 开发环境的要求，打包好的 APP 程序需独立可运行，不需再另外安装软件或执行其他的操作。

7.3.3.2.4 APP 安装

APP 打包好后，将 APP 安装和集成到科学计算与系统建模仿真环境中，实现 APP 的可查询、可运行、可管理。APP 安装和卸载都是在科学计算与系统建模仿真环境中操作，APP 安装成功后才能在科学计算与系统建模仿真环境中使用。具体流程图如下所示。



图 7-30 APP 安装流程图

7.3.3.2.5 APP 使用

APP 安装成功后，可以在科学计算与系统建模仿真环境中查询到 APP 信息，用户可以在科学计算与系统建模仿真环境启动并使用 APP。具体流程图如下所示。

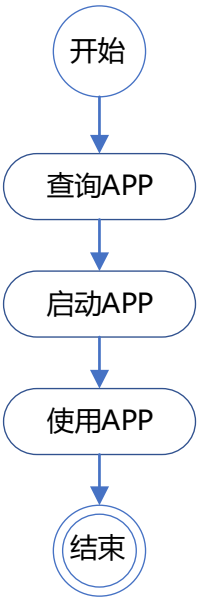


图 7-31 APP 使用流程图

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

附录 1：内核级开放接口

附录 1.1 科学计算环境数学算法替换用户手册

附录 1.2 系统建模仿真环境内核算法替换用户手册

附录 2：平台层开放接口

附录 2.1 科学计算环境平台层 API 详细说明

附录 2.2 系统建模仿真环境平台层 API 详细说明

附录 3：应用资源开发手册

附录 3.1 科学计算环境 APP 开发用户手册

附录 3.2 系统建模仿真环境 APP 开发用户手册

附录 3.3 模型库和函数库开发规范

苏州同元软件信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。